PROGRAMMING CONCEPTS

CHEAT SHEETS

A Complete Reference for Programming Concepts, Data Structures & Algorithm Patterns

Pseudocode · Problem Solving · OOP & Functional · Big O · Arrays & Hashing · Trees & Heaps · Graphs · Binary Search & Sorting · Two Pointers & Windows · Backtracking · Dynamic Programming · Greedy & Math

104 REFERENCE SHEETS · 16 SECTIONS

CONTENTS

FOUNDATIONS

001–004 Getting Started
005–013 Core Concepts
014–020 Paradigms & Design
021–026 Complexity Analysis
027–035 Linear Data Structures

STRUCTURES & PATTERNS

036–044 Trees & Heaps
045–055 Graphs
056–061 Searching & Sorting
062–071 Array & String Patterns
072–077 Recursion & Backtracking

ADVANCED TECHNIQUES

078–088 Dynamic Programming
089–090 Greedy Algorithms
091–095 Advanced Data Structures & Design
096–100 Math & Number Theory
101–102 String Algorithms
103–104 Quick Reference

Table of Contents

104 sheets · 16 sections

GETTING STARTED 001 – 004

001 Using This Set
002 Pseudocode Convent…
003 Problem-Solving Fra…
004 Edge Cases & Dry Ru…

CORE CONCEPTS 005 – 013

005 Variables, Values & Ty…
006 Numbers & Arithmetic
007 Bitwise Operations (P…
008 Booleans & Condition…
009 Loops & Iteration
010 Functions & Paramet…
011 Scope & Closures
012 Memory: Stack, Heap …
013 Error Handling & Defe…

PARADIGMS & DESIGN 014 – 020

014 Programming Paradig…
015 Functional Programm…
016 Iterators, Generators …
017 OOP: Classes, Object…
018 OOP: Inheritance, Pol…
019 SOLID & Design Patte…
020 Concurrency Basics

COMPLEXITY ANALYSIS 021 – 026

021 Big O Notation
022 Common Complexity …
023 Analyzing Loops & Co…
024 Space Complexity & …
025 Amortized Analysis &…
026 Constraints → Target …

LINEAR DATA STRUCTURES 027 – 035

027 Arrays & Dynamic Arr…
028 Strings: Characters &…
029 Regular Expressions (…
030 Linked Lists — Basics
031 Linked Lists — Techni…
032 Stacks
033 Queues & Deques
034 Hash Maps
035 Hash Sets & Counting…

TREES & HEAPS 036 – 044

036 Tree Terminology & Pr…
037 Binary Tree Traversals
038 Binary Tree Recursion…
039 Binary Search Trees
040 Self-Balancing Trees
041 Tree Construction & …
042 Lowest Common Anc…
043 Heaps & Priority Que…
044 Heap Patterns: Top-K,…

GRAPHS 045 – 055

045 Graph Basics & Repre…
046 Depth-First Search
047 Breadth-First Search
048 Grid Graphs
049 Topological Sort
050 Union-Find (Disjoint …
051 Shortest Paths: Dijkst…
052 Bellman-Ford, Floyd-…
053 Minimum Spanning Tr…
054 Bipartite Graphs & C…
055 Connectivity: SCCs, B…

SEARCHING & SORTING 056 – 061

056 Binary Search Funda…
057 Binary Search Variati…
058 Elementary Sorts
059 Merge Sort & Quick S…
060 Non-Comparison Sor…
061 Custom Sorting & Qui…

ARRAY & STRING PATTERNS 062 – 071

062 Two Pointers
063 Sliding Window — Fix…
064 Sliding Window — Var…
065 Prefix Sums & Differe…
066 Kadane's Algorithm & …
067 Monotonic Stack
068 Monotonic Deque
069 Intervals & Sweep Line
070 Matrix Traversal & Ma…
071 Cyclic Sort & In-Plac…

RECURSION & BACKTRACKING 072 – 077

072 Recursion Fundamen…
073 Divide & Conquer
074 Backtracking Template
075 Subsets & Combinati…
076 Permutations
077 Board & Grid Backtra…

DYNAMIC PROGRAMMING 078 – 088

078 DP Fundamentals & …
079 Tabulation & Space O…
080 1D DP Patterns
081 2D Grid DP
082 Knapsack Family
083 String DP
084 Longest Increasing Su…
085 Interval DP
086 State Machine DP
087 Tree & DAG DP
088 Bitmask DP

GREEDY ALGORITHMS 089 – 090

089 Greedy Fundamentals
090 Greedy Patterns

ADVANCED DATA STRUCTURES & DESIGN 091 – 095

091 Tries (Prefix Trees)
092 Segment Trees
093 Fenwick Trees (Binary…
094 LRU & LFU Caches
095 Data Structure Desig…

MATH & NUMBER THEORY 096 – 100

096 Number Theory & Dig…
097 Modular Arithmetic &…
098 Combinatorics & Cou…
099 Randomized Algorith…
100 Computational Geom…

STRING ALGORITHMS 101 – 102

101 Linear-Time String Al…
102 Rolling Hash & Rabin-…

QUICK REFERENCE 103 – 104

103 Pattern Recognition …
104 Algorithm Templates …

Using This Set

scope · pseudocode-first · sheet layout · study order · Blind 75 / NeetCode 150 mapping · pattern groups

WHAT THIS SET COVERS

A **language-agnostic** reference for the ideas every programmer needs and the algorithm patterns behind coding-interview sites like LeetCode and NeetCode. Every code block is **pseudocode** (see Sheet 002), so the same page works whether you solve in Python, Java, JavaScript, C++, or anything else. Translate the pseudocode into your language as practice. Pattern sheets follow one layout: **idea → pseudocode → complexity (time · space) → recognize-it-when signals → practice problems**. Bitwise (007) and RegEx (029) are one-page primers with their own full sets.

SUGGESTED STUDY ORDER

STEP	SHEETS
1. Foundations	002, 003, 021–023
2. Core structures	027–035
3. Array patterns	062–066, 056
4. Trees & graphs	036–039, 045–049
5. Recursion & DP	072, 074, 078–082

THE 16 GROUPS

SHEETS	GROUP	YOU LEARN
001–004	Getting Started	conventions, problem-solving process
005–013	Core Concepts	types, loops, functions, memory, errors
014–020	Paradigms & Design	OOP, functional, SOLID, concurrency
021–026	Complexity	Big O, analysis, input-size targets
027–035	Linear Structures	arrays, strings, lists, stacks, hashing
036–044	Trees & Heaps	traversals, BSTs, heaps, top-k
045–055	Graphs	BFS/DFS, topo sort, shortest paths
056–061	Search & Sort	binary search, sorting algorithms
062–071	Array Patterns	two pointers, windows, prefix sums
072–077	Backtracking	recursion, subsets, permutations
078–088	Dynamic Prog.	memo, tabulation, DP families
089–090	Greedy	exchange argument, scheduling
091–095	Advanced DS	tries, segment/Fenwick trees, LRU
096–100	Math	number theory, mod, combinatorics
101–102	String Algos	KMP, Z, Manacher, rolling hash
103–104	Quick Reference	pattern finder, templates

NEETCODE ROADMAP → SHEETS

ROADMAP TOPIC	SHEETS
Arrays & Hashing	027, 028, 034, 035
Two Pointers	062
Sliding Window	063, 064
Stack	032, 067
Binary Search	056, 057
Linked List	030, 031
Trees	036–042
Tries	091
Heap / Priority Queue	043, 044
Backtracking	074–077
Graphs	045–050, 054
Advanced Graphs	051–053, 055
1-D Dynamic Programming	078–080
2-D Dynamic Programming	081–086
Greedy	089, 090
Intervals	069
Math & Geometry	070, 096–100
Bit Manipulation	007, 088

⚠ COMMON MISTAKES

- Reading pseudocode without coding it: you only learn a pattern by **typing it in your own language** and running it
- Jumping to DP before arrays, hashing, and recursion: most DP is recursion plus a cache
- Memorizing problem answers instead of patterns: new problems reuse patterns, not answers

✓ TIPS & BEST PRACTICES

- Work one roadmap topic at a time; read the sheet, then solve 3–5 problems
- Use Sheet 103 when you're stuck on **which** technique applies
- Re-solve a problem a week later from a blank editor to test recall

QUICK SUMMARY

- Pseudocode — one notation, any language (Sheet 002)
- Groups 1–4 — concepts and Big O
- Groups 5–15 — data structures and patterns
- Group 16 — quick lookup and templates

Pseudocode Conventions

THE HOUSE STYLE

Every code block in this set uses one small, consistent pseudocode. It reads like plain English but is precise enough to translate line by line into a real language. **Indentation** marks blocks, `←` assigns, and `=` compares, so there is never confusion between the two.

WHY PSEUDOCODE

- Shows the **idea**, not language syntax
- Arrays are **0-indexed**
- `for … to …` ranges are **inclusive**

A COMPLETE EXAMPLE

```
// Find target in a sorted array. Returns index or -1.
function binarySearch(arr, target)
    lo ← 0
    hi ← length(arr) - 1
    while lo ≤ hi
        mid ← lo + (hi - lo) div 2
        if arr[mid] = target
            return mid
        else if arr[mid] < target
            lo ← mid + 1
        else
            hi ← mid - 1
    return -1

for i ← 0 to n - 1              // i = 0, 1, …, n-1
for i ← n - 1 down to 0         // counts backward
for each x in items             // values, not indexes
(a, b) ← (b, a)                 // swap via tuple
```

CODE EXPLAINED

- → `function name(params)` — defines a function; body is indented
- → `←` — assignment; `=` is always a comparison
- → `div` — integer division (`7 div 2` is `3`)
- → `length(arr)` — number of elements
- → `while` / `if` / `else` — no colons or `end`
- → `(a, b)` — tuple: return or unpack several values

OPERATORS

SYMBOL	MEANING
`←`	assign
`= / ≠`	equal / not equal
`< ≤ > ≥`	comparisons
`+ - * /`	arithmetic (`/` is real division)
`div / mod`	integer quotient / remainder
`and or not`	boolean logic (short-circuit)
`& | ^ ~ << >>`	bitwise (Sheet 007)

COLLECTIONS

OPERATION	MEANING
`new Array(n, 0)`	n slots filled with 0
`list.append(x)`	add to end
`map[key] ← v`	insert or update
`map.has(key)`	key present?
`stack.push / pop / peek`	LIFO
`queue.enqueue / dequeue`	FIFO
`heap.push / pop / peek`	min-heap unless noted

OTHER CONVENTIONS

FORM	MEANING
`null`	no value / empty pointer
`∞ / -∞`	beyond any value
`// text`	comment
`arr[i][j]`	row i, column j
`s[i]`	character at index i
`s[i..j]`	substring i through j inclusive

⚠ COMMON MISTAKES

- Reading `=` as assignment: here it **only** compares
- Treating `0 to n` as exclusive: it includes `n`, so write `n - 1`
- Translating `div` as JavaScript `/` : use `Math.floor(a / b)`

✓ TIPS & BEST PRACTICES

- Map `heap` to your language's priority queue (`heapq` , `PriorityQueue`)
- Write `lo + (hi - lo) div 2` everywhere; it never overflows
- Keep indentation strict in interview pseudocode

QUICK SUMMARY

- `←` — assign · `=` — compare
- `div` / `mod` — integer quotient / remainder
- `for i ← a to b` — inclusive range
- `(a, b)` — return or unpack multiple values

Problem-Solving Framework

UMPIRE · clarify inputs · brute force first · optimize · state the tradeoffs · write test cases · talk through the plan

A REPEATABLE PROCESS

Most failed interview answers come from **coding too soon**. A fixed process keeps you from solving the wrong problem. **UMPIRE** (Understand, Match, Plan, Implement, Review, Evaluate) is a common checklist: clarify first, name a pattern, plan in pseudocode, then code and test.

GROUND RULES

- Say a **brute-force** answer out loud first, with its complexity
- Read the **constraints**; they hint at the target Big O (Sheet 026)
- Plan in pseudocode or comments **before** writing real code
- Test with a small example by hand, not just by running

THE PROCESS AS PSEUDOCODE

```
function solve(problem)
    // U — Understand
    restate the problem in your own words
    ask: input size? sorted? duplicates? negatives? empty?
    write 2-3 examples, including an edge case
    // M — Match
    pattern ← recognizeSignals(problem)      // Sheet 103
    // P — Plan
    write brute force + its Big O
    improve: which work is repeated? can a structure remove it?
    outline the steps in pseudocode
    // I — Implement
    code the plan, one helper at a time
    // R — Review
    trace your example through the code (Sheet 004)
    // E — Evaluate
    state final time · space and any tradeoffs
```

CODE EXPLAINED

- → **Understand** — clarify inputs, outputs, and limits
- → **Match** — map signals to a known pattern
- → **Plan** — brute force first, then optimize
- → **Implement** — small functions, clear names
- → **Review** — dry-run before you hit Run
- → **Evaluate** — Big O and what you'd do with more time

CLARIFYING QUESTIONS

ASK ABOUT	WHY IT MATTERS
`input size`	sets the target complexity
`sorted?`	enables binary search / two pointers
`duplicates?`	changes counting and dedup logic
`negatives / zero?`	breaks some window and product tricks
`empty input?`	what to return
`in place?`	whether extra memory is allowed
`one answer or all?`	search vs. enumerate (backtracking)

OPTIMIZATION MOVES

BOTTLENECK	TRY
`repeated lookups`	hash map / set (O(1))
`nested loop on sorted data`	two pointers / binary search
`recomputed range sums`	prefix sums
`recomputed subproblems`	memoization / DP
`need min/max repeatedly`	heap
`checking every subarray`	sliding window
`exploring all choices`	backtracking + pruning

⚠ COMMON MISTAKES

- Starting to code before confirming the output format: you solve a different problem
- Skipping brute force: you lose an easy partial answer and a baseline to optimize
- Optimizing silently: interviewers grade your **reasoning**, so narrate tradeoffs

✓ TIPS & BEST PRACTICES

- Write the examples as test cases you can run at the end
- If stuck for two minutes, list the data structures you know and ask which one removes the slow step
- End by naming what you'd improve with more time

QUICK SUMMARY

- `Understand` — clarify before coding
- `Match` — signals → pattern
- `Plan` — brute force, then better
- `Review` — dry-run, then state Big O

Edge Cases & Dry Runs

BREAK IT BEFORE THEY DO

Correct code for the normal case still fails on **edge cases**: inputs at the boundaries of what the problem allows. A **dry run** (tracing code by hand with a small input, writing each variable in a table) catches off-by-one errors and wrong loop conditions before the computer does.

WHEN TO CHECK

- After planning: list edge cases next to your examples
- After coding: trace one normal and one edge case
- Off-by-one bugs live at **loop bounds** and **slice ends**
- Overflow lives in **sums, products, and midpoints**

DRY RUN: TRACE TABLE

```
// Sum of positive numbers
function sumPositive(arr)
    total ← 0
    for i ← 0 to length(arr) - 1
        if arr[i] > 0
            total ← total + arr[i]
    return total

// Trace with arr = [3, -1, 4]
//  i | arr[i] | arr[i] > 0 | total
//  - |   -    |     -      |   0
//  0 |   3    |    true    |   3
//  1 |  -1    |    false   |   3
//  2 |   4    |    true    |   7
// return 7  ✓

// Edge: arr = [] → "0 to -1" is empty, returns 0 ✓
```

OFF-BY-ONE CHECKS

CHECK	ASK YOURSELF
`first iteration`	is index 0 handled?
`last iteration`	is `n - 1` reached, and not `n`?
`empty range`	does `0 to -1` run zero times?
`< vs ≤`	should the equal case run?
`window size`	is it `r - l + 1`?
`mid point`	`lo + (hi - lo) div 2`

EDGE-CASE CHECKLIST

CATEGORY	CASES TO TRY
`size`	empty, one element, two elements, max size
`values`	zero, negatives, all equal, min/max integer
`order`	already sorted, reverse sorted, alternating
`duplicates`	all duplicates, none, duplicates at ends
`strings`	empty string, one char, spaces, mixed case, Unicode
`trees`	null root, single node, skewed (linked-list shape)
`graphs`	no edges, self-loop, disconnected, cycle

⚠ COMMON MISTAKES

- Testing only the example from the prompt: it was chosen to be easy
- Returning a default like `0` for empty input without checking the spec: it may want `-1` or an error
- Forgetting `null` children in trees: most crashes are `node.left.val` on a leaf

✓ TIPS & BEST PRACTICES

- Keep a personal list of edge cases you missed; it becomes your checklist
- Trace with the **smallest** input that exercises every branch
- Guard clauses (`if arr is empty return …`) at the top keep the main logic clean

QUICK SUMMARY

- `Edge cases` — boundaries of the allowed input
- `Trace table` — one row per iteration
- `Off-by-one` — check loop starts, ends, and `<` vs `≤`
- `Overflow` — sums, products, midpoints

Variables, Values & Types

primitive vs composite · static vs dynamic typing · strong vs weak · constants · literals · type conversion · null

NAMES, VALUES, AND TYPES

A **variable** is a name bound to a value. Every value has a **type** that decides what operations make sense (`+` on numbers adds, on strings joins). Languages differ in **when** types are checked (static: at compile time; dynamic: at run time) and **how strictly** (strong: no silent conversions; weak: converts for you).

KEY IDEAS

- **Primitive**: a single value (int, float, bool, char)
- **Composite**: built from other values (array, map, object)
- **Constant**: a name that can't be rebound
- `null` means "no value"; using it like a value crashes

DECLARING AND CONVERTING

```
count ← 0                        // integer
ratio ← 0.75                     // floating point
done ← false                     // boolean
initial ← 'A'                    // character
name ← "Ada"                     // string
const MAX_SIZE ← 100             // constant: can't be reassigned
items ← [1, 2, 3]                // array / list
point ← (3, 4)                   // tuple: fixed-size group
user ← { "name": "Ada", "age": 36 }   // map / record
nothing ← null                   // absence of a value

toInt("42")                      // 42
toFloat(7)                       // 7.0
toString(3.5)                    // "3.5"
truncate(4.9)                    // 4
toInt("4.9")                     // error in most languages
"5" + 3                          // "53" (JS)  ·  error (Python)
```

CODE EXPLAINED

- → `const` — binding is fixed; the object it points to may still change
- → `(3, 4)` — tuple: ordered, usually immutable
- → `{ … }` — map: key → value pairs
- → `toInt` / `toFloat` — **explicit** conversion (cast)
- → `"5" + 3` — **implicit** conversion: depends on the language

TYPING SYSTEMS

KIND	MEANING	EXAMPLES
`static`	types checked before running	Java, C#, Go, Rust, TS
`dynamic`	types checked while running	Python, JS, Ruby
`strong`	few silent conversions	Python, Java, Rust
`weak`	converts freely	JavaScript, PHP, C
`inferred`	compiler deduces the type	`var` (Java), `let` (Rust)

COMMON TYPES

TYPE	HOLDS
`int`	whole numbers (fixed or big)
`float / double`	real numbers, approximate
`bool`	true / false
`char`	one character / code point
`string`	sequence of characters
`array / list`	ordered values by index
`map / dict`	key → value lookup

⚠ COMMON MISTAKES

- Assuming `"5" + 3` means the same everywhere: it's `"53"` in JS and an error in Python
- Comparing values of different types (`"1" = 1`): false or surprising
- Using a `null` variable before assigning it: null-reference crash

✓ TIPS & BEST PRACTICES

- Convert input text to numbers once, at the edge of your program
- Use constants for magic numbers (`MAX_SIZE`) so intent is clear
- In static languages, let types document the code; in dynamic ones, add type hints

QUICK SUMMARY

- `variable` — name bound to a value
- `static / dynamic` — when types are checked
- `strong / weak` — how freely types convert
- `toInt(x)` — explicit conversion beats implicit

Sheet 006

Numbers & Arithmetic

integer division · mod with negatives · overflow · 32/64-bit limits · floating-point error · epsilon compare · rounding

COMPUTER ARITHMETIC IS NOT SCHOOL MATH

Integers have **fixed sizes** and can **overflow**; floats are **binary approximations**, so `0.1 + 0.2` isn't exactly `0.3`. Integer division and `mod` round differently across languages when negatives are involved. Knowing these rules prevents a whole class of wrong answers.

RULES OF THUMB

- Sums of many 32-bit values may need **64-bit**
- Never compare floats with `=`; use a tolerance
- Use integer math when exact answers are required
- Fix negative `mod` with `((a mod m) + m) mod m`

ARITHMETIC BEHAVIOR

```
7 div 2                  // 3
-7 div 2                 // -4 (Python: floor)  ·  -3 (Java/C/JS: truncate)
7 mod 3                  // 1
-7 mod 3                 // 2 (Python)  ·  -1 (Java/C/JS)
((a mod m) + m) mod m    // always in 0..m-1

(a + b - 1) div b        // ceil(a / b) for positive ints
0.1 + 0.2                // 0.30000000000000004
abs(a - b) < 1e-9        // float "equality" with tolerance

2147483647 + 1           // -2147483648 (32-bit overflow wraps)
mid ← lo + (hi - lo) div 2   // avoids overflow of lo + hi

floor(-2.5)              // -3
ceil(-2.5)               // -2
truncate(-2.5)           // -2   (toward zero)
round(2.5)               // 2 (Python, banker's)  ·  3 (JS Math.round)
```

CODE EXPLAINED

- → `div` — rounding direction matters for negatives
- → `mod` — sign of result follows the divisor (Python) or dividend (Java/C/JS)
- → `(a + b - 1) div b` — integer ceiling without floats
- → `1e-9` — epsilon; scale it to the size of your values
- → `lo + (hi - lo) div 2` — safe midpoint

INTEGER LIMITS

TYPE	RANGE
`int8`	−128 … 127
`int16`	−32,768 … 32,767
`int32`	±2.1 × 10^9 (2^{31} − 1)
`uint32`	0 … 4.29 × 10^9
`int64`	±9.2 × 10^{18} (2^{63} − 1)
`float64`	≈15–17 digits; exact ints to 2^{53}

WHERE OVERFLOW HIDES

EXPRESSION	FIX
`sum of n values`	64-bit accumulator
`a * b`	64-bit, or mod each step
`(lo + hi) / 2`	`lo + (hi - lo) div 2`
`abs(INT_MIN)`	has no 32-bit positive
`factorials`	big ints or `mod` 10^9+7
`reverse integer`	check before multiplying by 10

⚠ COMMON MISTAKES

- Translating Python's `//` and `%` into Java/C for negatives: results differ
- Using floats for money: rounding errors accumulate; use integer cents
- Checking overflow **after** it happens: the wrapped value already lies

✓ TIPS & BEST PRACTICES

- When a problem says "return modulo 10^9+7", the true answer overflows; reduce at every step
- JavaScript numbers are all float64: integers above 2^{53} lose precision (use `BigInt`)
- Python ints never overflow, but your translated Java solution will

QUICK SUMMARY

- `div` / `mod` — check negative behavior
- `0.1 + 0.2` — floats are approximate
- `int32` — about 2.1 billion
- `lo + (hi - lo) div 2` — overflow-safe midpoint

Sheet 007

Bitwise Operations (Primer)

AND / OR / XOR / NOT · shifts · x & 1 odd test · x & (x-1) · x & -x · XOR cancel · bit masks · see Bitwise set

WORKING WITH INDIVIDUAL BITS

Integers are stored in binary, and **bitwise operators** work on each bit position independently. They power fast parity checks, subsets-as-integers (bitmasks), and tricks like XOR-cancellation. This is a one-page primer; the **Bitwise set** covers two's complement, masks, and bit-DP in depth.

NOTATION

- Bitwise uses C-style symbols: `&` `|` `^` `~` `<<` `>>`
- `and` / `or` stay **boolean**; `&` / `|` are **bitwise**
- Bit `i` is worth `2^i`; bit 0 is the rightmost
- Negative numbers use **two's complement**: `-x = ~x + 1`

OPERATORS AND CLASSIC TRICKS

```
// x = 12 = 1100₂   y = 10 = 1010₂
x & y            // 1000₂ = 8    AND: both bits set
x | y            // 1110₂ = 14   OR: either bit set
x ^ y            // 0110₂ = 6    XOR: bits differ
~x               // -13          NOT: flip every bit
x << 1           // 11000₂ = 24  shift left = × 2
x >> 2           // 11₂ = 3      shift right = div 4

(x >> i) & 1     // read bit i
x | (1 << i)     // set bit i
x & ~(1 << i)    // clear bit i
x ^ (1 << i)     // toggle bit i
x & 1 = 1        // x is odd
x & (x - 1)      // drop lowest set bit · = 0 → power of 2
x & -x           // keep only lowest set bit
a ^ a = 0        // XOR cancels pairs: XOR of all = the single one
```

CODE EXPLAINED

- → `&` — mask: keep only selected bits
- → `|` — turn bits on
- → `^` — toggle; `a ^ b ^ b = a`
- → `<<` / `>>` — fast × / ÷ by powers of 2
- → `x & (x - 1)` — used to count set bits
- → `x & -x` — lowbit (Fenwick trees, Sheet 093)

TRUTH TABLE

A	B	A & B	A | B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

WHERE IT SHOWS UP

PROBLEM	TRICK
Single Number	XOR everything
Number of 1 Bits	loop `x ← x & (x - 1)`
Power of Two	`x > 0 and x & (x - 1) = 0`
Counting Bits	`bits[i] = bits[i >> 1] + (i & 1)`
Subsets	masks `0` … $2^n - 1$ (Sheet 075)
Missing Number	XOR indexes with values

⚠ COMMON MISTAKES

- Precedence: `x & 1 = 1` parses as `x & (1 = 1)` in C/Java/JS; add parentheses
- Right-shifting negatives: arithmetic `>>` keeps the sign; use `>>>` (Java/JS) for logical shift
- Shifting by ≥ word size (e.g. `1 << 32` on int32) is undefined or wraps

✓ TIPS & BEST PRACTICES

- Write binary in comments when debugging (1100_2)
- Use `1L << i` (Java) or 64-bit types for masks above bit 30
- Full coverage: see the **Bitwise** cheat sheet set

QUICK SUMMARY

- `&` `|` `^` `~` — AND, OR, XOR, NOT
- `x & (x - 1)` — clear lowest set bit
- `x & -x` — isolate lowest set bit
- `a ^ a = 0` — XOR cancels pairs

Sheet 008

Booleans & Conditionals

truth tables · short-circuit · De Morgan's laws · if / else if / else · switch / match · ternary · guard clauses

MAKING DECISIONS

Conditionals choose which code runs based on **boolean** expressions. `and` / `or` **short-circuit**: the right side runs only if it can change the result, which makes guards like `i < n and arr[i] > 0` safe. Prefer flat code with **guard clauses** over deeply nested `if` blocks.

QUICK FACTS

- `a and b` stops at the first false
- `a or b` stops at the first true
- `match` / `switch` picks one branch by value
- Some languages treat `0`, `""`, `[]`, `null` as **falsy**

CONDITIONAL FORMS

```
if age ≥ 18 and hasId
    allow()
else if age ≥ 16
    allowWithGuardian()
else
    deny()

if i < length(arr) and arr[i] > 0   // index checked first: safe

label ← if score ≥ 50 then "pass" else "fail"   // ternary

match command                       // switch / match
    case "start": begin()
    case "stop", "halt": finish()
    default: unknown(command)

function ship(order)                // guard clauses
    if order = null return "no order"
    if order.items is empty return "nothing to ship"
    return sendPackage(order)       // main path, not nested
```

CODE EXPLAINED

→ `else if` — checked only if earlier tests failed

→ short-circuit — guards protect the right side

→ `if … then … else` — expression that returns a value

→ `match` — compare one value against many cases

→ guard clause — return early for bad cases

TRUTH TABLE

A	B	A AND B	A OR B	NOT A
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

DE MORGAN'S LAWS

EXPRESSION	EQUALS
`not (a and b)`	`(not a) or (not b)`
`not (a or b)`	`(not a) and (not b)`
`not (x < 5)`	`x ≥ 5`
`not (lo ≤ x ≤ hi)`	`x < lo or x > hi`

FALSY VALUES

VALUE	FALSY IN
`0`, `0.0`	Python, JS, C
`""`	Python, JS
`[]` / `{}`	Python only
`null` / `None`	Python, JS
`NaN`	JS

⚠ COMMON MISTAKES

- Writing `if x = 5` in C/JS: that's assignment; use `==` (or `===` in JS)
- Reversing guard order `arr[i] > 0 and i < n`: crashes before the bounds check
- Forgetting `break` in C/Java/JS `switch`: execution falls through to the next case

✓ TIPS & BEST PRACTICES

- Negate a condition with De Morgan instead of wrapping it in `not (…)`
- Order `else if` chains from most specific to most general
- In JS, `[]` and `{}` are **truthy**; check `.length` explicitly

QUICK SUMMARY

- `and` / `or` — short-circuit left to right
- `if … then … else` — conditional expression
- `match` — multi-way branch
- Guard clause — exit early, keep the happy path flat

Loops & Iteration

counted loop · for each · while · do-while · break / continue · nested loops · off-by-one · loop invariants

REPEATING WORK

Loops repeat a block while some condition holds. Pick the loop that matches the job: **counted** when you need the index, **for-each** for values, **while** when the end depends on what happens inside. A **loop invariant** (something true before and after every iteration) is the tool for proving a loop correct.

QUICK FACTS

- `for i ← 0 to n - 1` runs **n** times
- Nested loops multiply: n × m iterations
- `break` exits the loop; `continue` skips to the next pass
- Don't add or remove items from a collection while iterating it

LOOP FORMS

```
for i ← 0 to n - 1              // counted: need the index
    print(i, arr[i])

for each x in arr               // for-each: just the values
    total ← total + x

while lo < hi                   // condition checked before each pass
    lo ← lo + 1

repeat                          // do-while: body runs at least once
    line ← readLine()
until line = "quit"

for i ← 0 to rows - 1           // nested: rows × cols passes
    for j ← 0 to cols - 1
        grid[i][j] ← 0

for each x in arr
    if x < 0 continue           // skip negatives
    if x = target break         // stop at target
```

CODE EXPLAINED

- → `to` — inclusive end, so `n - 1` for n items
- → `for each` — no index; use counted loop when you need one
- → `while` — may run zero times
- → `repeat … until` — runs at least once
- → `continue` / `break` — skip one pass / leave loop

WHICH LOOP?

LOOP	USE WHEN
`for i ← a to b`	you need the index or a fixed count
`for each`	you only need each value
`while`	end depends on work inside the loop
`repeat … until`	body must run once (menus, input)
`down to`	iterate backward (delete-safe, DP)

LOOP INVARIANT EXAMPLE

POINT	WHAT'S TRUE
`before loop`	`total` = sum of first 0 items
`after pass i`	`total` = sum of `arr[0..i]`
`loop ends`	`total` = sum of all items ✓
`binary search`	target, if present, is in `lo..hi`

⚠ COMMON MISTAKES

- Running to `n` instead of `n - 1`: index out of range on the last pass
- `while` loop that never changes its condition variable: infinite loop
- Removing items while iterating forward: skips the item after each removal

✓ TIPS & BEST PRACTICES

- Iterate **backward** when deleting by index
- Name loop variables for meaning (`row` , `col`) in nested loops
- Write the invariant as a comment for any tricky `while` loop

QUICK SUMMARY

- `for i ← 0 to n - 1` — n passes
- `for each` — values only
- `while` / `repeat` — condition-driven
- `break` / `continue` — exit / skip

Functions & Parameters

define / call · return values · pass by value vs reference · default params · pure vs side effects · call stack frames

REUSABLE BLOCKS OF LOGIC

A **function** names a piece of logic, takes **parameters**, and **returns** a value. Each call gets its own **stack frame** holding its parameters and locals, which is why recursion works. Whether a function can change the caller's data depends on how arguments are passed.

VOCABULARY

- **Parameter**: the name in the definition
- **Argument**: the value passed in a call
- **Pure**: same input → same output, no side effects
- **Side effect**: changes something outside the function

DEFINING AND CALLING

```
function area(width, height ← 1)      // height has a default
    return width * height
area(3, 4)                            // 12
area(5)                               // 5

function minMax(arr)                  // return several values
    return (min(arr), max(arr))
(lo, hi) ← minMax([4, 1, 9])          // lo = 1, hi = 9

function addOne(n)                    // n is a copy of the value
    n ← n + 1
x ← 5
addOne(x)                             // x is still 5

function addItem(list)                // list refers to caller's object
    list.append(99)
a ← [1]
addItem(a)                            // a is now [1, 99]
```

CODE EXPLAINED

- → `height ← 1` — default used when argument is omitted
- → `return (a, b)` — tuple return, unpacked by caller
- → numbers — copied, so the caller can't see changes
- → objects — the **reference** is copied; mutations are shared
- → reassigning a parameter never changes the caller's variable

ARGUMENT PASSING

MODEL	BEHAVIOR	LANGUAGES
`by value`	callee gets a copy	C, Go, primitives everywhere
`by reference`	callee can rebind caller's var	C++ `&`, C# `ref`
`by sharing`	copy of a reference; mutation shared	Python, Java, JS objects

PURE VS IMPURE

FUNCTION	PURE?
`square(x)`	yes
`sort(list) in place`	no: mutates input
`now()` / `random()`	no: output varies
`print(x)`	no: I/O side effect

⚠ COMMON MISTAKES

- Using a mutable default (`list ← []`) in Python: the same list is shared across calls
- Expecting `n ← n + 1` inside a function to change the caller's number
- Forgetting `return` : the caller gets `null` / `None` / `undefined`

✓ TIPS & BEST PRACTICES

- Keep functions small: one job, a name that says what it returns
- Prefer returning new values over mutating arguments unless performance needs it
- In interviews, write helper functions for each step; it reads like the plan

QUICK SUMMARY

- `function f(a, b ← 1)` — params with default
- `return (x, y)` — multiple results
- `by sharing` — mutations visible, rebinding not
- `pure` — easier to test and reason about

Scope & Closures

local vs global · block vs function scope · lexical scope · shadowing · closures · captured variables · loop-closure pitfall

WHERE NAMES ARE VISIBLE

Scope decides where a variable can be read. Most languages use **lexical scope**: a function sees the variables of the code it was **written inside**. A **closure** is a function plus the variables it captured from that enclosing scope; they stay alive as long as the function does.

SCOPE LEVELS

- **Global**: visible everywhere in the file/program
- **Function**: visible inside one function call
- **Block**: visible inside `{ }` / an indented block (not in Python)
- **Enclosing**: outer function's locals, seen by inner functions

SCOPE, SHADOWING, CLOSURES

```
total ← 0                          // global
function addToTotal(x)
    total ← total + x              // Python needs `global total`

x ← 1
function shadow()
    x ← 2                          // new local x shadows the global
    return x                       // 2; global x is still 1

function makeCounter()
    count ← 0                      // captured by the closure below
    function next()
        count ← count + 1          // Python needs `nonlocal count`
        return count
    return next

c ← makeCounter()
c()                                // 1
c()                                // 2  (count survives between calls)
d ← makeCounter()
d()                                // 1  (each closure has its own count)
```

CODE EXPLAINED

- → global — readable everywhere; writing often needs a keyword
- → shadowing — inner name hides the outer one
- → closure — `next` keeps `count` alive after `makeCounter` returns
- → each call to `makeCounter` creates a fresh `count`
- → closures power callbacks, memoization, and private state

SCOPE RULES BY LANGUAGE

LANGUAGE	BLOCK SCOPE?	WRITE OUTER VAR
Python	no (function only)	`global` / `nonlocal`
JavaScript	`let` / `const` yes · `var` no	just assign
Java / C#	yes	fields; lambdas need effectively-final
C / C++	yes	C++ lambdas: `[&]` capture

LOOP-CLOSURE PITFALL

CODE	CALLS PRINT
`JS var i + callbacks`	3, 3, 3
`JS let i + callbacks`	0, 1, 2
`Python lambda: i in loop`	2, 2, 2
`Python lambda i=i: i`	0, 1, 2

⚠ COMMON MISTAKES

- Assuming a Python `if` block creates a new scope: it doesn't
- Closures in a loop capturing the **variable**, not its value at that time
- Accidentally shadowing a built-in (`list` , `max` , `sum`) with a local name

✓ TIPS & BEST PRACTICES

- Keep variables in the smallest scope that works
- Pass values as parameters instead of reading globals; functions get easier to test
- Use closures for small private state (counters, memo caches)

QUICK SUMMARY

- `lexical scope` — decided by where code is written
- `shadowing` — inner name hides outer
- `closure` — function + captured variables
- `nonlocal` / `global` — Python's write-outer keywords

Sheet 012

Memory: Stack, Heap & References

call stack · heap allocation · references / pointers · aliasing · shallow vs deep copy · garbage collection · stack overflow

WHERE VALUES LIVE

Each function call gets a **stack frame** for its locals; frames are freed automatically when the call returns. Objects created with `new` live on the **heap** and outlive the call that made them. Variables that hold objects actually hold **references**, so two names can point at the same object (**aliasing**).

KEY FACTS

- The stack is small (often ~1–8 MB): deep recursion overflows it
- Heap memory is reclaimed by GC, ownership rules, or manual `free`
- Assignment copies the **reference**, not the object
- Python's default recursion limit is 1000 frames

REFERENCES, ALIASES, COPIES

```
function build(n)                  // one stack frame per call
    size ← n * 2                   // local value: lives in the frame
    node ← new Node(size)          // object on the heap; frame holds a reference
    return node                    // frame is freed; the heap object survives

a ← [1, 2, 3]
b ← a                              // alias: same array
b[0] ← 99                          // a[0] is 99 too

c ← copy(a)                        // shallow copy: new outer array
c[1] ← 7                           // a unchanged

grid ← [[0, 0], [0, 0]]
g2 ← copy(grid)                    // shallow: rows are still shared
g2[0][0] ← 5                       // grid[0][0] is 5 too!
g3 ← deepCopy(grid)                // fully independent

rows ← new Array(3, [])            // trap: 3 references to ONE list
for i ← 0 to 2
    rows[i] ← []                   // fix: a separate list per slot
```

CODE EXPLAINED

- → stack frame — params + locals for one call
- → `new` — allocates on the heap
- → `b ← a` — aliasing, not copying
- → shallow copy — copies the outer container only
- → deep copy — copies nested objects too
- → `new Array(3, [])` — shares one inner list

STACK VS HEAP

	STACK	HEAP
holds	frames, locals, return address	objects, arrays
lifetime	until the call returns	until unreachable / freed
size	small, fixed	large, grows
speed	very fast (pointer bump)	slower allocation
failure	stack overflow	out of memory / leak

MEMORY MANAGEMENT

APPROACH	LANGUAGES
manual `malloc` / `free`	C
RAII / ownership	C++, Rust
tracing GC	Java, C#, Go, JS
ref counting + cycle GC	Python
ARC (auto ref counting)	Swift

⚠ COMMON MISTAKES

- Building a 2D grid as `[[0] * m] * n` (Python): every row is the same list
- "Copying" with `b ← a` then mutating `b`: you changed `a`
- Recursing 10^5 deep in Python or Java: stack overflow; use an explicit stack

✓ TIPS & BEST PRACTICES

- Draw boxes (objects) and arrows (references) when debugging aliasing
- Return new objects from helpers instead of mutating shared ones
- Convert deep recursion to iteration with your own stack (Sheet 072)

QUICK SUMMARY

- `stack` — per-call frames, auto-freed
- `heap` — objects that outlive calls
- `b ← a` — alias, same object
- `deepCopy` — independent nested copy

Error Handling & Defensive Coding

try / catch / finally · throw · error codes vs Result types · assertions · input validation · fail fast · sentinel values

HANDLING WHAT GOES WRONG

Errors are inevitable: bad input, missing files, broken invariants. **Exceptions** jump out of normal flow to the nearest matching handler; **error values** (Go, Rust) make the caller check explicitly. Either way, **fail fast**: detect bad state as early as possible and report it clearly.

PRINCIPLES

- Validate at the **boundary** (user input, files, network)
- Catch only errors you can actually handle
- `finally` runs whether or not an error occurred
- `assert` checks **your** assumptions, not user input

THROWING, CATCHING, RETURNING ERRORS

```
function parseAge(text)
    if text is empty
        throw ValueError("age is required")   // fail fast
    n ← toInt(text)
    if n < 0 or n > 150
        throw ValueError("age out of range")
    return n

try
    age ← parseAge(input)
catch ValueError as e
    print("bad input: " + e.message)        // handle and recover
finally
    closeFile()                             // always runs

(value, err) ← parseAgeResult(input)        // error as a value
if err ≠ null
    return err                              // caller must check

idx ← indexOf(arr, target)                  // sentinel: -1 = not found
assert lo ≤ hi                              // internal invariant
```

CODE EXPLAINED

- → `throw` — stop and signal an error
- → `catch` — handle a specific error type
- → `finally` — cleanup that always runs
- → `(value, err)` — Go / Rust `Result` style
- → sentinel — special return like `-1` or `null`
- → `assert` — crash loudly if a bug breaks an invariant

ERROR STRATEGIES

STRATEGY	BEST FOR	WATCH OUT FOR
`exceptions`	rare, unrecoverable-here errors	hidden control flow
`Result / error value`	expected failures (parse, I/O)	verbose checks
`sentinel value`	"not found" in searches	caller forgets to check
`Optional / Maybe`	value may be absent	unwrapping without check
`assertions`	programmer bugs	often disabled in production

⚠ COMMON MISTAKES

- Catching every exception and ignoring it: hides bugs and corrupts state
- Using exceptions for normal control flow (e.g. ending a loop): slow and unclear
- Returning `-1` when `-1` is a valid answer: sentinel collides with real data

✓ TIPS & BEST PRACTICES

- Throw specific error types with messages that say **what** and **why**
- Release resources with `finally` or `using` / `with` / `defer`
- In interviews, state input assumptions instead of over-validating

QUICK SUMMARY

- `throw` / `catch` / `finally` — exception flow
- `(value, err)` — errors as values
- `-1` / `null` — sentinel results
- `assert` — guard internal invariants

Programming Paradigms

imperative · procedural · object-oriented · functional · declarative · event-driven · multi-paradigm languages

STYLES OF PROGRAMMING

A **paradigm** is a way of organizing code. **Imperative** code says *how*: step-by-step statements that change state. **Declarative** code says *what*: describe the result and let the language work out the steps. Most modern languages are **multi-paradigm**, so you pick the style that fits each problem.

QUICK FACTS

- Procedural = imperative code organized into functions
- OOP bundles data and the functions that act on it
- Functional avoids mutation; functions are values
- SQL, HTML, and regex are declarative

ONE TASK, SEVERAL PARADIGMS

```
// Task: sum the squares of the even numbers in nums

// Imperative / procedural: HOW, step by step
total ← 0
for each n in nums
    if n mod 2 = 0
        total ← total + n * n

// Functional: compose transformations, no mutation
total ← sum(map(filter(nums, isEven), square))

// Declarative (SQL): describe the result
SELECT SUM(n * n) FROM nums WHERE n % 2 = 0

// Object-oriented: data + behavior in one object
total ← numberList.sumOfEvenSquares()

// Event-driven: code runs when something happens
button.onClick(lambda: show(total))
```

CODE EXPLAINED

- → imperative — explicit loop and a changing `total`
- → functional — `filter` → `map` → `sum` pipeline
- → declarative — no loop at all; engine decides
- → OOP — the object owns the logic
- → event-driven — a callback waits for an event

PARADIGMS AT A GLANCE

PARADIGM	CORE IDEA	TYPICAL LANGUAGES
`imperative`	statements change state	C, assembly
`procedural`	imperative + functions	C, Pascal, Go
`object-oriented`	objects with state + methods	Java, C#, C++
`functional`	pure functions, immutability	Haskell, Elixir, Clojure
`declarative`	describe the result	SQL, HTML, Prolog
`event-driven`	react to events / callbacks	JS in the browser, GUIs

MULTI-PARADIGM LANGUAGES

LANGUAGE	MIXES
`Python`	procedural, OOP, functional
`JavaScript`	event-driven, functional, OOP
`Java / C#`	OOP + streams / LINQ
`Rust`	procedural, functional
`Kotlin / Scala`	OOP + functional

⚠ COMMON MISTAKES

- Forcing every problem into classes: a 10-line script doesn't need a hierarchy
- Mixing mutation into a `map` callback: hard to read and breaks parallel use
- Thinking declarative means slower: SQL engines often beat hand-written loops

✓ TIPS & BEST PRACTICES

- Use imperative loops when you need early exit or index tricks
- Use functional pipelines for clear transform-filter-aggregate steps
- Interview solutions are usually imperative; know the functional form too

QUICK SUMMARY

- `imperative` — how, with changing state
- `declarative` — what, not how
- `OOP` — data + behavior together
- `functional` — pure functions and pipelines

Functional Programming

pure functions · immutability · first-class functions · higher-order functions · map / filter / reduce · composition · currying

FUNCTIONS AS VALUES

Functional programming treats functions as **first-class** values: store them in variables, pass them in, return them. Code is built from **pure functions** that don't mutate data, so results depend only on inputs. **Higher-order functions** like `map`, `filter`, and `reduce` replace most hand-written loops.

CORE TERMS

- **Pure**: no side effects, same output for same input
- **Immutable**: data is copied, never changed in place
- **Higher-order**: takes or returns a function
- **Currying**: `f(a, b)` becomes `f(a)(b)`

THE TOOLKIT

```
square ← lambda x: x * x                    // function stored in a variable
nums ← [1, 2, 3, 4, 5]
map(nums, square)                           // [1, 4, 9, 16, 25]
filter(nums, lambda x: x mod 2 = 0)         // [2, 4]
reduce(nums, lambda acc, x: acc + x, 0)     // 15

function compose(f, g)                      // returns a new function
    return lambda x: f(g(x))
inc ← lambda x: x + 1
compose(square, inc)(3)                     // square(inc(3)) = 16

function add(a)                             // curried
    return lambda b: a + b
add5 ← add(5)
add5(10)                                    // 15

sortBy(people, lambda p: p.age)             // pass behavior as an argument
raised ← map(salaries, lambda s: s * 1.1)   // new list; salaries unchanged
```

CODE EXPLAINED

- → `lambda x: …` — anonymous function
- → `map` — transform every item
- → `filter` — keep items that pass a test
- → `reduce` — fold items into one value (needs a start value)
- → `compose` — chain functions right to left
- → `add(5)` — partial application via currying

MORE HIGHER-ORDER HELPERS

FUNCTION	RESULT
`flatMap(xs, f)`	map, then flatten one level
`zip(a, b)`	pairs `(a[i], b[i])`
`any(xs, p)` / `all(xs, p)`	true if some / every item passes
`groupBy(xs, key)`	map of key → items
`takeWhile(xs, p)`	prefix while test passes

LANGUAGE EQUIVALENTS

LANGUAGE	MAP · FILTER · REDUCE
Python	`map` `filter` `functools.reduce`, comprehensions
JavaScript	`arr.map` `.filter` `.reduce`
Java	`stream().map().filter().reduce()`
C#	LINQ `Select` `Where` `Aggregate`
Kotlin	`map` `filter` `fold`

⚠ COMMON MISTAKES

- `reduce` without a start value on an empty list: error or `undefined`
- Side effects inside `map` (printing, pushing to an outer list): use a loop instead
- Chaining many passes over huge data: each step allocates a new list

✓ TIPS & BEST PRACTICES

- Prefer comprehensions in Python: `[x * x for x in nums if x % 2 == 0]`
- Name lambdas that grow past one line; readability beats cleverness
- Immutability makes concurrent code safer: nothing to lock

QUICK SUMMARY

- `map` / `filter` / `reduce` — transform, select, fold
- `lambda` — inline function value
- `compose(f, g)` — f after g
- `pure` — no side effects, easy to test

Sheet 016

Iterators, Generators & Lazy Evaluation

iterator protocol · hasNext / next · yield · lazy evaluation · infinite sequences · peeking iterator · flatten nested iterator

PRODUCING VALUES ONE AT A TIME

An **iterator** hands out a sequence one item at a time through `hasNext()` / `next()`, without exposing how it's stored. A **generator** is a function that **yields** values and pauses between them, so items are computed only when asked for (**lazy evaluation**). Laziness makes huge or even infinite sequences cheap.

QUICK FACTS

- `for each` loops use an iterator under the hood
- A generator keeps its local state between `yield`s
- Lazy = O(1) memory for a stream of any length
- Most iterators can be consumed only **once**

ITERATORS AND GENERATORS

```
it ← iterator([10, 20, 30])          // the iterator protocol
while it.hasNext()
    print(it.next())                 // 10, 20, 30

function countUpTo(n)                // generator: pauses at each yield
    for i ← 1 to n
        yield i
for each x in countUpTo(3)           // 1, 2, 3, made on demand
    print(x)

function naturals()                  // infinite, safe because lazy
    n ← 0
    while true
        yield n
        n ← n + 1
take(naturals(), 5)                  // [0, 1, 2, 3, 4]
```

CODE EXPLAINED

→ `hasNext` / `next` — the two-method contract

→ `yield` — return a value and pause

→ next call resumes right after the `yield`

→ `take` — stops pulling, so the loop ends

PEEKING ITERATOR (LC 284)

```
class PeekingIterator
    function init(it)
        this.it ← it
        this.buf ← this.advance()      // look one item ahead
    function advance()
        return if this.it.hasNext() then this.it.next() else null
    function peek()
        return this.buf
    function next()
        val ← this.buf
        this.buf ← this.advance()
        return val
    function hasNext()
        return this.buf ≠ null
```

EAGER VS LAZY

	EAGER	LAZY
builds	whole list now	one item per request
memory	O(n)	O(1)
infinite data	impossible	fine
reuse	iterate many times	usually once
Python	`[x for …]`	`(x for …)`

⚠ COMMON MISTAKES

- Iterating a generator twice: the second pass is empty
- Calling `next()` without checking `hasNext()`: exception at the end
- Turning an infinite generator into a list: never finishes

✓ TIPS & BEST PRACTICES

- Use a one-item **buffer** for peek/lookahead problems
- Flatten Nested List Iterator: keep a **stack** of iterators
- Stream large files line by line with a generator

QUICK SUMMARY

- `hasNext()` / `next()` — iterator protocol
- `yield` — generator pauses here
- `lazy` — compute on demand
- `buffer` — enables `peek()`

OOP: Classes, Objects & Encapsulation

BUNDLING DATA WITH BEHAVIOR

A **class** is a blueprint; an **object** (instance) is one thing built from it. The **constructor** sets up each new object's **fields**, and **methods** act on them through `this` / `self`. **Encapsulation** hides the fields and exposes only safe operations, so objects can't be put into invalid states.

VOCABULARY

- **Field** / attribute: data stored in each object
- **Method**: function that belongs to the class
- **Static**: belongs to the class, shared by all instances
- **Getter / setter**: controlled access to a field

A COMPLETE CLASS

```
class BankAccount
    static count ← 0                        // shared by every instance
    private balance                         // hidden field

    function init(owner, opening ← 0)       // constructor
        this.owner ← owner
        this.balance ← opening
        BankAccount.count ← BankAccount.count + 1

    function deposit(amount)                // method guards the invariant
        if amount ≤ 0
            throw ValueError("amount must be positive")
        this.balance ← this.balance + amount

    function getBalance()                   // getter: read-only access
        return this.balance

acct ← new BankAccount("Ada", 100)          // create an instance
acct.deposit(50)
acct.getBalance()                           // 150
acct.balance ← -5                           // error: balance is private
BankAccount.count                           // 1
```

CODE EXPLAINED

- → `class` — blueprint for objects
- → `init` — constructor runs on `new`
- → `this` — the current object (`self` in Python)
- → `private` — only the class's own methods can touch it
- → `static` — one copy for the whole class
- → methods validate input so state stays valid

ACCESS LEVELS

MODIFIER	VISIBLE TO	PYTHON / JS
`public`	everyone	default
`protected`	class + subclasses	`_name` (convention)
`private`	the class only	`__name` / `#name`
`package / internal`	same package / assembly	—

INSTANCE VS STATIC

	INSTANCE	STATIC
`belongs to`	one object	the class
`access`	`obj.x`	`Class.x`
`uses this`	yes	no
`example`	`acct.deposit()`	`Math.max()`

⚠ COMMON MISTAKES

- Making every field public, then validating everywhere it's changed
- Forgetting `this.` / `self.`: assigns a local variable instead of the field
- Storing per-object data in a static field: every instance shares it

✓ TIPS & BEST PRACTICES

- Keep invariants (e.g. balance ≥ 0) inside the class's methods
- Add getters freely; add setters only when outside code must change a field
- In interviews, a small class per entity (Node, LRUCache) keeps code clear

QUICK SUMMARY

- `class` / `new` — blueprint / instance
- `init` — constructor sets fields
- `private` — encapsulation
- `static` — shared across instances

Sheet 018

OOP: Inheritance, Polymorphism & Interfaces

extends · override · super · abstract class · interface · dynamic dispatch · composition over inheritance

REUSING AND SUBSTITUTING BEHAVIOR

Inheritance lets a subclass reuse and **override** a parent's behavior (IS-A). **Polymorphism** means code written for the parent type works with any subclass: the right method is chosen at run time (**dynamic dispatch**). **Interfaces** define a contract with no implementation; **abstract classes** provide a partial one.

KEY IDEAS

- `extends` — inherit from a class
- `implements` — promise to provide an interface's methods
- `super` — call the parent's version
- Prefer **composition** (HAS-A) for code reuse

INTERFACES, OVERRIDING, DISPATCH

```
interface Shape                          // contract: WHAT, not HOW
    function area()

abstract class BaseShape implements Shape
    function describe()                  // shared, uses abstract area()
        return "area = " + toString(this.area())

class Circle extends BaseShape
    function init(r)
        this.r ← r
    function area()                      // override
        return 3.14159 * this.r * this.r

class Square extends BaseShape
    function init(side)
        this.side ← side
    function area()
        return this.side * this.side

for each s in [new Circle(1), new Square(2)]
    print(s.describe())                  // area = 3.14159 · area = 4
```

CODE EXPLAINED

- → `interface` — methods only, no state
- → `abstract class` — can't be instantiated; shares code
- → override — subclass replaces a method
- → dispatch — `s.area()` runs the real object's version
- → `describe` is written once and works for every shape

ABSTRACT CLASS VS INTERFACE

	ABSTRACT CLASS	INTERFACE
`fields / state`	yes	no (constants only)
`method bodies`	some	none (or defaults)
`inherit how many`	one	many
`use for`	shared base code	capability (`Comparable`)

INHERITANCE VS COMPOSITION

	INHERITANCE	COMPOSITION
`relation`	IS-A (Circle is a Shape)	HAS-A (Car has an Engine)
`coupling`	tight to parent	loose
`change at run time`	no	swap the part
`risk`	fragile deep hierarchies	more delegation code

⚠ COMMON MISTAKES

- Inheriting just to reuse a helper: creates a false IS-A relation
- Overriding with a different signature: you overloaded, not overrode
- Hierarchies deeper than 2–3 levels: changes ripple unpredictably

✓ TIPS & BEST PRACTICES

- Mark intended overrides (`@Override` , `override`) so the compiler checks them
- Program to interfaces: accept `Shape` , not `Circle`
- Reach for composition first; inherit when the IS-A test truly holds

QUICK SUMMARY

- `extends` — inherit and override
- `interface` — pure contract
- `dynamic dispatch` — run-time method choice
- `composition` — build from parts

SOLID & Design Patterns

SRP · OCP · LSP · ISP · DIP · factory · strategy · observer

PRINCIPLES AND REUSABLE SHAPES

SOLID is five guidelines for object-oriented code that stays easy to change. **Design patterns** are named, reusable solutions to common design problems. Both aim at the same thing: depend on **abstractions**, keep each piece small, and add features by adding code instead of editing working code.

WHEN THEY MATTER

- System-design and OOP-design interview rounds
- Code that many people extend over years
- Overkill for a 30-line script or a LeetCode answer
- Patterns are vocabulary: say "use a strategy"

STRATEGY, FACTORY, OBSERVER

```
interface Discount                      // Strategy: swappable behavior
    function apply(price)
class NoDiscount implements Discount
    function apply(price)
        return price
class Sale implements Discount
    function apply(price)
        return price * 0.8
function checkout(cart, discount)       // depends on the interface (DIP)
    return discount.apply(sum(cart))

function makeDiscount(code)             // Factory: one place picks the class
    if code = "SALE" return new Sale()
    return new NoDiscount()

class Store                             // Observer: notify subscribers
    listeners ← []
    function subscribe(fn)
        listeners.append(fn)
    function publish(event)
        for each fn in listeners
            fn(event)
```

CODE EXPLAINED

- → strategy — new discount = new class, no edits (OCP)
- → `checkout` never names a concrete class
- → factory — hides construction decisions
- → observer — publisher doesn't know its subscribers

SOLID

LETTER	PRINCIPLE	IN ONE LINE
S	Single Responsibility	one reason to change per class
O	Open/Closed	extend with new code, don't edit old
L	Liskov Substitution	subclasses work wherever the parent does
I	Interface Segregation	small, focused interfaces
D	Dependency Inversion	depend on abstractions, not concretions

COMMON PATTERNS

PATTERN	INTENT
`factory`	create objects without naming the class
`singleton`	exactly one shared instance
`strategy`	swap algorithms at run time
`observer`	publish events to subscribers
`decorator`	wrap to add behavior
`adapter`	make one interface fit another

⚠ COMMON MISTAKES

- Applying patterns before there's a need: extra layers with no payoff
- Singletons as hidden globals: hard to test and to reason about
- A Square subclass of Rectangle that breaks `setWidth`: violates Liskov

✓ TIPS & BEST PRACTICES

- Refactor toward a pattern when the second or third variant appears
- Inject dependencies through constructors so tests can swap them
- In design interviews, name the pattern and the principle it serves

QUICK SUMMARY

- `SOLID` — five design guidelines
- `strategy` — pluggable behavior
- `factory` — centralized creation
- `observer` — event subscribers

Sheet 020

Concurrency Basics

threads vs processes · race condition · mutex / lock · semaphore · deadlock · async / await · producer-consumer

DOING SEVERAL THINGS AT ONCE

Concurrency is structuring a program as tasks that overlap in time; **parallelism** is actually running them at the same instant on multiple cores. When tasks share mutable data, unsynchronized access causes **race conditions**. **Locks** make a section exclusive, but taking locks in inconsistent order causes **deadlock**.

VOCABULARY

- **Process**: own memory; **thread**: shares process memory
- **Critical section**: code touching shared state
- **Mutex**: one holder at a time; **semaphore**: up to N
- **async/await**: overlap waiting (I/O) on one thread

RACES, LOCKS, ASYNC

```
counter ← 0
lock ← new Mutex()

function worker()
    repeat 1000 times
        lock.acquire()
        counter ← counter + 1          // read-modify-write: not atomic
        lock.release()

t1 ← startThread(worker)
t2 ← startThread(worker)
join(t1)
join(t2)                               // 2000 with the lock; often less without

// Deadlock: T1 takes A then B, T2 takes B then A → both wait forever
// Fix: every thread acquires locks in the same global order

async function loadPage()
    userTask ← fetch("/user")          // both requests start now
    postsTask ← fetch("/posts")
    return (await userTask, await postsTask)
```

CODE EXPLAINED

- → race — two threads interleave `read → add → write`
- → mutex — only one thread in the critical section
- → `join` — wait for a thread to finish
- → lock ordering — the standard deadlock fix
- → `await` — pause this task, not the whole thread

SYNCHRONIZATION TOOLS

TOOL	USE FOR
`mutex / lock`	exclusive access to shared data
`semaphore(n)`	limit to n concurrent users
`condition variable`	wait until a condition is true
`atomic variable`	lock-free counters and flags
`blocking queue`	producer-consumer hand-off

CLASSIC PROBLEMS

PROBLEM	USUAL TOOL
`producer-consumer`	bounded blocking queue
`readers-writers`	read/write lock
`dining philosophers`	lock ordering
`LC Print in Order`	semaphores / events
`LC FooBar Alternately`	two semaphores

⚠ COMMON MISTAKES

- Assuming `counter ← counter + 1` is one step: it's read, add, write
- Holding a lock while doing slow I/O: other threads stall
- Python threads for CPU-bound work: the GIL blocks real parallelism (use processes)

✓ TIPS & BEST PRACTICES

- Share as little mutable state as possible; pass messages instead
- Use `async` for many slow network calls, threads/processes for CPU work
- Release locks in `finally` (or `with` / `using`) so errors can't leak them

QUICK SUMMARY

- `race condition` — unsynchronized shared writes
- `mutex` / `semaphore` — 1 / N at a time
- `deadlock` — circular wait; fix with lock order
- `async` / `await` — overlap waiting

Big O Notation

O / Ω / Θ · upper bound · drop constants · dominant term · best / average / worst case · input size n

MEASURING HOW CODE SCALES

Big O describes how an algorithm's running time (or memory) **grows** as the input size **n** grows. It ignores constant factors and small terms, and keeps only the **dominant term**, because for large n that term is all that matters. It's the shared language for comparing solutions before you write them.

GROUND RULES

- n = input size (array length, node count, string length)
- Big O is usually quoted for the **worst case**
- Drop constants: O(3n) → O(n)
- Keep the biggest term: O(n² + n) → O(n²)

COUNTING STEPS

```
function first(arr)                 // O(1): same work for any n
    return arr[0]

function contains(arr, x)           // O(n): may look at every item
    for each v in arr
        if v = x return true
    return false

function hasDuplicate(arr)          // O(n²): checks every pair
    n ← length(arr)
    for i ← 0 to n - 1
        for j ← i + 1 to n - 1
            if arr[i] = arr[j] return true
    return false

// Exact count:  T(n) = 3n² + 5n + 20
// Drop constants and lower terms  →  O(n²)
```

CODE EXPLAINED

- → `O(1)` — constant: independent of n
- → `O(n)` — linear: one pass
- → `O(n²)` — quadratic: all pairs, about n²/2 checks
- → early `return` helps the best case, not the worst
- → the **growth rate** matters, not the exact count

NOTATION

SYMBOL	MEANS	THINK
`O(f)`	upper bound	at most
`Ω(f)`	lower bound	at least
`Θ(f)`	tight bound (both)	exactly, up to constants
`o(f)`	strictly smaller growth	much less than

BEST · AVERAGE · WORST

ALGORITHM	BEST	AVG	WORST
`linear search`	O(1)	O(n)	O(n)
`binary search`	O(1)	O(log n)	O(log n)
`hash lookup`	O(1)	O(1)	O(n)
`quicksort`	O(n log n)	O(n log n)	O(n²)
`insertion sort`	O(n)	O(n²)	O(n²)

SIMPLIFY

RAW	BIG O
`O(2n + 10)`	O(n)
`O(n²/2)`	O(n²)
`O(n + n²)`	O(n²)
`O(log₂ n)`	O(log n)
`O(n + m)`	O(n + m)

⚠ COMMON MISTAKES

- Saying O(n) is "fast": with n = 10^9 and a heavy loop body it's still seconds
- Merging different inputs: O(n + m) is not O(n) unless m ≤ n is guaranteed
- Confusing best case with Big O: "O(1) if the first item matches" isn't the bound

✓ TIPS & BEST PRACTICES

- State complexity in terms of named inputs: O(rows × cols), O(V + E)
- Quote time **and** space in every interview answer
- Mention average vs worst when they differ (hashing, quicksort)

QUICK SUMMARY

- `O` — upper bound on growth
- Drop constants and lower-order terms
- `Θ` — tight bound
- Worst case unless stated otherwise

Common Complexity Classes

O(1) · O(log n) · O(n) · O(n log n) · O(n²) · O(2ⁿ) · O(n!) · growth chart

THE COMPLEXITY LADDER

A handful of growth rates cover almost every algorithm. Each step up the ladder is dramatically slower for large inputs: at n = 1,000, O(n log n) is about 10,000 steps while O(n²) is a million and O(2ⁿ) is beyond astronomical. Learn to recognize each class from the **shape** of the code.

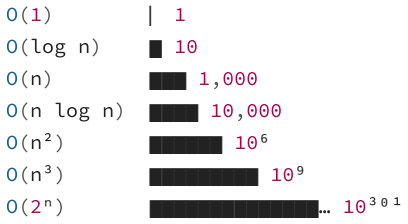
SHAPES TO RECOGNIZE

- Halving the problem each step → **log n**
- One pass over the data → **n**
- Sort, or divide-and-conquer with linear merge → **n log n**
- Every subset / every ordering → **2ⁿ** / **n!**

TYPICAL CODE PER CLASS

```
x ← arr[i]                          // O(1)        index, hash lookup
binarySearch(sorted, t)             // O(log n)    halves the range
for each x in arr                   // O(n)        single pass
sort(arr)                           // O(n log n)  comparison sort
for i …  for j …                    // O(n²)       all pairs
for i …  for j …  for k …            // O(n³)       all triples
subsets(items)                      // O(2ⁿ)       include / exclude each
permutations(items)                 // O(n!)       every ordering
```

GROWTH AT N = 1,000 (LOG SCALE)



APPROXIMATE OPERATION COUNTS

CLASS	N = 10	N = 100	N = 10³	N = 10⁶	NAME
O(1)	1	1	1	1	constant
O(log n)	3	7	10	20	logarithmic
O(√n)	3	10	32	1,000	square root
O(n)	10	100	10^3	10^6	linear
O(n log n)	33	664	$\approx 10^4$	$\approx 2 \times 10^7$	linearithmic
O(n²)	100	10^4	10^6	10^{12}	quadratic
O(n³)	10^3	10^6	10^9	10^{18}	cubic
O(2ⁿ)	1,024	$\approx 10^{30}$	$\approx 10^{301}$	—	exponential
O(n!)	3.6×10^6	$\approx 10^{158}$	—	—	factorial

⚠ COMMON MISTAKES

- Treating O(log n) and O(n) as "both fast": at 10^9 that's 30 steps vs a billion
- Calling 2^n "polynomial": exponential grows faster than any n^k
- Forgetting the log base doesn't matter: $\log_2$ and $\log_{10}$ differ by a constant

✓ TIPS & BEST PRACTICES

- Rough feel: $\log_2(10^6) \approx 20$, $\log_2(10^9) \approx 30$
- √n appears in prime checks and block decomposition
- Pair this sheet with 026 to pick a target class from the constraints

QUICK SUMMARY

- `log n` — halving
- `n log n` — sorting
- `n²` — pairs
- `2ⁿ` / `n!` — subsets / orderings

Sheet 023

Analyzing Loops & Code

sequential = add · nested = multiply · halving loop = log n · O(n + m) vs O(n·m) · hidden costs (slice, concat, contains)

READING COMPLEXITY FROM CODE

To find Big O, look at how many times the **innermost** work runs. Blocks that run **one after another add**; loops **inside** loops **multiply**. Then watch for **hidden costs**: library calls such as `insert`, slicing, and string `+` can be O(n) each and quietly turn a loop into O(n²).

RULES

- Sequential → add, keep the biggest term
- Nested → multiply the iteration counts
- Loop variable × or ÷ by a constant → log n
- Different inputs keep different letters

PATTERNS

```
for i ← 0 to n - 1                  // O(n)
    work()
for j ← 0 to n - 1                  // + O(n)  → sequential: O(n)
    work()

for i ← 0 to n - 1                  // nested → O(n · n) = O(n²)
    for j ← i + 1 to n - 1          // triangle n(n-1)/2 is still O(n²)
        work()
i ← n
while i > 1                         // halving → O(log n)
    i ← i div 2
for i ← 1 to n                      // inner loop doubles → O(n log n)
    j ← 1
    while j < n
        j ← j * 2

s ← ""
for each ch in text                 // hidden cost: + copies s each time
    s ← s + ch                      // → O(n²); use a string builder
```

CODE EXPLAINED

- → sequential blocks — add
- → nested loops — multiply
- → `i div 2` / `j * 2` — logarithmic loops
- → two inputs A, B — O(|A| · |B|), not n²
- → immutable string `+` — O(length) per append

HIDDEN COSTS

OPERATION	COST
`x in list`	O(n)
`x in set` / `map.has(k)`	O(1) average
`list.insert(0, x)` / `pop(0)`	O(n)
`deque` add / remove at ends	O(1)
slice `arr[a..b]`	O(b - a)
string `s + t`	O(|s| + |t|)
`sort(arr)`	O(n log n)

LOOP SHAPES

LOOP	RUNS
`i ← i + 1 to n`	n
`i ← i * 2 to n`	log n
`i * i ≤ n`	√n
two pointers moving inward	n total
sliding window (l and r only advance)	2n → O(n)
recursion halving + linear merge	n log n

⚠ COMMON MISTAKES

- Calling a two-pointer or sliding-window loop O(n²) because it has a nested `while`
- Ignoring the cost of `contains` on a list inside a loop: O(n²) total
- Using `n` for both dimensions of a grid: it's O(rows × cols)

✓ TIPS & BEST PRACTICES

- Count how many times each pointer **moves in total**, not per outer step
- Swap list lookups for a set when checking membership in a loop
- Build strings with a list + `join` or a builder

QUICK SUMMARY

- Sequential — add
- Nested — multiply
- `×2` / `÷2` — log n
- Library calls have costs too

Sheet 024

Space Complexity & Recursive Analysis

auxiliary vs total space · recursion depth · recursion tree · branches^depth · Master Theorem · memoization effect

MEMORY AND RECURSIVE COST

Space complexity counts extra memory as n grows. **Auxiliary** space is what your algorithm adds; **total** space also counts the input. Recursion uses hidden space: one stack frame per active call, so **depth** = space. For time, draw the **recursion tree**: work ≈ (number of calls) × (work per call).

QUICK RULES

- Recursion space = maximum **depth** × frame size
- Recursion time ≈ **branches^depth** without memoization
- Memoization: time ≈ (distinct states) × (work per state)
- Output arrays usually don't count as auxiliary space

ANALYZING RECURSION

```
function sumTo(n)                  // 1 call per level, depth n
    if n = 0 return 0              // time O(n), stack space O(n)
    return n + sumTo(n - 1)

function fib(n)                    // 2 branches, depth n
    if n ≤ 1 return n              // time O(2^n), space O(n)
    return fib(n - 1) + fib(n - 2)

memo ← new Map()
function fibMemo(n)                // n distinct states, O(1) each
    if n ≤ 1 return n              // time O(n), space O(n)
    if memo.has(n) return memo[n]
    memo[n] ← fibMemo(n - 1) + fibMemo(n - 2)
    return memo[n]

function binarySearch(lo, hi)      // 1 branch, depth log n
    …                              // T(n) = T(n/2) + O(1) → O(log n)

function mergeSort(arr)            // 2 branches on n/2, O(n) merge
    …                              // T(n) = 2T(n/2) + O(n) → O(n log n)
```

CODE EXPLAINED

- → depth — sets stack space
- → `fib` — tree of ~2^n calls; most repeat
- → memo — collapses the tree to n states
- → `T(n) = …` — recurrence; solve with the table below
- → merge sort also needs O(n) extra for merging

MASTER THEOREM: T(N) = A·T(N/B) + O(N^D)

CASE	RESULT
`d > log_b a`	$O(n^d)$
`d = log_b a`	$O(n^d \log n)$
`d < log_b a`	O(n^(log_b a))

COMMON RECURRENCES

RECURRENCE	EXAMPLE	BIG O
`T(n/2) + O(1)`	binary search	O(log n)
`T(n - 1) + O(1)`	linear recursion	O(n)
`2T(n/2) + O(1)`	tree traversal	O(n)
`2T(n/2) + O(n)`	merge sort	O(n log n)
`T(n - 1) + O(n)`	selection sort	$O(n^2)$
`2T(n - 1) + O(1)`	naive fib, subsets	$O(2^n)$

⚠ COMMON MISTAKES

- Calling recursive DFS O(1) space: the call stack is O(depth), up to O(n)
- Using the Master Theorem on `T(n - 1)` recurrences: it only applies to `T(n/b)`
- Forgetting memo table space: memoized DP still uses O(states) memory

✓ TIPS & BEST PRACTICES

- Draw the first three levels of the recursion tree; the pattern usually shows
- Balanced tree recursion is O(log n) space; skewed is O(n)
- Convert to an iterative loop to cut stack space to O(1) when possible

QUICK SUMMARY

- `space` — depth of recursion counts
- `branches^depth` — naive recursion time
- `states × work` — memoized time
- `2T(n/2)+O(n)` — O(n log n)

Amortized Analysis & Cost Tables

dynamic array doubling · amortized O(1) · hash map worst case · data structure ops table · sort / search table

AVERAGE COST OVER MANY OPERATIONS

Amortized analysis spreads the cost of a rare expensive step over many cheap ones. A dynamic array occasionally copies everything when it grows, but because it **doubles**, n appends cost under 3n total: **O(1) amortized** each. The tables below are the cost reference used throughout the rest of the set.

AMORTIZED VS AVERAGE

- **Amortized**: guaranteed over any sequence of operations
- **Average case**: expected over random inputs (hashing)
- Growing by a constant (+10) instead of ×2 → O(n) per append
- Hash maps resize too: O(1) amortized insert

DYNAMIC ARRAY APPEND

```
function append(x)
    if size = capacity
        capacity ← capacity * 2          // rare: O(n) copy
        newData ← new Array(capacity)
        for i ← 0 to size − 1
            newData[i] ← data[i]
        data ← newData
    data[size] ← x                       // usual: O(1)
    size ← size + 1

// n appends copy 1 + 2 + 4 + … + n < 2n items
// total O(n)  →  O(1) amortized per append
```

ALGORITHM COSTS

ALGORITHM	TIME	SPACE
binary search	O(log n)	O(1)
merge sort	O(n log n)	O(n)
quicksort	O(n log n) avg · O(n²)	O(log n)
heap sort	O(n log n)	O(1)
counting sort	O(n + k)	O(k)
BFS / DFS	O(V + E)	O(V)
Dijkstra (heap)	O((V + E) log V)	O(V)
union-find (both opts)	≈O(1) per op	O(n)

DATA STRUCTURE OPERATIONS

STRUCTURE	ACCESS	SEARCH	INSERT	DELETE	NOTES
array	O(1)	O(n)	O(n)	O(n)	append O(1) amortized
linked list	O(n)	O(n)	O(1)*	O(1)*	*at a known node
stack / queue / deque	O(1) ends	O(n)	O(1)	O(1)	ends only
hash map / set	—	O(1) avg	O(1) avg	O(1) avg	O(n) worst
balanced BST	O(log n)	O(log n)	O(log n)	O(log n)	sorted order
binary heap	O(1) min	O(n)	O(log n)	O(log n)	pop min/max
trie	—	O(L)	O(L)	O(L)	L = key length

⚠ COMMON MISTAKES

- Calling one append O(1) worst case: a resize step is O(n)
- Assuming hash maps are always O(1): bad hashes or attacks degrade to O(n)
- Saying quicksort uses O(1) space: recursion needs O(log n) on average

✓ TIPS & BEST PRACTICES

- Pre-size arrays/maps when n is known to avoid resizes
- Quote "O(1) amortized" for appends and hash inserts
- Keep this table handy when picking a structure (Sheet 003)

QUICK SUMMARY

- amortized — averaged over a sequence
- ×2 growth — O(1) amortized append
- hash map — O(1) avg, O(n) worst
- heap — O(1) peek, O(log n) push/pop

Sheet 026

Constraints → Target Complexity

$n \le 10 \cdot n \le 20 \cdot n \le 500 \cdot n \le 10^4 \cdot n \le 10^6 \cdot n \ge 10^9 \cdot \sim 10^8$ ops/sec · TLE fixes

LET THE LIMITS PICK THE ALGORITHM

Problem constraints tell you how fast your solution must be. A judge allows roughly **10^8 simple operations per second** in C++ or Java (about 10^7 in Python), with typical limits of 1–2 seconds. Plug n into candidate complexities, and anything far over ~10^8 won't pass.

HOW TO USE

- Find the largest n in the constraints
- Look up the allowed complexity below
- That narrows the pattern before you design
- Small n (≤ 20) is a hint that exponential is fine

N → TARGET COMPLEXITY

MAX N	ALLOWED	TYPICAL TECHNIQUES
$n \le 10$	O(n!)	permutations, brute-force backtracking
$n \le 20$	$O(2^n \cdot n)$	subsets, bitmask DP
$n \le 100$	$O(n^4)$	4 nested loops, small DP
$n \le 500$	$O(n^3)$	Floyd-Warshall, interval DP
$n \le 5 \times 10^3$	$O(n^2)$	2D DP, all pairs
$n \le 10^5$ – 10^6	O(n log n) / O(n)	sort, heap, binary search, two pointers, hashing
$n \le 10^9$+	O(log n) / O(√n) / O(1)	binary search on answer, math

BACK-OF-ENVELOPE CHECK

```
// budget: 10^8 ops (Java), 10^7 (Py)
n ← 10^5
n * n           // 10^10: too slow
n * log2(n)     // 1.7 × 10^6: fine
n ← 20
2^n * n         // 2 × 10^7: fine
n ← 5000
n * n           // 2.5 × 10^7: fine
n * n * n       // 10^11: too slow
```

TIME LIMIT EXCEEDED: FIXES

SYMPTOM	FIX
nested loop over 10^5 items	hash map, two pointers, sort + scan
recomputing overlapping subproblems	memoize (Sheet 078)
repeated range sums	prefix sums (Sheet 065)
repeated min/max of a window	monotonic deque (Sheet 068)
x in list inside a loop	use a set
deep recursion in Python	iterative version or raise the limit
slow I/O in big inputs	buffered / batch reading

⚠ COMMON MISTAKES

- Ignoring the constraints and coding $O(n^2)$ for $n = 10^5$
- Over-engineering O(n log n) when $n \le 100$ and $O(n^3)$ is simpler
- Forgetting the second dimension: grid $10^3 \times 10^3$ is already 10^6 cells

✓ TIPS & BEST PRACTICES

- Ask for constraints in interviews if they aren't stated
- Count operations of the **inner loop body**, not lines of code
- Python's constant factor is ~10× higher; leave extra headroom

QUICK SUMMARY

- 10^8 ops/sec — rough judge budget
- $n \le 20$ — exponential is OK
- $n \le 10^5$ — need O(n log n)
- $n \ge 10^9$ — log n or math

Arrays & Dynamic Arrays

contiguous memory · O(1) index · insert / delete O(n) · resize doubling · 2D arrays · row-major order · in-place edits

CONTIGUOUS, INDEXED STORAGE

An **array** stores items side by side in memory, so any index is reached in O(1) by arithmetic (`base + i × size`). Inserting or deleting in the middle shifts everything after it: O(n). A **dynamic array** (Python list, Java `ArrayList` , JS array, C++ `vector`) grows by doubling, making `append` O(1) amortized.

COSTS

- Read / write `arr[i]` : **O(1)**
- Append / pop at end: **O(1)** amortized
- Insert / delete at front or middle: **O(n)**
- Search unsorted: **O(n)**; sorted: **O(log n)**

WORKING WITH ARRAYS

```
arr ← new Array(5, 0)              // [0, 0, 0, 0, 0]
arr[2] ← 7                         // O(1) write
last ← arr[length(arr) - 1]        // O(1) read

list ← []                          // dynamic array
list.append(4)                     // O(1) amortized
list.insert(0, 9)                  // O(n): shifts right
list.removeAt(0)                   // O(n): shifts left
list.pop()                         // O(1): remove last

grid ← new Array(rows)             // 2D grid, row-major
for r ← 0 to rows - 1
    grid[r] ← new Array(cols, 0)

function reverse(arr, l, r)        // in place, O(1) extra space
    while l < r
        (arr[l], arr[r]) ← (arr[r], arr[l])
        (l, r) ← (l + 1, r - 1)
```

CODE EXPLAINED

- → `new Array(n, v)` — n slots, all `v`
- → `insert(0, x)` — every element moves
- → `pop()` — cheap; prefer working from the end
- → row-major — `grid[r][c]` rows stored one after another
- → `reverse` — building block for rotations

IN-PLACE TECHNIQUES

TECHNIQUE	IDEA
`write pointer`	copy keepers forward, O(1) space
`swap from both ends`	reverse, partition
`3 reversals`	rotate right by k
`fill from the back`	merge into an array with spare room
`use sign / index as a flag`	mark seen values (Sheet 071)

PRACTICE

PROBLEM	KEY MOVE
`Move Zeroes`	write pointer
`Remove Element`	write pointer
`Rotate Array`	3 reversals
`Merge Sorted Array`	fill from the back
`Product Except Self`	prefix × suffix

⚠ COMMON MISTAKES

- Inserting at index 0 inside a loop: O(n²) total; append and reverse once instead
- Reading `arr[n]` : the last valid index is `n - 1`
- Creating a 2D grid with one shared row (`[[0] * c] * r` in Python)

✓ TIPS & BEST PRACTICES

- Rotate by k: `k ← k mod n` first, then reverse all, first k, and the rest
- Pre-allocate when the final size is known
- Iterate backward when deleting by index

QUICK SUMMARY

- `arr[i]` — O(1) access
- `append` — O(1) amortized
- `insert(0, x)` — O(n)
- write pointer — in-place filtering

Strings: Characters & Techniques

TEXT AS AN ARRAY OF CHARACTERS

A string is a sequence of characters, and most string problems are array problems in disguise. Each character has a numeric **code**, so `code(c) - code('a')` maps `'a'…'z'` to `0…25`, the index into a **count[26]** array. In many languages strings are **immutable**, so build results with a list or builder, not repeated `+`.

QUICK FACTS

- Immutable in Java, Python, JS, C#; mutable in C++
- `s + t` creates a new string: O(len)
- `code('0')` =48, `code('A')` =65, `code('a')` =97
- Digit value: `code(c) - code('0')`

CORE TECHNIQUES

```
function isAnagram(a, b)                    // same letters, same counts
    if length(a) ≠ length(b) return false
    count ← new Array(26, 0)
    for i ← 0 to length(a) - 1
        x ← code(a[i]) - code('a')
        y ← code(b[i]) - code('a')
        count[x] ← count[x] + 1
        count[y] ← count[y] - 1
    return all(count, lambda v: v = 0)
function isPalindrome(s)                    // two pointers, skip symbols
    (l, r) ← (0, length(s) - 1)
    while l < r
        if not isAlnum(s[l])
            l ← l + 1
        else if not isAlnum(s[r])
            r ← r - 1
        else if lower(s[l]) ≠ lower(s[r])
            return false
        else
            (l, r) ← (l + 1, r - 1)
    return true
```

CODE EXPLAINED

→ +1 for `a`, -1 for `b`: all zeros means anagram

→ two pointers — compare ends, move inward

→ `isAlnum` / `lower` — normalize before comparing

OPERATION COSTS

OPERATION	COST
`length(s)`, `s[i]`	O(1)
`s[i..j]` substring	O(j - i)
`s + t`	O(|s| + |t|)
`compare s = t`	O(min length)
`split` / `join` / `reverse`	O(n)
`sort characters`	O(n log n)

PRACTICE

PROBLEM	KEY MOVE
`Valid Anagram`	count[26]
`Valid Palindrome`	two pointers
`Longest Common Prefix`	compare column by column
`Reverse Words in a String`	split, reverse, join
`String Compression`	read / write pointers
`Group Anagrams`	sorted key (Sheet 035)

⚠ COMMON MISTAKES

- Building output with `s ← s + c` in a loop: O(n²)
- Assuming only lowercase letters: ask about case, digits, Unicode
- Using count[26] for general Unicode text: use a hash map instead

✓ TIPS & BEST PRACTICES

- Convert to a char array when you need in-place edits
- Lowercase once at the start rather than in every comparison
- Sorting a word gives a canonical key for anagram grouping

QUICK SUMMARY

- `code(c) - code('a')` — letter → 0…25
- `count[26]` — frequency table
- two pointers — palindromes
- builder — efficient concatenation

Sheet 029

Regular Expressions (Primer)

literals · . * + ? · [a-z] classes · ^ $ anchors · groups · {m,n} · greedy vs lazy · see RegEx set

PATTERN MATCHING ON TEXT

A **regular expression** is a compact pattern that describes a set of strings, used to validate, search, extract, and replace text. Engines in most languages share the same core syntax shown here. This is a one-page primer; the **RegEx set** covers flags, lookaround, Unicode, performance, and per-language APIs.

GOOD TO KNOW

- Escape special characters with `\` : `\.` matches a dot
- Quantifiers are **greedy** by default; add `?` for lazy
- LeetCode "Regular Expression Matching" is DP (Sheet 083)
- Use raw strings (`r"…"`) in Python to keep backslashes

COMMON OPERATIONS

```
match("^\d{3}-\d{4}$", "555-1234")          // true: whole string
search("[A-Z]\w+", "say Hello there")        // "Hello"
findAll("\d+", "a1b22c333")                  // ["1", "22", "333"]
replace("\s+", " ", "a   b    c")           // "a b c"
groups("(\w+)@(\w+)\.com", "ada@site.com")   // ["ada", "site"]
match("colou?r", "color")                    // true: u is optional
match("^(cat|dog)s?$", "dogs")               // true: alternation
search("<.+>", "<a><b>")                     // "<a><b>"  greedy
search("<.+?>", "<a><b>")                    // "<a>"     lazy
search("\bcat\b", "concat cat")             // matches the word "cat" only
```

CODE EXPLAINED

→ `match` — pattern must fit the whole string (with `^…$`)

→ `search` — first match anywhere

→ `findAll` — every non-overlapping match

→ `groups` — text captured by `( )`

→ `+?` — lazy: as few characters as possible

CHARACTERS & CLASSES

PATTERN	MATCHES
`.`	any char except newline
`\d \w \s`	digit · word char · whitespace
`\D \W \S`	the opposite of each
`[abc] [^abc]`	one of · none of
`[a-z0-9]`	a range
`^ $`	start · end
`\b`	word boundary

QUANTIFIERS

PATTERN	COUNT
`*`	0 or more
`+`	1 or more
`?`	0 or 1
`{3}`	exactly 3
`{2,5}`	2 to 5
`{2,}`	2 or more
`*? +?`	lazy versions

GROUPS & LOGIC

PATTERN	MEANING
`( … )`	capture group
`(?: … )`	group, no capture
`a|b`	a or b
`\1`	same text as group 1
`(?= … )`	lookahead
`(?! … )`	negative lookahead
`(?<name> … )`	named group

⚠ COMMON MISTAKES

- Forgetting anchors: `\d+` "validates" `"abc123"` because it finds a match inside
- Unescaped `.` in domains: `site.com` also matches `siteXcom`
- Greedy `.*` swallowing too much; use `.*?` or a negated class `[^>]*`

✓ TIPS & BEST PRACTICES

- Test patterns interactively (regex101.com) with the right flavor selected
- Prefer simple string methods (`split` , `startsWith`) when they're enough
- Full coverage: see the **RegEx** cheat sheet set

QUICK SUMMARY

- `\d \w \s` — digit, word, space
- `* + ? {m,n}` — quantifiers
- `( )` — capture groups
- `^ $` — anchor the whole string

Linked Lists — Basics

node · singly vs doubly · head / tail · insert · delete · traversal · dummy / sentinel node

NODES LINKED BY POINTERS

A **linked list** is a chain of nodes, each holding a value and a pointer to the **next** node (and, in a **doubly** linked list, the previous one). Inserting or removing is O(1) once you hold the right node, but reaching index i takes O(i). A **dummy (sentinel) head** removes special cases when the real head might change.

COSTS

- Access / search: **O(n)**
- Insert / delete at a known node: **O(1)**
- Insert at head: **O(1)**; at tail: O(1) with a tail pointer
- Extra memory per node for pointers

NODES, TRAVERSAL, INSERT, DELETE

```
class Node
    function init(val, next ← null)
        this.val ← val
        this.next ← next

head ← new Node(1, new Node(2, new Node(3)))   // 1 → 2 → 3
node ← head                                    // traverse: O(n)
while node ≠ null
    print(node.val)
    node ← node.next
head ← new Node(0, head)                       // insert at head: O(1)
prev.next ← new Node(5, prev.next)             // insert after prev: O(1)

function removeAll(head, target)               // dummy avoids head special-case
    dummy ← new Node(0, head)
    prev ← dummy
    while prev.next ≠ null
        if prev.next.val = target
            prev.next ← prev.next.next         // unlink
        else
            prev ← prev.next
    return dummy.next
```

CODE EXPLAINED

- → `next ← null` — the last node points nowhere
- → traversal — follow `next` until `null`
- → insert — set the new node's `next` **before** relinking
- → dummy — a fake node before `head`
- → unlink — skip over the node; GC frees it
- → return `dummy.next` — the head may have changed

SINGLY VS DOUBLY

	SINGLY	DOUBLY
pointers / node	`next`	`prev`, `next`
delete given node	need previous node	O(1)
walk backward	no	yes
used in	stacks, hash chains	LRU cache, deques

LIST VS ARRAY

OPERATION	ARRAY	LINKED LIST
access i	O(1)	O(n)
insert at front	O(n)	O(1)
insert at known spot	O(n)	O(1)
memory	compact	pointer overhead

⚠ COMMON MISTAKES

- Losing the rest of the list: relinking before saving `node.next`
- Accessing `node.next.val` without checking `node.next ≠ null`
- Returning the old `head` after deleting the first node

✓ TIPS & BEST PRACTICES

- Practice: Design Linked List, Remove Linked List Elements, Remove Duplicates from Sorted List
- Use a dummy node whenever the head can change
- Draw boxes and arrows for 3 nodes before coding pointer updates

QUICK SUMMARY

- `node.next` — follow the chain
- `dummy` — sentinel before head
- `prev.next ← prev.next.next` — unlink
- O(1) insert / delete, O(n) access

Sheet 031

Linked Lists — Techniques

reverse in place · fast / slow pointers · Floyd cycle detection · find middle · merge two sorted · remove nth from end

POINTER PATTERNS THAT SOLVE MOST LIST PROBLEMS

Almost every linked-list problem combines a few moves: **reverse** links in place, run **fast and slow** pointers (to find the middle, detect a cycle, or keep a fixed gap), and **merge** with a dummy tail. All run in O(n) time and O(1) extra space.

RECOGNIZE IT WHEN

- "in place", "O(1) extra space" on a list
- "middle", "cycle", "nth from end"
- "reverse" all or part of a list
- "merge", "reorder", "palindrome list"

REVERSE · MIDDLE

```
function reverse(head)                  // O(n), O(1)
    prev ← null
    curr ← head
    while curr ≠ null
        nxt ← curr.next                 // save
        curr.next ← prev                // flip
        prev ← curr
        curr ← nxt
    return prev                         // new head

function middle(head)                   // fast: 2 steps
    slow ← head
    fast ← head
    while fast ≠ null and fast.next ≠ null
        slow ← slow.next
        fast ← fast.next.next
    return slow                         // 2nd middle if even
```

CYCLE · NTH FROM END

```
function hasCycle(head)                 // Floyd
    (slow, fast) ← (head, head)
    while fast ≠ null and fast.next ≠ null
        slow ← slow.next
        fast ← fast.next.next
        if slow = fast return true      // they meet
    return false

function removeNthFromEnd(head, n)      // gap of n
    dummy ← new Node(0, head)
    (fast, slow) ← (dummy, dummy)
    for i ← 1 to n + 1
        fast ← fast.next
    while fast ≠ null
        (fast, slow) ← (fast.next, slow.next)
    slow.next ← slow.next.next          // skip target
    return dummy.next
```

MERGE TWO SORTED LISTS

```
function mergeTwo(a, b)
    dummy ← new Node(0)
    tail ← dummy
    while a ≠ null and b ≠ null
        if a.val ≤ b.val
            (tail.next, a) ← (a, a.next)
        else
            (tail.next, b) ← (b, b.next)
        tail ← tail.next
    tail.next ← if a ≠ null then a else b
    return dummy.next
```

TECHNIQUE → PROBLEMS

TECHNIQUE	PROBLEMS
`reverse`	Reverse List, Reverse Nodes in k-Group
`fast / slow`	Middle of List, Palindrome List
`Floyd cycle`	Linked List Cycle I / II
`fixed gap`	Remove Nth Node From End
`merge + dummy`	Merge Two / k Sorted Lists
`split + reverse + merge`	Reorder List

⚠ COMMON MISTAKES

- Overwriting `curr.next` before saving it: the rest of the list is lost
- Loop condition `fast.next ≠ null` alone: crashes when `fast` is null
- Using `slow = fast` to start Floyd's loop: they're equal before moving

✓ TIPS & BEST PRACTICES

- Cycle start: after they meet, move one pointer to `head`; step both by 1 until they meet again
- Palindrome list: find middle, reverse second half, compare
- Use a dummy head for any merge or removal

QUICK SUMMARY

- `reverse` — prev / curr / next
- `slow` + `fast` — middle, cycles
- gap of n — nth from end
- dummy tail — merging

Stacks

LIFO · push / pop / peek · valid parentheses · min stack · evaluate RPN · undo history · array-backed stack

LAST IN, FIRST OUT

A **stack** only touches its top: `push` adds, `pop` removes the most recent item, `peek` looks without removing. All are O(1). Stacks model anything **nested** or **most-recent-first**: matching brackets, undo history, expression evaluation, and the call stack that makes recursion work.

RECOGNIZE IT WHEN

- Matching pairs or nesting: `()[]{}`, tags
- "Most recent" item matters (undo, backspace)
- Evaluate or parse expressions
- Next greater / smaller element → monotonic stack (067)

VALID PARENTHESES

```
function isValid(s)
    pairs ← { ')': '(', ']': '[', '}': '{' }
    stack ← []
    for each c in s
        if c in "([{"
            stack.push(c)
        else if stack.isEmpty()
            return false              // no opener
        else if stack.pop() ≠ pairs[c]
            return false              // wrong opener
    return stack.isEmpty()        // unclosed = invalid
```

EVALUATE REVERSE POLISH NOTATION

```
// ["2", "1", "+", "3", "*"]  →  (2 + 1) * 3 = 9
function evalRPN(tokens)
    stack ← []
    for each t in tokens
        if t in ["+", "-", "*", "/"]
            b ← stack.pop()           // right operand
            a ← stack.pop()
            stack.push(apply(t, a, b))
        else
            stack.push(toInt(t))
    return stack.pop()
```

MIN STACK: O(1) GETMIN

```
class MinStack
    function init()
        this.items ← []
        this.mins ← []                  // top = current minimum
    function push(x)
        this.items.push(x)
        if this.mins.isEmpty() or x ≤ this.mins.peek()
            this.mins.push(x)
    function pop()
        x ← this.items.pop()
        if x = this.mins.peek()
            this.mins.pop()
        return x
    function getMin()
        return this.mins.peek()
```

PRACTICE

PROBLEM	STACK HOLDS
Valid Parentheses	open brackets
Min Stack	values + running mins
Evaluate RPN	operands
Backspace String Compare	kept characters
Decode String	counts + partial strings
Asteroid Collision	surviving asteroids
Daily Temperatures	indexes (Sheet 067)

⚠ COMMON MISTAKES

- Popping an empty stack: check `isEmpty()` first
- Pushing `≤` vs `<` in MinStack: duplicates of the min must be pushed too
- Operand order in RPN: `a - b` needs `b` popped first

✓ TIPS & BEST PRACTICES

- A dynamic array is a stack: `append` / `pop` from the end
- Store **indexes**, not values, when you'll need positions later
- Convert recursion to iteration with an explicit stack

QUICK SUMMARY

- `push` / `pop` / `peek` — all O(1)
- LIFO — most recent first
- brackets — push opens, match on close
- two stacks — O(1) min

Queues & Deques

FIFO · enqueue / dequeue · circular buffer · deque · queue from two stacks · BFS use

FIRST IN, FIRST OUT

A **queue** adds at the back and removes from the front, so items leave in arrival order: the backbone of **BFS**, task scheduling, and buffering. A **deque** (double-ended queue) adds and removes at **both** ends in O(1), which makes it a stack, a queue, and a sliding-window tool at once.

QUICK FACTS

- `enqueue` / `dequeue` : **O(1)** with a proper queue
- Removing from the front of an **array** is O(n)
- A **circular buffer** is a fixed-size array queue
- Python `collections.deque` , Java `ArrayDeque`

QUEUE, DEQUE, CIRCULAR BUFFER

```
queue ← new Queue()
queue.enqueue("a")
queue.enqueue("b")
queue.dequeue()                    // "a"  first in, first out

dq ← new Deque()
dq.pushBack(1)
dq.pushFront(0)                    // [0, 1]; popBack / popFront

class CircularQueue                // fixed array that wraps
    function init(k)
        (this.data, this.head, this.size) ← (new Array(k), 0, 0)
    function enqueue(x)
        if this.size = length(this.data) return false   // full
        tail ← (this.head + this.size) mod length(this.data)
        this.data[tail] ← x
        this.size ← this.size + 1
        return true
    function dequeue()
        x ← this.data[this.head]
        this.head ← (this.head + 1) mod length(this.data)
        this.size ← this.size - 1
        return x
```

QUEUE FROM TWO STACKS

```
class MyQueue
    function init()
        this.in ← []
        this.out ← []
    function push(x)
        this.in.push(x)
    function pop()
        if this.out.isEmpty()
            while not this.in.isEmpty()
                this.out.push(this.in.pop())
        return this.out.pop()

// each item moves in → out once:
// O(1) amortized per operation
```

LANGUAGE EQUIVALENTS

LANGUAGE	QUEUE / DEQUE	AVOID
Python	`collections.deque`	`list.pop(0)` (O(n))
Java	`ArrayDeque`	`Stack` class (legacy)
JavaScript	index pointer or linked list	`arr.shift()` (O(n))
C++	`std::queue` , `std::deque`	`vector.erase(begin)`
C#	`Queue<T>` , `LinkedList<T>`	`List.RemoveAt(0)`

PRACTICE

PROBLEM	STRUCTURE
Implement Queue using Stacks	two stacks
Design Circular Queue	ring buffer
Number of Recent Calls	queue of times
Sliding Window Maximum	deque (068)
Rotting Oranges	BFS queue (048)

⚠ COMMON MISTAKES

- Using an array's `shift` / `pop(0)` as a queue: O(n) per dequeue
- Forgetting the `mod` wrap in a circular buffer: index runs off the end
- Distinguishing full vs empty by `head = tail` alone: keep a `size`

✓ TIPS & BEST PRACTICES

- BFS always uses a queue; a stack there would make it DFS
- Use a deque when you need both ends (sliding windows, 0-1 BFS)
- A JS queue: keep an array plus a moving `head` index

QUICK SUMMARY

- `enqueue` / `dequeue` — FIFO, O(1)
- `deque` — O(1) at both ends
- `(head + size) mod k` — ring buffer tail
- two stacks — amortized O(1) queue

Hash Maps

hash function · buckets · chaining vs open addressing · load factor · O(1) average · hashable keys · iteration order

KEY → VALUE IN O(1) AVERAGE

A **hash map** runs each key through a **hash function** to pick a **bucket** in an array, so lookups skip searching. Different keys can land in the same bucket (**collision**); buckets hold a short list (**chaining**) or probe to the next free slot (**open addressing**). When the **load factor** (items ÷ buckets) gets high, the table doubles and rehashes.

COSTS

- get / put / delete: **O(1)** average
- Worst case **O(n)**: everything in one bucket
- Resize: O(n), but O(1) amortized per insert
- No ordering by key (use a tree map for that)

INSIDE PUT / GET, AND EVERYDAY USE

```
function put(key, value)                    // separate chaining
    i ← hash(key) mod capacity              // choose a bucket
    for each entry in buckets[i]
        if entry.key = key
            entry.value ← value             // update existing key
            return
    buckets[i].append((key, value))
    size ← size + 1
    if size / capacity > 0.75               // load factor limit
        resize(capacity * 2)                // rehash all entries

function get(key)
    i ← hash(key) mod capacity
    for each entry in buckets[i]
        if entry.key = key return entry.value
    return null

ages ← new Map()
ages["ada"] ← 36
ages.has("bob")                             // false
ages.getOrDefault("bob", 0)                 // 0
```

CODE EXPLAINED

- → `hash(key)` — turns the key into an integer
- → `mod capacity` — maps it onto a bucket
- → update vs insert — keys are unique
- → `0.75` — common default load factor
- → `resize` — new array, re-insert every entry
- → `getOrDefault` — avoids missing-key errors

COLLISION STRATEGIES

STRATEGY	HOW	TRADE-OFF
`chaining`	list per bucket	simple; extra pointers
`linear probing`	try i+1, i+2, …	cache-friendly; clustering
`quadratic probing`	try i+1², i+2², …	less clustering
`double hashing`	step by a 2nd hash	even spread

ITERATION ORDER

TYPE	ORDER
`Python dict`	insertion (3.7+)
`JS Map`	insertion
`Java HashMap`	unspecified
`Java LinkedHashMap`	insertion
`Java TreeMap / C++ map`	sorted by key
`C++ unordered_map`	unspecified

⚠ COMMON MISTAKES

- Using a mutable list as a key: Python rejects it; elsewhere a changed key gets lost
- Overriding `equals` without `hashCode` (Java): equal keys land in different buckets
- Relying on `HashMap` iteration order: it can change between runs

✓ TIPS & BEST PRACTICES

- Use a tuple or string as a composite key: `(row, col)` or `"r,c"`
- Use a counting helper (`Counter` , `getOrDefault`) instead of `if has … else …`
- Need sorted keys or floor/ceiling lookups? Use a tree map (Sheet 040)

QUICK SUMMARY

- `hash(key) mod n` — bucket index
- collision — chaining or probing
- load factor — triggers resize
- O(1) avg · O(n) worst

Sheet 035

Hash Sets & Counting Patterns

set membership · dedupe · frequency map · complement lookup (Two Sum) · group by key · longest consecutive sequence

THE MOST-USED INTERVIEW TOOL

A **hash set** answers "have I seen this?" in O(1); a **hash map** answers "how many?" or "where?". Replacing an inner search loop with a lookup is the single most common way to turn O(n²) into O(n). The art is choosing the **key**: the value itself, its complement, or a canonical form.

RECOGNIZE IT WHEN

- "Find a pair that sums to …" (complement lookup)
- "Contains duplicate", "first unique", "seen before"
- "Group items that are equivalent" (canonical key)
- Counts or frequencies of anything

TWO SUM: COMPLEMENT LOOKUP

```
function twoSum(nums, target)              // O(n) time, O(n) space
    seen ← new Map()                       // value → index
    for i ← 0 to length(nums) - 1
        need ← target - nums[i]
        if seen.has(need)
            return (seen[need], i)
        seen[nums[i]] ← i                  // add AFTER checking
    return null
```

COUNTING IDIOMS

GOAL	KEY / STRUCTURE
contains duplicate	set; collision = duplicate
first unique character	count map, second pass
frequencies	`count[x] ← count[x] + 1`
isomorphic strings	two maps: a→b and b→a
top-k frequent	count map + heap / buckets
subarray sum = k	prefix-sum counts (Sheet 065)

GROUP ANAGRAMS: CANONICAL KEY

```
function groupAnagrams(words)
    groups ← new Map()
    for each w in words
        key ← sorted(w)                    // or 26-count tuple
        if not groups.has(key)
            groups[key] ← []
        groups[key].append(w)
    return values(groups)

// "eat","tea","ate" → key "aet"
// O(n · k log k), k = word length
```

LONGEST CONSECUTIVE SEQUENCE

```
function longestConsecutive(nums)      // O(n)
    set ← toSet(nums)
    best ← 0
    for each x in set
        if not set.has(x - 1)          // x starts a run
            len ← 1
            while set.has(x + len)
                len ← len + 1
            best ← max(best, len)
    return best
```

⚠ COMMON MISTAKES

- Adding to `seen` before checking: a number can pair with itself
- Sorting each word as a key for very long words: count tuples are O(k)
- Starting the consecutive scan from every number: O(n²); only start at run heads

✓ TIPS & BEST PRACTICES

- Ask "what would I need to have seen?" and store exactly that
- Use `getOrDefault` / `Counter` to keep counting code short
- Sets trade memory for speed: mention O(n) space

QUICK SUMMARY

- `set.has(x)` — O(1) membership
- complement — `target - x`
- canonical key — group equivalents
- run heads — `not has(x - 1)`

Sheet 036

Tree Terminology & Properties

root · leaf · depth vs height · binary / n-ary · full · complete · perfect · balanced

HIERARCHIES OF NODES

A **tree** is a connected structure with no cycles: one **root**, and every other node has exactly one **parent**. In a **binary tree** each node has at most two children, `left` and `right`. Most tree algorithms are short recursive functions because every subtree is itself a tree.

HANDY FACTS

- n nodes → exactly **n − 1** edges
- Level d holds at most 2^d nodes
- Perfect tree of height h: $2^{h+1} - 1$ nodes
- Complete tree height: $\lfloor \log_2 n \rfloor$

ANATOMY

```
          1          ← root   (depth 0)
        /   \
       2     3       ← depth 1
      / \     \
     4   5     6     ← depth 2

leaves: 4, 5, 6
parent of 5: 2      siblings: 4 and 5
subtree at 2: {2, 4, 5}
height of tree: 2 (edges, root → deepest leaf)
height of node 3: 1
```

NODE AND BASIC MEASURES

```
class TreeNode
    function init(val, left ← null, right ← null)
        (this.val, this.left, this.right) ← (val, left, right)

function height(node)           // edges; empty tree = -1
    if node = null return -1
    return 1 + max(height(node.left), height(node.right))

function countNodes(node)
    if node = null return 0
    return 1 + countNodes(node.left) + countNodes(node.right)

function depthOf(root, target, d ← 0)
    if root = null return -1
    if root = target return d
    l ← depthOf(root.left, target, d + 1)
    return if l ≠ -1 then l else depthOf(root.right, target, d + 1)
```

TERMS

TERM	MEANING
`root`	the top node, no parent
`leaf`	node with no children
`depth`	edges from the root down to a node
`height`	edges from a node down to its deepest leaf
`subtree`	a node and all its descendants
`degree`	number of children
`n-ary`	up to n children per node

BINARY TREE SHAPES

SHAPE	DEFINITION
`full`	every node has 0 or 2 children
`complete`	all levels full except the last, filled left to right
`perfect`	all internal nodes have 2 children, leaves at one depth
`balanced`	subtree heights differ by ≤ 1 everywhere
`degenerate`	each node has one child (a linked list)

⚠ COMMON MISTAKES

- Mixing height conventions: LeetCode's "max depth" counts **nodes** (empty = 0)
- Calling any tree with two children per node "perfect" or "complete"
- Forgetting the `null` base case: every recursion must handle an empty subtree

✓ TIPS & BEST PRACTICES

- Draw a 5–7 node example before coding any tree function
- A heap is a **complete** tree, which is why it fits in an array
- Balanced → height O(log n); skewed → O(n)

QUICK SUMMARY

- `root` / `leaf` — top / no children
- `depth` ↓ from root · `height` ↑ to leaves
- `n - 1` edges for n nodes
- complete · full · perfect · balanced

Binary Tree Traversals

preorder · inorder · postorder · iterative with stack · level order (BFS) · Morris traversal

VISITING EVERY NODE

Depth-first traversals go down one branch before backtracking; the only difference is **when** the node is visited: before (**pre**order), between (**in**order), or after (**post**order) its children. **Level order** (breadth-first) visits the tree row by row with a queue. All are O(n) time.

WHICH ORDER?

- **Preorder**: copy or serialize a tree
- **Inorder**: BST values come out **sorted**
- **Postorder**: children before parent (sizes, delete)
- **Level order**: anything per level, nearest-first

RECURSIVE DFS

```
function preorder(node)          // root, left, right
    if node = null return
    visit(node)
    preorder(node.left)
    preorder(node.right)
function inorder(node)           // left, root, right
    if node = null return
    inorder(node.left)
    visit(node)
    inorder(node.right)
function postorder(node)         // left, right, root
    if node = null return
    postorder(node.left)
    postorder(node.right)
    visit(node)
```

ITERATIVE INORDER (EXPLICIT STACK)

```
function inorderIterative(root)
    stack ← []
    node ← root
    while node ≠ null or not stack.isEmpty()
        while node ≠ null            // dive left
            stack.push(node)
            node ← node.left
        node ← stack.pop()
        visit(node)
        node ← node.right            // then go right

// preorder: pop, visit, push right, push left
// postorder: reverse of (root, right, left)
```

LEVEL ORDER (BFS)

```
function levelOrder(root)
    if root = null return []
    result ← []
    queue ← new Queue([root])
    while not queue.isEmpty()
        level ← []
        count ← length(queue)          // nodes on this level
        repeat count times
            node ← queue.dequeue()
            level.append(node.val)
            if node.left ≠ null
                queue.enqueue(node.left)
            if node.right ≠ null
                queue.enqueue(node.right)
        result.append(level)
    return result
```

ORDERS FOR THE TREE ON SHEET 036

TRAVERSAL	OUTPUT
preorder	1 2 4 5 3 6
inorder	4 2 5 1 3 6
postorder	4 5 2 6 3 1
level order	[1] [2 3] [4 5 6]
space (DFS)	O(h) stack
space (BFS)	O(widest level)
Morris	O(1) space via temp links

⚠ COMMON MISTAKES

- Reading `length(queue)` inside the inner loop: the level mixes with the next one
- Recursive DFS on a 10^5-deep skewed tree: stack overflow
- Pushing left before right in iterative preorder: visits right subtree first

✓ TIPS & BEST PRACTICES

- Right Side View, Zigzag, Level Averages: all small tweaks of level order
- Validate BST and Kth Smallest: inorder
- Morris traversal threads `predecessor.right ← node` to get O(1) space

QUICK SUMMARY

- pre — node first · in — middle · post — last
- inorder of BST — sorted
- BFS — queue + level size
- all traversals — O(n) time

Binary Tree Recursion Patterns

max depth · invert · same tree / subtree · balanced check · diameter · path sum · return-a-tuple recursion

THINK IN SUBTREES

Nearly every binary-tree problem is a recursive function that trusts its calls on `left` and `right`. Decide which way information flows: **top-down** passes values **down** as parameters (remaining sum, bounds); **bottom-up returns** values from children (height, balanced flag). When the answer can pass through any node, combine bottom-up returns with a **running best**.

TEMPLATE QUESTIONS

- What does the function return for `null`?
- What do I need from each child?
- What do I combine at the current node?
- Does the answer bend through a node? Track `best`

CLASSIC ONE-LINERS

```
function maxDepth(node)
    if node = null return 0
    return 1 + max(maxDepth(node.left), maxDepth(node.right))

function invert(node)
    if node = null return null
    (l, r) ← (invert(node.left), invert(node.right))
    (node.left, node.right) ← (r, l)
    return node

function isSame(a, b)
    if a = null or b = null return a = b
    return a.val = b.val
        and isSame(a.left, b.left)
        and isSame(a.right, b.right)
```

TOP-DOWN: PASS STATE DOWN

```
function hasPathSum(node, target)
    if node = null return false
    rest ← target − node.val
    if node.left = null and node.right = null
        return rest = 0              // leaf check
    return hasPathSum(node.left, rest)
        or hasPathSum(node.right, rest)

function isSubtree(root, sub)
    if root = null return false
    return isSame(root, sub)
        or isSubtree(root.left, sub)
        or isSubtree(root.right, sub)
```

BOTTOM-UP + RUNNING BEST

```
function diameter(root)              // longest path (edges)
    best ← 0
    function depth(node)
        if node = null return 0
        l ← depth(node.left)
        r ← depth(node.right)
        best ← max(best, l + r)      // bends here
        return 1 + max(l, r)         // longest arm upward
    depth(root)
    return best
```

RETURN A TUPLE

```
function isBalanced(root)
    function check(node)             // → (balanced, height)
        if node = null return (true, 0)
        (lb, lh) ← check(node.left)
        (rb, rh) ← check(node.right)
        ok ← lb and rb and abs(lh − rh) ≤ 1
        return (ok, 1 + max(lh, rh))
    (ok, h) ← check(root)
    return ok

// one pass: O(n) instead of O(n²)
```

⚠ COMMON MISTAKES

- Calling `height()` inside `isBalanced` at every node: O(n²) instead of O(n)
- Treating a node with one child as a leaf in path-sum problems
- Returning the diameter instead of the arm length from the helper

✓ TIPS & BEST PRACTICES

- Practice: Max Depth, Invert, Same Tree, Subtree, Path Sum, Diameter, Balanced
- Use a closure variable or a one-element array for `best`
- Write the `null` case first, then the leaf case if needed

QUICK SUMMARY

- top-down — parameters carry state
- bottom-up — return values carry state
- `best` — answers that bend through a node
- tuple return — several facts in one pass

Binary Search Trees

BST property · search / insert / delete · inorder = sorted · validate BST · kth smallest · successor · floor / ceiling

ORDERED BINARY TREES

In a **binary search tree**, every value in a node's left subtree is **smaller** and every value in its right subtree is **larger**. That lets search, insert, and delete follow a single path: **O(h)**, which is O(log n) when balanced and O(n) when skewed. An inorder traversal visits the values in **sorted order**.

RECOGNIZE IT WHEN

- Ordered data with frequent inserts and lookups
- "kth smallest", "successor", "closest value"
- "validate BST", "range sum of BST"
- Floor / ceiling: largest ≤ x, smallest ≥ x

SEARCH, INSERT, VALIDATE

```
function search(node, x)                    // O(h)
    while node ≠ null and node.val ≠ x
        node ← if x < node.val then node.left else node.right
    return node

function insert(node, x)
    if node = null return new TreeNode(x)
    if x < node.val
        node.left ← insert(node.left, x)
    else
        node.right ← insert(node.right, x)
    return node

function isValidBST(node, lo ← -∞, hi ← ∞)
    if node = null return true
    if node.val ≤ lo or node.val ≥ hi return false
    return isValidBST(node.left, lo, node.val)
        and isValidBST(node.right, node.val, hi)
```

DELETE

```
function delete(node, x)
    if node = null return null
    if x < node.val
        node.left ← delete(node.left, x)
    else if x > node.val
        node.right ← delete(node.right, x)
    else
        if node.left = null return node.right
        if node.right = null return node.left
        succ ← node.right                   // 2 children:
        while succ.left ≠ null              // min of right
            succ ← succ.left
        node.val ← succ.val
        node.right ← delete(node.right, succ.val)
    return node
```

FLOOR (LARGEST ≤ X)

```
function floor(node, x)
    best ← null
    while node ≠ null
        if node.val = x return x
        if node.val < x
            best ← node.val          // candidate; try bigger
            node ← node.right
        else
            node ← node.left
    return best
// ceiling: mirror (val > x → candidate, go left)
```

PRACTICE

PROBLEM	KEY MOVE
Search in a BST	follow one path
Validate BST	pass (lo, hi) bounds
Kth Smallest in BST	inorder, count to k
LCA of BST	split point (Sheet 042)
Delete Node in BST	successor swap
Inorder Successor	floor/ceiling walk

⚠ COMMON MISTAKES

- Validating by comparing only parent and child: a deep left node can still exceed the root
- Inserting sorted data into a plain BST: it becomes a linked list, O(n) per op
- Forgetting to reassign `node.left ← …` after recursive insert/delete

✓ TIPS & BEST PRACTICES

- Use a library ordered map/set (Sheet 040) instead of writing a balanced BST
- Kth smallest iteratively: inorder with a stack, stop at the kth pop
- Duplicate values: decide on a side (e.g. ≥ goes right) and keep it consistent

QUICK SUMMARY

- left < node < right — BST property
- O(h) — search / insert / delete
- inorder — sorted output
- bounds (lo, hi) — validation

Self-Balancing Trees

AVL trees · rotations · red-black rules · B-trees · skip lists · ordered map / TreeMap · O(log n) guarantee

KEEPING HEIGHT O(LOG N)

A plain BST degrades to O(n) when inputs arrive in order. **Self-balancing** trees restore balance after each insert or delete with **rotations**, guaranteeing O(log n) operations. You'll rarely implement one in an interview, but you'll use them constantly through **ordered maps and sets** such as Java `TreeMap` and C++ `std::map`.

FAMILY MEMBERS

- **AVL**: strict balance, faster lookups
- **Red-black**: looser balance, faster inserts (most libraries)
- **B-tree**: many keys per node; databases and file systems
- **Skip list**: layered linked lists, expected O(log n)

ROTATION (RIGHT)

```
//        y                x
//       / \              / \
//      x   C    →       A   y
//     / \                  / \
//    A   B                B   C
function rotateRight(y)
    x ← y.left
    y.left ← x.right            // B moves
    x.right ← y
    updateHeight(y)             // child first
    updateHeight(x)
    return x                    // new root
```

AVL REBALANCE

```
function rebalance(node)
    updateHeight(node)
    bf ← height(node.left) − height(node.right)
    if bf > 1                                   // left-heavy
        if balance(node.left) < 0
            node.left ← rotateLeft(node.left)   // LR case
        return rotateRight(node)                // LL case
    if bf < −1                                  // right-heavy
        if balance(node.right) > 0
            node.right ← rotateRight(node.right)   // RL
        return rotateLeft(node)                 // RR case
    return node
// call on every node along the insert/delete path
```

RED-BLACK RULES

#	RULE
1	every node is red or black
2	the root is black
3	a red node has black children
4	every root-to-null path has the same number of black nodes
→	height ≤ $2 \cdot \log_2(n + 1)$

ORDERED MAP / SET APIS

LANGUAGE	TYPE	FLOOR / CEILING
Java	`TreeMap`, `TreeSet`	`floorKey` `ceilingKey`
C++	`std::map`, `std::set`	`lower_bound` `upper_bound`
C#	`SortedDictionary`, `SortedSet`	`GetViewBetween`
Python	`sortedcontainers.SortedList`	`bisect_left` / `bisect_right`
JavaScript	none built in	sorted array + binary search

⚠ COMMON MISTAKES

- Assuming a hash map keeps keys ordered: only tree maps do
- Updating the parent's height before the child's after a rotation
- Reaching for a sorted array when there are many inserts: each insert is O(n)

✓ TIPS & BEST PRACTICES

- Need "next larger" or "range between" with updates? Use an ordered map
- Calendar booking, sliding-window median, and stock price problems use ordered sets
- Python lacks a built-in: `SortedList` is allowed on LeetCode

QUICK SUMMARY

- rotation — O(1) local restructure
- AVL — $|\text{balance}| \le 1$ at every node
- red-black — the usual library tree
- `TreeMap` / `std::map` — O(log n) ordered ops

Tree Construction & Serialization

build from preorder + inorder · build from postorder + inorder · serialize / deserialize · null markers · sorted array → BST

REBUILDING AND ENCODING TREES

One traversal alone can't pin down a tree's shape, but **preorder + inorder** can: preorder gives each subtree's root, and that root's position in inorder splits left from right. To save a tree as text, **serialize** a traversal with explicit **null markers**, then rebuild it by reading tokens in the same order.

KEY FACTS

- A hash map value → inorder index makes building O(n)
- Postorder + inorder: take roots from the **end**, build right first
- Null markers (`#`) make a single traversal enough
- Sorted array → balanced BST: middle element is the root

BUILD FROM PREORDER + INORDER

```
function buildTree(preorder, inorder)
    pos ← map of value → index in inorder
    p ← 0                                  // next root in preorder
    function build(lo, hi)                 // inorder range
        if lo > hi return null
        root ← new TreeNode(preorder[p])
        p ← p + 1
        mid ← pos[root.val]
        root.left ← build(lo, mid - 1)     // left first
        root.right ← build(mid + 1, hi)
        return root
    return build(0, length(inorder) - 1)
```

WHICH INPUTS FIX THE SHAPE?

GIVEN	UNIQUE TREE?
preorder + inorder	yes
postorder + inorder	yes
level order + inorder	yes
preorder + postorder	only if full
preorder of a BST	yes
traversal + null markers	yes

SERIALIZE / DESERIALIZE (PREORDER)

```
function serialize(node)
    if node = null return "#"
    return toString(node.val) + ","
        + serialize(node.left) + ","
        + serialize(node.right)

function deserialize(data)
    tokens ← iterator(split(data, ","))
    function build()
        t ← tokens.next()
        if t = "#" return null
        node ← new TreeNode(toInt(t))
        node.left ← build()
        node.right ← build()
        return node
    return build()
```

SORTED ARRAY → BALANCED BST

```
function toBST(arr, lo, hi)
    if lo > hi return null
    mid ← (lo + hi) div 2
    node ← new TreeNode(arr[mid])
    node.left ← toBST(arr, lo, mid - 1)
    node.right ← toBST(arr, mid + 1, hi)
    return node

// [1,2,3,4,5] → root 3, left {1,2}, right {4,5}
// "1,2,#,#,3,4,#,#,5,#,#" encodes:
//     1
//    / \
//   2   3
//      / \
//     4   5
```

⚠ COMMON MISTAKES

- Searching inorder linearly for each root: O(n²); use the index map
- Building right before left with preorder: the order must match the traversal
- Serializing without null markers: different trees produce the same string

✓ TIPS & BEST PRACTICES

- Use an iterator or index pointer shared across recursive calls
- BFS (level-order) serialization also works; it matches LeetCode's format
- Practice: Construct from Pre+In, Serialize/Deserialize, Sorted Array to BST

QUICK SUMMARY

- preorder root + inorder split — rebuild
- `#` — null marker
- index map — O(n) build
- middle element — balanced BST

Lowest Common Ancestor & Tree Paths

LCA in binary tree · LCA in BST · parent pointers · max path sum · root-to-leaf paths · binary lifting

COMMON ANCESTORS AND PATH SUMS

The **lowest common ancestor** (LCA) of p and q is the deepest node that has both in its subtree: where their paths from the root **split**. Path problems ask for the best sequence of connected nodes; when a path may bend through any node, compute the best **downward arm** from each child and combine them at the node.

OTHER LCA METHODS

- **Parent pointers**: put p's ancestors in a set, walk up from q
- **Binary lifting**: `up[k][v]` = 2^k-th ancestor; O(log n) per query
- `up[k][v] = up[k-1][ up[k-1][v] ]`
- Lift the deeper node to equal depth, then jump both

LCA IN A BINARY TREE

```
function lca(node, p, q)                    // O(n)
    if node = null or node = p or node = q
        return node
    left ← lca(node.left, p, q)
    right ← lca(node.right, p, q)
    if left ≠ null and right ≠ null
        return node                         // split here
    return if left ≠ null then left else right
```

LCA IN A BST

```
function lcaBST(node, p, q)                 // O(h)
    while node ≠ null
        if p.val < node.val and q.val < node.val
            node ← node.left                // both left
        else if p.val > node.val and q.val > node.val
            node ← node.right               // both right
        else
            return node                     // split
```

BINARY TREE MAXIMUM PATH SUM

```
function maxPathSum(root)
    best ← -∞
    function gain(node)             // best downward arm
        if node = null return 0
        l ← max(gain(node.left), 0)     // drop negatives
        r ← max(gain(node.right), 0)
        best ← max(best, node.val + l + r)
        return node.val + max(l, r)
    gain(root)
    return best
```

ALL ROOT-TO-LEAF PATHS

```
function paths(node, path, out)
    if node = null return
    path.append(node.val)
    if node.left = null and node.right = null
        out.append(copy(path))      // record a copy
    paths(node.left, path, out)
    paths(node.right, path, out)
    path.pop()                      // backtrack

// Path Sum II: also pass the remaining target
```

⚠ COMMON MISTAKES

- Returning `node.val + l + r` from `gain`: a parent can only extend one arm
- Appending `path` itself instead of a copy: every saved path ends up empty
- Using the BST shortcut on a plain binary tree: values aren't ordered

✓ TIPS & BEST PRACTICES

- Practice: LCA of BST, LCA of Binary Tree, Max Path Sum, Path Sum II, Binary Tree Paths
- Initialize `best` to `-∞`, not 0: all values may be negative
- Many queries on a static tree: precompute binary lifting

QUICK SUMMARY

- LCA — where p and q split
- BST LCA — walk until values straddle
- `gain` — best arm; `best` — bend
- append / recurse / pop — path backtracking

Heaps & Priority Queues

min vs max heap · array layout 2i+1 · sift up / down · heapify O(n) · push / pop O(log n) · heap sort

FAST ACCESS TO THE SMALLEST (OR LARGEST)

A **binary heap** is a complete binary tree where each parent is ≤ its children (**min-heap**) or ≥ them (**max-heap**). Because it's complete, it lives in a plain array with no pointers. It gives O(1) `peek` and O(log n) `push` / `pop`, which makes it the standard **priority queue**.

ARRAY LAYOUT (0-INDEXED)

- children of i: `2i + 1`, `2i + 2`
- parent of i: `(i - 1) div 2`
- root (min or max) at index 0
- Heapify an array bottom-up: **O(n)**

PUSH AND POP (MIN-HEAP)

```
function push(heap, x)                  // O(log n)
    heap.append(x)
    i ← length(heap) - 1
    while i > 0                         // sift up
        p ← (i - 1) div 2
        if heap[p] ≤ heap[i] break
        swap(heap, i, p)
        i ← p

function pop(heap)                      // O(log n)
    top ← heap[0]
    heap[0] ← heap[length(heap) - 1]    // move last to root
    heap.removeLast()
    siftDown(heap, 0)
    return top
```

SIFT DOWN AND HEAPIFY

```
function siftDown(heap, i)
    n ← length(heap)
    while true
        small ← i
        for each c in [2 * i + 1, 2 * i + 2]
            if c < n and heap[c] < heap[small]
                small ← c
        if small = i return
        swap(heap, i, small)
        i ← small

function heapify(arr)                   // O(n) total
    for i ← length(arr) div 2 - 1 down to 0
        siftDown(arr, i)
```

ARRAY ↔ TREE

```
index:  0  1  2  3  4  5
value: [1, 3, 2, 7, 4, 5]

          1
        /   \
       3     2
      / \   /
     7   4 5
```

PRIORITY QUEUES BY LANGUAGE

LANGUAGE	DEFAULT	MAX-HEAP TRICK
Python `heapq`	min	push `-x`
Java `PriorityQueue`	min	`Collections.reverseOrder()`
C++ `priority_queue`	**max**	`greater<int>` for min
C# `PriorityQueue<T,P>`	min priority	negate priority
JavaScript	none	write one or use a library

⚠ COMMON MISTAKES

- Assuming a heap is sorted: only the root is guaranteed; siblings are unordered
- Forgetting C++ `priority_queue` is a **max**-heap by default
- Searching a heap for an arbitrary value: that's O(n)

✓ TIPS & BEST PRACTICES

- Store tuples `(priority, tiebreak, item)` so items never get compared directly
- Heap sort: heapify, then pop n times; O(n log n), O(1) extra
- Need decrease-key? Push a new entry and skip stale ones on pop (Sheet 051)

QUICK SUMMARY

- parent ≤ children — min-heap
- `2i+1`, `2i+2`, `(i-1) div 2` — array links
- push / pop — O(log n)
- heapify — O(n)

Sheet 044

Heap Patterns: Top-K, Merge K & Two Heaps

k largest (min-heap of size k) · kth element · merge k sorted lists · two heaps median · task scheduling

HEAPS IN INTERVIEW PROBLEMS

Reach for a heap whenever you repeatedly need the **current** smallest or largest item while items keep arriving or changing. Three shapes cover most problems: a **size-k heap** for top-k, a heap of **list heads** to merge k sorted sources, and **two heaps** split at the median.

RECOGNIZE IT WHEN

- "k largest / smallest / closest / most frequent"
- "merge k sorted …"
- "running median", "data stream"
- Scheduling by earliest end time or highest count

TOP-K: MIN-HEAP OF SIZE K

```
function kthLargest(nums, k)        // O(n log k), O(k)
    heap ← new MinHeap()
    for each x in nums
        heap.push(x)
        if heap.size() > k
            heap.pop()              // drop the smallest
    return heap.peek()              // kth largest
// k smallest: use a max-heap of size k
```

PRACTICE

PROBLEM	HEAP HOLDS
Kth Largest in a Stream	k largest so far
K Closest Points	max-heap by distance, size k
Top K Frequent Elements	(count, value)
Last Stone Weight	max-heap of stones
Task Scheduler	max-heap by remaining count
Meeting Rooms II	min-heap of end times

MERGE K SORTED LISTS

```
function mergeK(lists)              // O(N log k)
    heap ← new MinHeap()            // (val, i, node)
    for i ← 0 to length(lists) - 1
        if lists[i] ≠ null
            heap.push((lists[i].val, i, lists[i]))
    dummy ← new Node(0)
    tail ← dummy
    while not heap.isEmpty()
        (v, i, node) ← heap.pop()
        tail.next ← node
        tail ← node
        if node.next ≠ null
            heap.push((node.next.val, i, node.next))
    return dummy.next
```

TWO HEAPS: RUNNING MEDIAN

```
class MedianFinder
    low ← new MaxHeap()             // smaller half
    high ← new MinHeap()            // larger half
    function add(x)
        low.push(x)
        high.push(low.pop())        // pass max up
        if high.size() > low.size()
            low.push(high.pop())    // keep low ≥ high
    function median()
        if low.size() > high.size()
            return low.peek()
        return (low.peek() + high.peek()) / 2
```

⚠ COMMON MISTAKES

- Using a max-heap of all n items for top-k: O(n log n) instead of O(n log k)
- Pushing nodes without a tiebreak index: equal values make the heap compare nodes
- Letting the two median heaps drift more than one apart in size

✓ TIPS & BEST PRACTICES

- Top-k by frequency with small counts: bucket sort is O(n) (Sheet 060)
- Quickselect finds the kth element in O(n) average (Sheet 061)
- Lazy deletion: mark removed items and skip them when they reach the top

QUICK SUMMARY

- size-k min-heap — k largest
- heap of heads — merge k sorted
- two heaps — median in O(log n)
- heap of end times — room counting

Sheet 045

Graph Basics & Representations

vertices / edges · directed vs undirected · weighted · adjacency list · adjacency matrix · edge list · in / out degree

NODES AND EDGES

A **graph** is a set of **vertices** (nodes) joined by **edges**. Edges can be **directed** (one-way) or **undirected**, and **weighted** or not. Trees, grids, maps, dependencies, and social networks are all graphs. Most problems hand you an **edge list**; your first step is usually to build an **adjacency list**.

VOCABULARY

- V = number of vertices, E = number of edges
- **Degree**: edges at a node (in / out for directed)
- **DAG**: directed graph with no cycles
- **Sparse**: E ≈ V; **dense**: E ≈ V²

EXAMPLE GRAPH

```
 0 --- 1
  \   /
   \ /
    2 --- 3

edges = [(0,1), (0,2), (1,2), (2,3)]

adjacency list:
 0: [1, 2]
 1: [0, 2]
 2: [0, 1, 3]
 3: [2]

degree(2) = 3
```

BUILDING REPRESENTATIONS

```
adj ← new Array(n)                    // adjacency list: O(V + E)
for i ← 0 to n − 1
    adj[i] ← []                       // separate list per node
for each (u, v) in edges
    adj[u].append(v)
    adj[v].append(u)                  // skip this line if directed

matrix ← grid(n, n, 0)                // adjacency matrix: O(V²)
for each (u, v) in edges
    matrix[u][v] ← 1
    matrix[v][u] ← 1

for each (u, v, w) in weightedEdges   // weighted: store pairs
    adj[u].append((v, w))

indeg ← new Array(n, 0)               // directed in-degree
for each (u, v) in edges
    indeg[v] ← indeg[v] + 1
```

REPRESENTATIONS COMPARED

	ADJACENCY LIST	MATRIX	EDGE LIST
space	O(V + E)	O(V²)	O(E)
is u–v an edge?	O(deg u)	O(1)	O(E)
list neighbors	O(deg u)	O(V)	O(E)
best for	most problems	dense graphs, Floyd	Kruskal, Bellman-Ford

GRAPH KINDS

KIND	MEANING
connected	path between every pair
cyclic / acyclic	has / has no cycle
tree	connected, acyclic, V − 1 edges
bipartite	2-colorable (Sheet 054)
implicit	neighbors computed (grids, words)

⚠ COMMON MISTAKES

- Creating `new Array(n, [])` : every node shares one neighbor list
- Adding only `u → v` for an undirected graph: traversal misses half the edges
- Using a V² matrix for V = 10^5: 10^{10} cells

✓ TIPS & BEST PRACTICES

- Nodes labeled by strings? Map them to `0…n-1` or use a map of lists
- Ask: directed? weighted? connected? self-loops or duplicate edges?
- Grids are graphs with 4 implicit neighbors per cell (Sheet 048)

QUICK SUMMARY

- adjacency list — default choice
- matrix — O(1) edge check, O(V²) space
- `(v, w)` pairs — weighted edges
- in-degree — dependencies (Sheet 049)

Depth-First Search

recursive DFS · iterative with stack · visited set · connected components · path exists · clone graph

GO DEEP, THEN BACKTRACK

Depth-first search follows one path as far as it can, then backs up and tries the next branch. A **visited** set prevents infinite loops on cycles. DFS answers reachability questions, counts **connected components**, detects cycles, and drives backtracking. Recursive or with an explicit stack, it runs in **O(V + E)** time and O(V) space.

RECOGNIZE IT WHEN

- "Is there a path from A to B?"
- "How many groups / islands / provinces?"
- "Copy / explore the whole structure"
- Order doesn't need to be nearest-first

RECURSIVE DFS AND COMPONENTS

```
function dfs(u, adj, visited)
    visited.add(u)
    for each v in adj[u]
        if not visited.has(v)
            dfs(v, adj, visited)

function countComponents(n, adj)
    visited ← new Set()
    count ← 0
    for u ← 0 to n − 1
        if not visited.has(u)
            dfs(u, adj, visited)      // one component
            count ← count + 1
    return count
```

ITERATIVE DFS (EXPLICIT STACK)

```
function dfsIterative(start, adj)
    visited ← new Set([start])
    stack ← [start]
    while not stack.isEmpty()
        u ← stack.pop()
        visit(u)
        for each v in adj[u]
            if not visited.has(v)
                visited.add(v)
                stack.push(v)

// same as BFS, but a stack instead of a queue
// avoids recursion-depth limits
```

CLONE GRAPH

```
function cloneGraph(node)
    copies ← new Map()                // original → clone
    function clone(n)
        if copies.has(n) return copies[n]
        c ← new GraphNode(n.val)
        copies[n] ← c                 // register BEFORE recursing
        for each nb in n.neighbors
            c.neighbors.append(clone(nb))
        return c
    return if node = null then null else clone(node)
```

PRACTICE

PROBLEM	DFS ROLE
Find if Path Exists	reachability
Number of Provinces	count components
Clone Graph	copy with a map
Keys and Rooms	reach all nodes?
All Paths Source → Target	DFS + backtracking

⚠ COMMON MISTAKES

- Marking visited **after** the recursive call: cycles recurse forever
- Forgetting the outer loop over all nodes: disconnected parts are skipped
- Recursing 10^5 levels deep in Python: switch to the iterative version

✓ TIPS & BEST PRACTICES

- The map in Clone Graph doubles as the visited set
- Track the path (push before, pop after) when you need the route itself
- Reset `visited` between independent searches

QUICK SUMMARY

- `visited` — prevents revisits and loops
- outer loop — one DFS per component
- stack — iterative DFS
- O(V + E) — time

Breadth-First Search

queue · level-by-level · shortest path (unweighted) · multi-source BFS · mark visited on enqueue · word ladder

EXPLORE IN RINGS

Breadth-first search visits all nodes 1 step away, then 2 steps, and so on, using a **queue**. Because of that order, the first time BFS reaches a node is along a **shortest path** (fewest edges) in an **unweighted** graph. Start from several nodes at once (**multi-source**) to get the distance to the nearest source.

RECOGNIZE IT WHEN

- "Minimum number of steps / moves / transformations"
- "Nearest", "shortest" with equal-cost moves
- "Level by level", "spreads each minute"
- States can be words, boards, or (cell, keys) pairs

SHORTEST PATH (UNWEIGHTED)

```
function shortestPath(adj, src, dst)
    dist ← new Map()
    dist[src] ← 0
    queue ← new Queue([src])
    while not queue.isEmpty()
        u ← queue.dequeue()
        if u = dst return dist[u]
        for each v in adj[u]
            if not dist.has(v)   // mark on enqueue
                dist[v] ← dist[u] + 1
                queue.enqueue(v)
    return -1                    // unreachable

// multi-source: put every source in the
// queue with dist 0 before the loop
```

WORD LADDER (IMPLICIT GRAPH)

```
function ladderLength(begin, end, words)
    wordSet ← toSet(words)
    queue ← new Queue([(begin, 1)])
    seen ← new Set([begin])
    while not queue.isEmpty()
        (w, steps) ← queue.dequeue()
        if w = end return steps
        for i ← 0 to length(w) - 1
            for each c in "abcdefghijklmnopqrstuvwxyz"
                nxt ← w[0..i-1] + c + w[i+1..]
                if wordSet.has(nxt) and not seen.has(nxt)
                    seen.add(nxt)
                    queue.enqueue((nxt, steps + 1))
    return 0
```

BFS VS DFS

	BFS	DFS
structure	queue	stack / recursion
shortest path (unweighted)	yes	no
memory	O(widest level)	O(deepest path)
finds	nearest first	any path, components
weighted edges	no: use Dijkstra (051)	no

PRACTICE

PROBLEM	STATE
Binary Tree Level Order	node
Rotting Oranges	cell (multi-source)
Word Ladder	word
Open the Lock	4-digit string
Shortest Path in Binary Matrix	cell, 8 directions

⚠ COMMON MISTAKES

- Marking visited on dequeue: the same node is enqueued many times
- Using BFS distances on weighted graphs: fewest edges ≠ lowest cost
- Rebuilding strings for each neighbor in Word Ladder without a set lookup

✓ TIPS & BEST PRACTICES

- Store `(node, distance)` in the queue or keep a `dist` map
- Bidirectional BFS (from both ends) cuts the search on huge graphs
- Process one level at a time with `count ← length(queue)`

QUICK SUMMARY

- queue — nearest first
- mark on enqueue — no duplicates
- multi-source — all starts at level 0
- O(V + E) — time

Grid Graphs

4 / 8 directions array · bounds check · number of islands · flood fill · rotting oranges · in-place visited marking

MATRICES AS GRAPHS

In a grid, each cell is a node and its **4 neighbors** (up, down, left, right) are its edges. A **direction array** keeps neighbor code short, a **bounds check** keeps it safe, and you can often mark visited cells **in place** instead of using a set. DFS flood fill counts regions; BFS spreads in rings for minimum steps.

RECOGNIZE IT WHEN

- 2D board of `'1'` / `'0'`, walls, water, oranges
- "Islands", "regions", "flood fill" → DFS or BFS
- "Minimum minutes / steps" → BFS
- "Reach the border / ocean" → start from the edges

NUMBER OF ISLANDS (DFS FLOOD FILL)

```
DIRS ← [(1, 0), (-1, 0), (0, 1), (0, -1)]

function numIslands(grid)
    (R, C) ← (rows(grid), cols(grid))
    function sink(r, c)
        if r < 0 or r ≥ R or c < 0 or c ≥ C
            return                  // off grid
        if grid[r][c] ≠ '1' return  // water/seen
        grid[r][c] ← '0'            // mark seen
        for each (dr, dc) in DIRS
            sink(r + dr, c + dc)
    count ← 0
    for r ← 0 to R - 1
        for c ← 0 to C - 1
            if grid[r][c] = '1'
                sink(r, c)
                count ← count + 1
    return count
```

ROTTING ORANGES (MULTI-SOURCE BFS)

```
function orangesRotting(grid)
    queue ← new Queue()
    fresh ← 0
    for each cell (r, c)
        if grid[r][c] = 2 queue.enqueue((r, c))
        if grid[r][c] = 1 fresh ← fresh + 1
    minutes ← 0
    while fresh > 0 and not queue.isEmpty()
        repeat length(queue) times     // one minute
            (r, c) ← queue.dequeue()
            for each (dr, dc) in DIRS
                (nr, nc) ← (r + dr, c + dc)
                if inBounds(nr, nc) and grid[nr][nc] = 1
                    grid[nr][nc] ← 2
                    fresh ← fresh - 1
                    queue.enqueue((nr, nc))
        minutes ← minutes + 1
    return if fresh = 0 then minutes else -1
```

DIRECTION ARRAYS

MOVES	DIRS
4-way	(1,0) (−1,0) (0,1) (0,−1)
8-way	add (1,1) (1,−1) (−1,1) (−1,−1)
knight	(±1,±2) (±2,±1)
cell id	`r * C + c` (for sets / union-find)

PRACTICE

PROBLEM	APPROACH
Flood Fill / Max Area of Island	DFS
Rotting Oranges, Walls and Gates	multi-source BFS
Pacific Atlantic Water Flow	DFS inward from each ocean
Surrounded Regions	save border-connected cells first

⚠ COMMON MISTAKES

- Checking `grid[r][c]` before the bounds check: index error at the edges
- Mixing up rows and columns: `grid[r][c]`, `R = rows`, `C = cols`
- Modifying the input when the problem says not to: copy it or use a visited set

✓ TIPS & BEST PRACTICES

- Reverse the search: start from borders or targets instead of every cell
- Count fresh cells first so you can stop early and detect -1
- Deep DFS on a 1000×1000 grid can overflow the stack; BFS avoids that

QUICK SUMMARY

- `DIRS` — neighbor offsets
- bounds check — first, always
- in-place marking — O(1) visited
- multi-source BFS — spreading over time

Topological Sort

DAG · Kahn's algorithm (in-degree) · DFS postorder · course schedule · cycle detection · alien dictionary

ORDERING DEPENDENCIES

A **topological order** lists the nodes of a directed graph so every edge `u → v` has u before v: a valid order to take courses or build modules. It exists only for a **DAG** (no cycles), so the same algorithms also **detect cycles**. **Kahn's algorithm** repeatedly removes nodes with in-degree 0; the DFS version outputs nodes in reverse finishing order.

RECOGNIZE IT WHEN

- Prerequisites, dependencies, build order
- "Can all tasks be finished?" (cycle check)
- Derive an order from pairwise rules (Alien Dictionary)
- Both methods: **O(V + E)**

KAHN'S ALGORITHM (BFS)

```
function topoSort(n, edges)        // (u, v): u before v
    adj ← n empty lists
    indeg ← new Array(n, 0)
    for each (u, v) in edges
        adj[u].append(v)
        indeg[v] ← indeg[v] + 1
    queue ← nodes with indeg = 0
    order ← []
    while not queue.isEmpty()
        u ← queue.dequeue()
        order.append(u)
        for each v in adj[u]
            indeg[v] ← indeg[v] - 1
            if indeg[v] = 0
                queue.enqueue(v)
    if length(order) < n
        return null                // cycle
    return order
```

DFS WITH THREE STATES

```
function topoDFS(n, adj)
    state ← new Array(n, 0)  // 0 new · 1 active · 2 done
    order ← []
    function visit(u)
        if state[u] = 1 return false   // back edge: cycle
        if state[u] = 2 return true
        state[u] ← 1
        for each v in adj[u]
            if not visit(v) return false
        state[u] ← 2
        order.append(u)                // after descendants
        return true
    for u ← 0 to n - 1
        if not visit(u) return null
    return reverse(order)
```

PRACTICE

PROBLEM	TWIST
Course Schedule	just detect a cycle
Course Schedule II	return the order
Alien Dictionary	edge from first differing letters of adjacent words
Parallel Courses	BFS levels = semesters
Minimum Height Trees	peel leaves layer by layer
Sequence Reconstruction	unique iff queue size is always 1

⚠ COMMON MISTAKES

- Reversing edge direction: `[course, prereq]` means `prereq → course`
- Using a plain visited set for directed cycle detection: needs the "active" state
- Forgetting to return null / false when not every node was output

✓ TIPS & BEST PRACTICES

- Kahn's is easier to get right and handles "levels" naturally
- Use a min-heap instead of a queue for the lexicographically smallest order
- Alien Dictionary: a longer word before its own prefix is invalid

QUICK SUMMARY

- in-degree 0 — ready to take
- `length(order) < n` — cycle
- 3 states — DFS cycle detection
- reverse postorder — DFS topo order

Union-Find (Disjoint Set)

parent array · find · path compression · union by rank / size · component count · redundant connection

TRACKING CONNECTED GROUPS

Union-Find (disjoint set union) keeps items in groups and answers "are a and b connected?" while edges are added. Each group is a tree; `find` returns its root, `union` links two roots. With **path compression** and **union by size/rank**, each operation is nearly O(1): O(α(n)), where α grows so slowly it's under 5 for any practical n.

RECOGNIZE IT WHEN

- Edges arrive one at a time; ask about connectivity
- "Number of connected components / provinces"
- "Which edge creates a cycle?" (redundant connection)
- Grouping equivalent items (accounts, synonyms)

UNION-FIND WITH BOTH OPTIMIZATIONS

```
class UnionFind
    function init(n)
        this.parent ← [0, 1, …, n - 1]      // each node is its own root
        this.size ← new Array(n, 1)
        this.count ← n                      // number of groups
    function find(x)
        while this.parent[x] ≠ x
            this.parent[x] ← this.parent[this.parent[x]]   // path halving
            x ← this.parent[x]
        return x
    function union(a, b)
        (ra, rb) ← (this.find(a), this.find(b))
        if ra = rb return false             // already connected
        if this.size[ra] < this.size[rb]
            (ra, rb) ← (rb, ra)             // smaller tree goes under
        this.parent[rb] ← ra
        this.size[ra] ← this.size[ra] + this.size[rb]
        this.count ← this.count - 1
        return true
```

CODE EXPLAINED

- → `parent[x] = x` — x is a root
- → path halving — point to grandparent while climbing
- → union by size — keeps trees shallow
- → `union` returns false — the edge closes a cycle
- → `count` — components left

REDUNDANT CONNECTION

```
function findRedundant(edges)    // nodes 1..n
    uf ← new UnionFind(length(edges) + 1)
    for each (u, v) in edges
        if not uf.union(u, v)
            return (u, v)    // closes a cycle
    return null
```

COST PER OPERATION

VERSION	FIND / UNION
naive	O(n)
union by size only	O(log n)
path compression only	O(log n) amortized
both	O(α(n)) ≈ O(1)

PRACTICE

PROBLEM
Number of Provinces
Graph Valid Tree
Accounts Merge
Number of Islands II
Kruskal's MST (053)

⚠ COMMON MISTAKES

- Comparing `parent[a] = parent[b]` instead of `find(a) = find(b)`
- Linking `a` under `b` instead of root under root: breaks the trees
- Off-by-one with 1-indexed nodes: allocate `n + 1` slots

✓ TIPS & BEST PRACTICES

- Graph Valid Tree: exactly n - 1 edges **and** no union returns false
- Map strings or grid cells (`r * C + c`) to integer ids first
- Union-Find can't split groups; for deletions process edges in reverse

QUICK SUMMARY

- `find` — root with path compression
- `union` — attach smaller root under larger
- false from `union` — cycle
- O(α(n)) — effectively constant

Shortest Paths: Dijkstra

non-negative weights · min-heap · relaxation · dist array · stale-entry skip · O((V + E) log V) · network delay time

CHEAPEST PATHS WITH NON-NEGATIVE WEIGHTS

Dijkstra's algorithm finds the lowest-cost path from one source to every node when no edge is negative. It always expands the **closest unfinished** node (via a min-heap); once popped, that node's distance is final. **Relaxing** an edge means improving `dist[v]` through u. Instead of decrease-key, push duplicates and **skip stale** heap entries.

RECOGNIZE IT WHEN

- Weighted edges, all ≥ 0
- "Minimum cost / time / effort" from a start
- Network delay, flight prices without stop limits
- Grid where moves have different costs

DIJKSTRA WITH A MIN-HEAP

```
function dijkstra(n, adj, src)        // adj[u] = [(v, w), …], w ≥ 0
    dist ← new Array(n, ∞)
    prev ← new Array(n, null)
    dist[src] ← 0
    heap ← new MinHeap([(0, src)])    // (distance, node)
    while not heap.isEmpty()
        (d, u) ← heap.pop()
        if d > dist[u] continue       // stale entry: skip
        for each (v, w) in adj[u]
            if d + w < dist[v]        // relax
                dist[v] ← d + w
                prev[v] ← u
                heap.push((dist[v], v))
    return (dist, prev)

function pathTo(prev, t)              // walk predecessors back
    path ← []
    while t ≠ null
        path.append(t)
        t ← prev[t]
    return reverse(path)
```

CODE EXPLAINED

- → `dist` — best known cost; ∞ = unreached
- → heap — always pop the smallest tentative distance
- → stale check — node already finalized cheaper
- → relax — `d + w < dist[v]`
- → `prev` — rebuild the actual route
- → Network Delay Time: answer is `max(dist)` or -1

IMPLEMENTATIONS

PRIORITY STRUCTURE	TIME	BEST WHEN
`array scan`	O(V²)	dense graphs
`binary heap`	O((V + E) log V)	typical sparse graphs
`Fibonacci heap`	O(E + V log V)	theory

VARIANTS

PROBLEM	CHANGE
`Network Delay Time`	plain Dijkstra, take max
`Path with Max Probability`	max-heap, multiply
`Path With Minimum Effort`	cost = max edge on path
`Swim in Rising Water`	cost = max cell on path
`Cheapest Flights (k stops)`	Bellman-Ford (052)

⚠ COMMON MISTAKES

- Running Dijkstra with negative edges: finalized nodes can later get cheaper
- Skipping the stale check: correct but much slower on dense graphs
- Marking visited on push instead of on pop: can lock in a worse distance

✓ TIPS & BEST PRACTICES

- Stop early when you pop the target if you only need one distance
- For a grid, nodes are cells and `w` is the cell's cost
- Equal weights everywhere? Plain BFS is enough

QUICK SUMMARY

- min-heap — closest first
- relax — `dist[v] ← d + w`
- skip if `d > dist[u]` — stale
- O((V + E) log V)

Bellman-Ford, Floyd-Warshall & 0-1 BFS

negative edges · negative cycle check · k-stops limit · all-pairs DP · 0-1 BFS with deque

BEYOND DIJKSTRA

Bellman-Ford relaxes every edge V - 1 times, so it handles **negative weights**, detects **negative cycles**, and can cap the number of edges used ("at most k stops"). **Floyd-Warshall** computes **all-pairs** shortest paths with a triple loop. **0-1 BFS** solves graphs whose weights are only 0 or 1 in O(V + E) with a deque.

WHICH ALGORITHM?

GRAPH	USE	TIME
unweighted	BFS	O(V + E)
weights 0 / 1	0-1 BFS	O(V + E)
weights ≥ 0	Dijkstra	O((V+E) log V)
negatives / ≤ k edges	Bellman-Ford	O(V · E)
all pairs, V ≤ 400	Floyd-Warshall	O(V³)

BELLMAN-FORD

```
function bellmanFord(n, edges, src)
    dist ← new Array(n, ∞)
    dist[src] ← 0
    repeat n - 1 times              // ≤ n-1 edges per path
        for each (u, v, w) in edges
            if dist[u] + w < dist[v]
                dist[v] ← dist[u] + w
    for each (u, v, w) in edges     // one extra round
        if dist[u] + w < dist[v]
            return null             // negative cycle
    return dist
```

CHEAPEST FLIGHTS WITHIN K STOPS

```
function cheapest(n, flights, src, dst, k)
    dist ← new Array(n, ∞)
    dist[src] ← 0
    repeat k + 1 times              // k stops = k+1 edges
        next ← copy(dist)           // read last round only
        for each (u, v, w) in flights
            if dist[u] + w < next[v]
                next[v] ← dist[u] + w
        dist ← next
    return if dist[dst] = ∞ then -1 else dist[dst]
```

FLOYD-WARSHALL (ALL PAIRS)

```
// d[i][j] = edge weight or ∞; d[i][i] = 0
for k ← 0 to n - 1                  // allow k as a stop
    for i ← 0 to n - 1
        for j ← 0 to n - 1
            if d[i][k] + d[k][j] < d[i][j]
                d[i][j] ← d[i][k] + d[k][j]
// any d[i][i] < 0 → negative cycle
```

0-1 BFS (DEQUE)

```
function zeroOneBFS(adj, src)
    dist ← new Array(n, ∞)
    dist[src] ← 0
    dq ← new Deque([src])
    while not dq.isEmpty()
        u ← dq.popFront()
        for each (v, w) in adj[u]  // w is 0 or 1
            if dist[u] + w < dist[v]
                dist[v] ← dist[u] + w
                if w = 0 dq.pushFront(v)
                else dq.pushBack(v)
    return dist
```

⚠ COMMON MISTAKES

- k-stops with in-place updates: one round can chain several edges
- Putting `k` in an inner loop of Floyd-Warshall: it must be the **outer** loop
- Treating `∞ + w` as smaller than ∞ in languages using a big int sentinel

✓ TIPS & BEST PRACTICES

- Use a large finite sentinel (e.g. 10^{18}) and check `dist[u] ≠ ∞` before adding
- Floyd-Warshall also answers reachability (transitive closure) with booleans
- 0-1 BFS: "min walls to break", "min arrow changes" in grids

QUICK SUMMARY

- Bellman-Ford — V - 1 rounds, negatives OK
- copy per round — edge-count limit
- Floyd — k outermost, O(V³)
- 0-1 BFS — front for 0, back for 1

Minimum Spanning Trees

Kruskal · Prim · cut property · union-find use · min cost to connect points · dense vs sparse choice

CONNECT EVERYTHING FOR THE LEAST COST

A **minimum spanning tree** (MST) connects all V vertices of a weighted, undirected graph with V − 1 edges and the smallest total weight. Both classic algorithms are greedy and rest on the **cut property**: the cheapest edge crossing any split of the vertices is safe to include. **Kruskal** adds edges cheapest-first; **Prim** grows one tree outward.

RECOGNIZE IT WHEN

- "Connect all points / cities / houses at minimum cost"
- Undirected, weighted, every node must be reached
- No start or end node (that would be shortest path)
- MST is not unique when weights tie, but its cost is

KRUSKAL (SORT + UNION-FIND)

```
function kruskal(n, edges)          // edges: (w, u, v)
    sort(edges) by w
    uf ← new UnionFind(n)           // Sheet 050
    (total, used) ← (0, 0)
    for each (w, u, v) in edges
        if uf.union(u, v)           // joins 2 parts
            total ← total + w
            used ← used + 1
            if used = n - 1 break
    if used < n - 1
        return -1                   // disconnected
    return total
```

PRIM (MIN-HEAP)

```
function prim(n, adj)               // adj[u] = [(v, w)]
    inTree ← new Array(n, false)
    heap ← new MinHeap([(0, 0)])    // (weight, node)
    (total, count) ← (0, 0)
    while not heap.isEmpty() and count < n
        (w, u) ← heap.pop()
        if inTree[u] continue       // stale
        inTree[u] ← true
        total ← total + w
        count ← count + 1
        for each (v, wv) in adj[u]
            if not inTree[v]
                heap.push((wv, v))
    return total
```

KRUSKAL VS PRIM

	KRUSKAL	PRIM
grows	a forest that merges	one tree
needs	edge list + union-find	adjacency + heap
time	O(E log E)	O(E log V); O(V²) with array
best for	sparse graphs	dense / complete graphs

PRACTICE

PROBLEM	NOTE
Min Cost to Connect All Points	complete graph: O(n²) Prim
Connecting Cities With Min Cost	Kruskal, check connected
Optimize Water Distribution	virtual node 0 = wells
Critical & Pseudo-Critical Edges	rerun MST with/without edge

⚠ COMMON MISTAKES

- Using an MST to answer shortest-path questions: different problems
- Running Prim from one node on a disconnected graph and not checking `count`
- Building all n² edges for 10³ points and then sorting: works, but Prim's O(n²) array is faster

✓ TIPS & BEST PRACTICES

- "Pay to build at a node" costs: add a virtual node with edges of that cost
- Maximum spanning tree: sort descending (or negate weights)
- Stop Kruskal once `n - 1` edges are used

QUICK SUMMARY

- MST — V − 1 edges, minimum total
- Kruskal — sort edges + union-find
- Prim — heap of crossing edges
- cut property — why greedy works

Bipartite Graphs & Cycle Detection

2-coloring · BFS / DFS coloring · undirected cycle (parent check) · directed cycle (3 colors) · is graph bipartite

TWO-COLORING AND FINDING CYCLES

A graph is **bipartite** if its nodes split into two groups with every edge going **between** them; equivalently, it can be **2-colored**, and it has **no odd-length cycle**. Cycle detection differs by edge type: in an **undirected** graph, any visited neighbor other than your parent closes a cycle; in a **directed** graph you need three states to find a **back edge**.

RECOGNIZE IT WHEN

- "Split into two teams / groups with no conflicts"
- "Is this graph a valid tree?" (connected + no cycle)
- "Can all courses be finished?" (directed cycle)
- "Which nodes are safe / never reach a cycle?"

BIPARTITE CHECK (BFS COLORING)

```
function isBipartite(n, adj)
    color ← new Array(n, -1)              // -1 = uncolored
    for s ← 0 to n - 1                    // every component
        if color[s] ≠ -1 continue
        color[s] ← 0
        queue ← new Queue([s])
        while not queue.isEmpty()
            u ← queue.dequeue()
            for each v in adj[u]
                if color[v] = -1
                    color[v] ← 1 - color[u]   // opposite color
                    queue.enqueue(v)
                else if color[v] = color[u]
                    return false              // edge inside one group
    return true
```

CODE EXPLAINED

- → colors 0 / 1 — the two groups
- → outer loop — graphs may be disconnected
- → `1 - color[u]` — neighbors get the other color
- → same color on both ends — odd cycle, not bipartite
- → DFS coloring works the same way

UNDIRECTED CYCLE (PARENT CHECK)

```
function hasCycle(u, parent, adj, seen)
    seen.add(u)
    for each v in adj[u]
        if not seen.has(v)
            if hasCycle(v, u, adj, seen)
                return true
        else if v ≠ parent
            return true        // not the edge we came on
    return false
```

DIRECTED CYCLE (THREE STATES)

```
// 0 = new, 1 = on current path, 2 = done
function hasCycleDir(u, adj, state)
    state[u] ← 1
    for each v in adj[u]
        if state[v] = 1
            return true        // back edge
        if state[v] = 0 and hasCycleDir(v, adj, state)
            return true
    state[u] ← 2
    return false
```

⚠ COMMON MISTAKES

- Using the undirected parent trick on a directed graph: misses or invents cycles
- Using a plain visited set for directed graphs: a node reached twice isn't a cycle
- Checking only from node 0: other components may hold the conflict

✓ TIPS & BEST PRACTICES

- Practice: Is Graph Bipartite, Possible Bipartition, Graph Valid Tree, Course Schedule
- Find Eventual Safe States: nodes that end in state 2 without hitting a cycle
- Undirected cycles are also easy with union-find (Sheet 050)

QUICK SUMMARY

- bipartite ⇔ 2-colorable ⇔ no odd cycle
- undirected — visited neighbor ≠ parent
- directed — edge into state 1
- loop over all nodes — disconnected graphs

Connectivity: SCCs, Bridges & Euler Paths

Kosaraju · Tarjan low-link · bridges · articulation points · Euler path · Hierholzer · reconstruct itinerary

DEEPER GRAPH STRUCTURE

Tarjan's low-link DFS records each node's discovery time and the lowest time reachable through its subtree. An edge u–v is a **bridge** if v's subtree can't reach above u. **Kosaraju** finds **strongly connected components** (SCCs) with two DFS passes. **Hierholzer** builds an **Euler path** that uses every edge exactly once.

QUICK FACTS

- Bridge: `low[v] > disc[u]`
- Articulation point: `low[v] ≥ disc[u]` (root: ≥ 2 children)
- SCC graph collapsed to nodes is a DAG
- All three run in **O(V + E)**

BRIDGES (TARJAN LOW-LINK)

```
function bridges(n, adj)
    disc ← new Array(n, -1)
    low ← new Array(n, 0)
    (time, out) ← (0, [])
    function dfs(u, parent)
        disc[u] ← time
        low[u] ← time
        time ← time + 1
        for each v in adj[u]
            if v = parent continue
            if disc[v] = -1
                dfs(v, u)
                low[u] ← min(low[u], low[v])
                if low[v] > disc[u]
                    out.append((u, v))    // bridge
            else
                low[u] ← min(low[u], disc[v])
    for u ← 0 to n - 1
        if disc[u] = -1 dfs(u, -1)
    return out
```

SCCS (KOSARAJU)

```
function kosaraju(n, adj)
    order ← []                    // 1) finish order
    seen ← new Set()
    function dfs1(u)
        seen.add(u)
        for each v in adj[u]
            if not seen.has(v) dfs1(v)
        order.append(u)
    for u ← 0 to n - 1
        if not seen.has(u) dfs1(u)
    radj ← reversed edges         // 2) transpose
    comp ← new Array(n, -1)
    c ← 0
    function dfs2(u)
        comp[u] ← c
        for each v in radj[u]
            if comp[v] = -1 dfs2(v)
    for each u in reverse(order)     // 3)
        if comp[u] = -1
            dfs2(u)
            c ← c + 1
    return comp
```

EULER PATH (HIERHOLZER)

```
function findItinerary(tickets)   // Reconstruct Itinerary
    adj ← map airport → min-heap of destinations
    route ← []
    function visit(a)
        while adj[a] is not empty
            visit(adj[a].pop())   // smallest first
        route.append(a)           // add when stuck
    visit("JFK")
    return reverse(route)
```

WHEN AN EULER PATH EXISTS

GRAPH	CONDITION*
undirected circuit	every degree even
undirected path	0 or 2 odd-degree nodes
directed circuit	in = out everywhere
directed path	one start (out − in = 1), one end
* plus	all edges in one connected part

⚠ COMMON MISTAKES

- Skipping the parent edge by node, not edge id: breaks with parallel edges
- Using `low[v]` instead of `disc[v]` for back edges in the bridge DFS
- Appending to the route before exploring in Hierholzer: wrong order

✓ TIPS & BEST PRACTICES

- Practice: Critical Connections (bridges), Reconstruct Itinerary, Valid Arrangement of Pairs
- Tarjan's SCC does it in one pass with a stack; Kosaraju is easier to remember
- Use iterative DFS for V ≈ 10^5 to avoid stack overflow

QUICK SUMMARY

- `disc` / `low` — Tarjan's low-link
- `low[v] > disc[u]` — bridge
- Kosaraju — finish order, transpose, DFS
- Hierholzer — post-order, then reverse

Sheet 056

Binary Search Fundamentals

sorted precondition · lo / hi / mid · overflow-safe mid · lo ≤ hi vs lo < hi · lower bound · upper bound · first / last occurrence

HALVE THE SEARCH SPACE

Binary search finds a target in a **sorted** array by comparing with the middle and discarding half each step: **O(log n)**. Most bugs come from mixing interval styles, so pick one template and keep its rules. The most useful forms return the **boundary**: the first index where a condition becomes true (`lower bound` , `upper bound`).

GROUND RULES

- Data must be sorted, or the condition must be **monotonic**
- Use `mid ← lo + (hi - lo) div 2` (no overflow)
- Every branch must shrink the range, or it loops forever
- 10^9 items → about 30 steps

BOUNDARY SEARCH (HALF-OPEN [LO, HI))

```
function lowerBound(arr, x)      // first i: arr[i] ≥ x
    (lo, hi) ← (0, length(arr))
    while lo < hi
        mid ← lo + (hi - lo) div 2
        if arr[mid] < x
            lo ← mid + 1
        else
            hi ← mid
    return lo                        // length(arr) if none

function upperBound(arr, x)      // first i: arr[i] > x
    (lo, hi) ← (0, length(arr))
    while lo < hi
        mid ← lo + (hi - lo) div 2
        if arr[mid] ≤ x
            lo ← mid + 1
        else
            hi ← mid
    return lo
```

EXACT MATCH (CLOSED [LO, HI])

```
function search(arr, x)
    (lo, hi) ← (0, length(arr) - 1)
    while lo ≤ hi
        mid ← lo + (hi - lo) div 2
        if arr[mid] = x
            return mid
        else if arr[mid] < x
            lo ← mid + 1
        else
            hi ← mid - 1
    return -1

// arr = [1, 2, 2, 2, 5]
// lowerBound(2) = 1   upperBound(2) = 4
// first 2 at 1, last 2 at 4 - 1 = 3
// count of 2s = 4 - 1 = 3
```

TWO TEMPLATES

	CLOSED [LO, HI]	HALF-OPEN [LO, HI)
start	`hi ← n - 1`	`hi ← n`
loop	`while lo ≤ hi`	`while lo < hi`
shrink	`lo ← mid + 1` / `hi ← mid - 1`	`lo ← mid + 1` / `hi ← mid`
ends with	`lo = hi + 1`	`lo = hi` (the boundary)
best for	find an exact value	first true / insert position

LIBRARY VERSIONS

LANGUAGE	LOWER / UPPER
Python	`bisect_left` / `bisect_right`
C++	`lower_bound` / `upper_bound`
Java	`Arrays.binarySearch` (-insert - 1)
C#	`Array.BinarySearch` (~insert)
JavaScript	none: write it

⚠ COMMON MISTAKES

- `hi ← mid` with `while lo ≤ hi` : infinite loop when `lo = hi`
- `(lo + hi) / 2` overflowing 32-bit ints, or using float division
- Returning `lo` without checking `lo < n and arr[lo] = x`

✓ TIPS & BEST PRACTICES

- Search Insert Position = `lowerBound`
- First / last occurrence = `lowerBound(x)` / `upperBound(x) - 1`
- Practice: Binary Search, Search Insert Position, First and Last Position, Sqrt(x)

QUICK SUMMARY

- `lowerBound` — first ≥ x
- `upperBound` — first > x
- half-open — boundary searches
- O(log n) — sorted input required

Binary Search Variations & Search on Answer

rotated array · find peak · 2D matrix · monotonic predicate · search on answer (Koko, ship capacity) · real-valued bisection

BINARY SEARCH BEYOND SORTED ARRAYS

Binary search works whenever you can look at the middle and **rule out half**. That includes rotated arrays (one half is always sorted), peaks (walk uphill), and most powerfully **search on the answer**: if a yes/no test `ok(x)` flips from false to true exactly once as x grows, binary search over x finds the smallest x that works.

RECOGNIZE IT WHEN

- "Minimum capacity / speed / days so that …"
- "Maximize the minimum" or "minimize the maximum"
- Answer range is huge (up to 10^9) but checking one value is O(n)
- Sorted-but-rotated input, or "find any peak"

SEARCH IN ROTATED SORTED ARRAY

```
function searchRotated(arr, t)
    (lo, hi) ← (0, length(arr) − 1)
    while lo ≤ hi
        mid ← lo + (hi − lo) div 2
        if arr[mid] = t return mid
        if arr[lo] ≤ arr[mid]          // left half sorted
            if arr[lo] ≤ t and t < arr[mid]
                hi ← mid − 1
            else
                lo ← mid + 1
        else                           // right half sorted
            if arr[mid] < t and t ≤ arr[hi]
                lo ← mid + 1
            else
                hi ← mid − 1
    return −1
```

FIND PEAK ELEMENT

```
function findPeak(arr)
    (lo, hi) ← (0, length(arr) − 1)
    while lo < hi
        mid ← lo + (hi − lo) div 2
        if arr[mid] < arr[mid + 1]
            lo ← mid + 1           // uphill right
        else
            hi ← mid               // mid may be peak
    return lo

// min of rotated array: same shape,
// compare arr[mid] > arr[hi] → lo ← mid + 1
```

SEARCH ON THE ANSWER (KOKO EATING BANANAS)

```
function minEatingSpeed(piles, h)
    function canFinish(k)                // monotonic in k
        hours ← 0
        for each p in piles
            hours ← hours + (p + k − 1) div k
        return hours ≤ h
    (lo, hi) ← (1, max(piles))
    while lo < hi                        // first k that works
        mid ← lo + (hi − lo) div 2
        if canFinish(mid)
            hi ← mid
        else
            lo ← mid + 1
    return lo
```

MORE VARIATIONS

PROBLEM	TRICK
Search a 2D Matrix	index i → `(i div C, i mod C)`
Ship Within D Days	answer in `[max, sum]`
Split Array Largest Sum	minimize the max
Median of Two Arrays	binary search the cut
Real-valued √x	repeat 60× on `(lo+hi)/2`

⚠ COMMON MISTAKES

- Wrong answer range: Koko's low end is 1, not 0 (division by zero)
- Testing `t < arr[mid]` without the `arr[lo] ≤ t` side in rotated search
- Real-valued loops with `while lo < hi`: floats may never meet; use a fixed count

✓ TIPS & BEST PRACTICES

- Write `ok(x)` first and confirm it's monotonic before coding the search
- "Minimize" → first true; "maximize" → last true (flip the test)
- Duplicates in rotated arrays: shrink `lo` and `hi` by one when ends equal

QUICK SUMMARY

- rotated — one half is always sorted
- peak — move toward the bigger neighbor
- search on answer — binary search over x
- monotonic `ok(x)` — the requirement

Elementary Sorts

bubble · selection · insertion · stable vs unstable · in-place · nearly sorted input

THE SIMPLE O(N²) SORTS

Bubble, selection, and insertion sort are slow on big inputs but worth knowing: they're the base cases inside real library sorts, they introduce **stability** and **in-place** sorting, and their inner loops show up in interview problems. **Insertion sort** is the useful one: O(n) on nearly sorted data and fastest for tiny arrays.

TERMS

- **Stable**: equal items keep their original order
- **In-place**: O(1) extra memory
- **Adaptive**: faster when input is nearly sorted
- Library sorts switch to insertion sort below ~16–32 items

BUBBLE SORT

```
function bubbleSort(a)          // swap adjacent pairs
    n ← length(a)
    for i ← 0 to n − 2
        swapped ← false
        for j ← 0 to n − 2 − i
            if a[j] > a[j + 1]
                swap(a, j, j + 1)
                swapped ← true
        if not swapped
            break               // sorted: O(n) best
// after pass i, the largest i+1 items are final
```

SELECTION SORT

```
function selectionSort(a)       // pick min of the rest
    n ← length(a)
    for i ← 0 to n − 2
        m ← i
        for j ← i + 1 to n − 1
            if a[j] < a[m]
                m ← j
        swap(a, i, m)           // only n − 1 swaps
// unstable: the long swap can jump
// over an equal item
```

INSERTION SORT

```
function insertionSort(a)       // sorted prefix
    for i ← 1 to length(a) − 1
        x ← a[i]
        j ← i − 1
        while j ≥ 0 and a[j] > x
            a[j + 1] ← a[j]     // shift right
            j ← j − 1
        a[j + 1] ← x            // into the gap
```

COMPARED

SORT	BEST	AVG / WORST	STABLE	SWAPS
bubble	O(n)	O(n²)	yes	O(n²)
selection	O(n²)	O(n²)	no	O(n)
insertion	O(n)	O(n²)	yes	O(n²) shifts
all three		space O(1)		

⚠ COMMON MISTAKES

- Using `a[j] ≥ x` in insertion sort: moves equal items and breaks stability
- Calling selection sort stable: its swaps reorder equal keys
- Writing these in an interview when `sort()` is allowed: use the library

✓ TIPS & BEST PRACTICES

- Insertion sort is ideal for streams of items arriving in nearly sorted order
- Selection sort minimizes writes: useful when writes are expensive
- Practice: Sort Colors (Dutch flag), Insertion Sort List, Sort an Array

QUICK SUMMARY

- bubble — adjacent swaps, early exit
- selection — min of the rest, fewest swaps
- insertion — shift into sorted prefix
- O(n²) — fine only for small n

Merge Sort & Quick Sort

THE DIVIDE-AND-CONQUER SORTS

Merge sort splits the array in half, sorts each half, then **merges** them: always O(n log n), stable, but needs O(n) extra space. **Quick sort partitions** around a pivot (smaller left, larger right) and recurses: O(n log n) on average, in place, but O(n²) if pivots are bad. A random pivot makes the worst case very unlikely.

QUICK FACTS

- Merge sort: best for linked lists and external (disk) sorting
- Quick sort: fastest in practice on arrays
- Merge step counts **inversions** for free
- Partition also powers quickselect (Sheet 061)

MERGE SORT

```
function mergeSort(a, lo, hi)      // sorts a[lo..hi]
    if lo ≥ hi return
    mid ← (lo + hi) div 2
    mergeSort(a, lo, mid)
    mergeSort(a, mid + 1, hi)
    merge(a, lo, mid, hi)

function merge(a, lo, mid, hi)
    tmp ← []
    (i, j) ← (lo, mid + 1)
    while i ≤ mid and j ≤ hi
        if a[i] ≤ a[j]             // ≤ keeps it stable
            tmp.append(a[i])
            i ← i + 1
        else
            tmp.append(a[j])       // inv += mid - i + 1
            j ← j + 1
    append a[i..mid] and a[j..hi] to tmp
    copy tmp into a[lo..hi]
```

QUICK SORT (LOMUTO PARTITION)

```
function quickSort(a, lo, hi)
    if lo ≥ hi return
    p ← partition(a, lo, hi)
    quickSort(a, lo, p - 1)
    quickSort(a, p + 1, hi)

function partition(a, lo, hi)
    swap(a, randomInt(lo, hi), hi) // random pivot
    pivot ← a[hi]
    i ← lo                         // next small slot
    for j ← lo to hi - 1
        if a[j] < pivot
            swap(a, i, j)
            i ← i + 1
    swap(a, i, hi)                 // pivot to place
    return i
```

MERGE VS QUICK

	MERGE SORT	QUICK SORT
time	O(n log n) always	O(n log n) avg · O(n²) worst
extra space	O(n)	O(log n) recursion
stable	yes	no
work happens	after recursion (merge)	before recursion (partition)
worst case trigger	none	sorted input + fixed pivot

PARTITION SCHEMES

SCHEME	NOTE
Lomuto	simple; pivot at the end
Hoare	two pointers inward; fewer swaps
3-way (Dutch flag)	<, =, > pivot; fast on duplicates
median-of-3	pivot = median of first, mid, last

⚠ COMMON MISTAKES

- Always picking the last element as pivot: sorted input becomes O(n²)
- Allocating a new array in every merge call: copy into one shared buffer
- Using `<` in the merge comparison: breaks stability

✓ TIPS & BEST PRACTICES

- Count of Smaller Numbers After Self and Reverse Pairs: count during merge
- Sort a linked list with merge sort: O(1) extra space, no random access needed
- Many duplicates? Use 3-way partitioning

QUICK SUMMARY

- merge sort — split, sort, merge; stable
- quick sort — partition, recurse; in place
- random pivot — avoids O(n²)
- merge step — counts inversions

Non-Comparison Sorts

counting sort · radix sort · bucket sort · O(n + k) · top-k frequent via buckets · when they beat O(n log n)

SORTING WITHOUT COMPARISONS

Comparison sorts can't beat O(n log n). When keys are **small integers** (or can be split into digits), you can place items directly by value instead. **Counting sort** tallies each value, **radix sort** runs a stable counting pass per digit, and **bucket sort** spreads items into ranges. They run in **O(n + k)**, where k is the value range or bucket count.

RECOGNIZE IT WHEN

- Values in a small range (`0…100` , letters, ages)
- Frequencies are bounded by n → bucket by frequency
- Fixed-width keys: digits, dates, IP addresses
- "Linear time" is required and keys are integers

COUNTING SORT

```
function countingSort(a, k)      // values 0..k-1
    count ← new Array(k, 0)
    for each x in a
        count[x] ← count[x] + 1
    i ← 0
    for v ← 0 to k - 1
        repeat count[v] times
            a[i] ← v
            i ← i + 1

// stable version (for records):
// prefix-sum the counts, then place
// items from right to left
```

RADIX SORT (LSD, BASE 10)

```
function radixSort(a)            // non-negative ints
    exp ← 1
    while max(a) div exp > 0
        buckets ← 10 empty lists
        for each x in a
            d ← (x div exp) mod 10
            buckets[d].append(x)    // stable
        a ← concat(buckets)
        exp ← exp * 10
    return a
// O(d · (n + 10)), d = digit count
```

BUCKET BY FREQUENCY: TOP K FREQUENT

```
function topKFrequent(nums, k)
    count ← frequency map of nums
    buckets ← n + 1 empty lists          // index = frequency
    for each (x, c) in count
        buckets[c].append(x)
    result ← []
    for f ← length(nums) down to 1
        for each x in buckets[f]
            result.append(x)
            if length(result) = k return result
```

COMPARED

SORT	TIME	SPACE	STABLE
counting	O(n + k)	O(k)	yes*
radix	O(d(n + b))	O(n + b)	yes
bucket	O(n) avg	O(n)	depends

* placing records

⚠ COMMON MISTAKES

- Counting sort on values up to 10^9: the count array is far too big
- Radix sort with an unstable per-digit pass: later digits scramble earlier work
- Negative numbers in radix sort: offset them or sort negatives separately

✓ TIPS & BEST PRACTICES

- Sort Colors and H-Index are counting-sort problems in disguise
- Maximum Gap: buckets by range give O(n) via the pigeonhole principle
- Letters: a count[26] array is a counting sort

QUICK SUMMARY

- counting — tally, then rewrite
- radix — stable pass per digit
- bucket — group by range or frequency
- O(n + k) — beats n log n for small k

Sheet 061

Custom Sorting & Quickselect

comparator contract · multi-key sort · stability · sort-then-scan · quickselect O(n) avg · kth largest

SORTING YOUR WAY, AND SELECTING WITHOUT SORTING

Real problems sort records by custom rules: by a field, descending, or by several keys. A **comparator** returns negative, zero, or positive; a **key function** returns a value to sort by (tuples give multi-key order). When you need only the **kth** element, **quickselect** partitions like quick sort but recurses into one side: O(n) on average.

RECOGNIZE IT WHEN

- "Sort by … then by …", custom ordering rules
- Sorting first makes a greedy or two-pointer scan possible
- "kth largest / smallest" with no other ordering needed
- "Largest number formed by …" (concatenation order)

COMPARATORS AND KEYS

```
// comparator: < 0 → a first, 0 → tie, > 0 → b first
sort(people, lambda a, b: compare(a.age, b.age))      // ascending
sort(people, lambda a, b: compare(b.score, a.score))  // descending
sortBy(people, lambda p: (p.last, p.first))           // multi-key tuple
sortBy(words, lambda w: (-count[w], w))     // freq desc, then A→Z

// Largest Number: put a before b if a+b > b+a
sort(strs, lambda a, b: compare(b + a, a + b))
// ["3","30","34","5","9"] → "9534330"

intervals ← sortBy(intervals, lambda iv: iv.start)   // sort, then scan
for each iv in intervals
    …merge / count overlaps (Sheet 069)
```

QUICKSELECT

```
function kthLargest(a, k)      // O(n) avg
    target ← length(a) - k     // sorted index
    (lo, hi) ← (0, length(a) - 1)
    while true
        p ← partition(a, lo, hi)  // 059
        if p = target
            return a[p]
        if p < target
            lo ← p + 1
        else
            hi ← p - 1
```

LIBRARY SORT STABILITY

LANGUAGE	SORT	STABLE?
Python	`sorted`, `list.sort` (Timsort)	yes
Java	`Arrays.sort` objects / primitives	yes / no
JavaScript	`Array.prototype.sort`	yes (ES2019+)
C++	`std::sort` / `std::stable_sort`	no / yes
C#	`Array.Sort` / LINQ `OrderBy`	no / yes

COMPARATOR CONTRACT

RULE	MEANING
consistent	same inputs → same answer
antisymmetric	`cmp(a,b) = -cmp(b,a)`
transitive	a<b and b<c → a<c
no subtraction	`a - b` can overflow

⚠ COMMON MISTAKES

- Returning a boolean from a comparator (JS, Java): sorting silently breaks
- `a.x - b.x` with large values: overflow flips the sign; use `compare`
- Quickselect with a fixed pivot on sorted input: degrades to O(n²)

✓ TIPS & BEST PRACTICES

- Negate numeric keys for descending order inside a tuple key
- Python 3 has no `cmp=`; wrap with `functools.cmp_to_key`
- Practice: Largest Number, Sort Characters by Frequency, Kth Largest, K Closest Points

QUICK SUMMARY

- comparator — negative / zero / positive
- key tuple — multi-key sort
- stable sort — sort by secondary key first
- quickselect — kth element in O(n) avg

Two Pointers

opposite ends · same direction (read / write) · sorted Two Sum · 3Sum · container with most water · remove duplicates in place

TWO INDEXES INSTEAD OF TWO LOOPS

Two pointers replace a nested O(n²) loop with two indexes that each move at most n times: O(n). **Opposite ends** pointers work on sorted data: move the side that can improve the answer. **Same-direction** pointers (read and write) filter or compact an array in place. Sorting first (O(n log n)) often unlocks the pattern.

RECOGNIZE IT WHEN

- Sorted array + pair/triplet with a target sum
- "In place", "O(1) extra space", remove / move items
- Palindromes, reversing, comparing from both ends
- Maximize area / width between two positions

OPPOSITE ENDS: TWO SUM (SORTED)

```
function twoSumSorted(arr, target)
    (l, r) ← (0, length(arr) − 1)
    while l < r
        s ← arr[l] + arr[r]
        if s = target
            return (l, r)
        if s < target
            l ← l + 1              // need bigger
        else
            r ← r − 1              // need smaller
    return null
```

SAME DIRECTION: REMOVE DUPLICATES

```
function removeDuplicates(arr)     // sorted input
    if arr is empty return 0
    w ← 1                          // next write slot
    for r ← 1 to length(arr) − 1
        if arr[r] ≠ arr[w − 1]
            arr[w] ← arr[r]        // keep it
            w ← w + 1
    return w                       // new length

// Move Zeroes: keep non-zeros, then fill
// the rest with 0
```

3SUM (SORT + ANCHOR + TWO POINTERS)

```
function threeSum(nums)                    // O(n²)
    sort(nums)
    out ← []
    for i ← 0 to length(nums) − 3
        if i > 0 and nums[i] = nums[i − 1]
            continue                       // skip duplicate anchor
        (l, r) ← (i + 1, length(nums) − 1)
        while l < r
            s ← nums[i] + nums[l] + nums[r]
            if s < 0
                l ← l + 1
            else if s > 0
                r ← r − 1
            else
                out.append((nums[i], nums[l], nums[r]))
                l ← l + 1
                while l < r and nums[l] = nums[l − 1]
                    l ← l + 1              // skip duplicate
                r ← r − 1
    return out
```

CONTAINER WITH MOST WATER

```
function maxArea(h)
    (l, r, best) ← (0, length(h) − 1, 0)
    while l < r
        area ← (r − l) * min(h[l], h[r])
        best ← max(best, area)
        if h[l] < h[r]
            l ← l + 1      // shorter side
        else               // limits area
            r ← r − 1
    return best
```

⚠ COMMON MISTAKES

- Using opposite-end pointers on unsorted data: the move logic has no basis
- Forgetting to skip duplicates in 3Sum: repeated triplets in the output
- Moving the taller wall in Container With Most Water: can never improve the area

✓ TIPS & BEST PRACTICES

- Practice: Valid Palindrome, Two Sum II, 3Sum, Container With Most Water, Move Zeroes
- Ask "which pointer's move can improve things?" and move only that one
- 4Sum = two nested anchors + two pointers: O(n³)

QUICK SUMMARY

- opposite ends — sorted pairs, O(n)
- read / write — in-place filtering
- sort + anchor — kSum
- each pointer moves ≤ n times

Sliding Window — Fixed Size

add right / drop left · window sum / average · max sum of size k · permutation in string · find all anagrams

A WINDOW THAT SLIDES ONE STEP AT A TIME

For questions about **every contiguous block of size k**, don't recompute each block. Keep a running summary of the current **window**, and as it slides right, **add** the new item and **remove** the one that fell off the left. Each step is O(1) (or O(26) for letter counts), so the whole scan is **O(n)** instead of O(n·k).

RECOGNIZE IT WHEN

- "Subarray / substring of length k"
- "Average / sum / max of every window"
- "Permutation / anagram of p inside s" (window = len(p))
- Window size is **fixed** by the problem

FIND ALL ANAGRAMS (FIXED WINDOW OF LETTER COUNTS)

```
function findAnagrams(s, p)
    k ← length(p)
    need ← letterCounts(p)                      // count[26]
    have ← new Array(26, 0)
    out ← []
    for r ← 0 to length(s) - 1
        have[idx(s[r])] ← have[idx(s[r])] + 1             // add right
        if r ≥ k
            have[idx(s[r - k])] ← have[idx(s[r - k])] - 1   // drop left
        if r ≥ k - 1 and have = need                // 26 compares: O(1)
            out.append(r - k + 1)                   // window start
    return out
```

CODE EXPLAINED

- → `idx(c)` — `code(c) - code('a')`
- → window is `s[r-k+1 .. r]`
- → drop `s[r - k]` once the window is full
- → `have = need` — compare two count arrays
- → Permutation in String: return true on first match

MAX SUM OF K CONSECUTIVE ITEMS

```
function maxSumK(arr, k)
    window ← 0
    for i ← 0 to k - 1
        window ← window + arr[i]
    best ← window
    for r ← k to length(arr) - 1
        window ← window + arr[r] - arr[r - k]
        best ← max(best, window)
    return best
```

PRACTICE

PROBLEM	WINDOW STATE
Maximum Average Subarray I	running sum
Max Vowels in Substring of Length k	vowel count
Permutation in String	count[26]
Find All Anagrams	count[26]
Sliding Window Maximum	monotonic deque (068)
Contains Duplicate II	set of last k values

⚠ COMMON MISTAKES

- Off-by-one on the drop index: the item leaving is `arr[r - k]`
- Recording a result before the first window is full (`r < k - 1`)
- Re-summing the window each step: back to O(n·k)

✓ TIPS & BEST PRACTICES

- Track a `matches` counter of letters with equal counts for O(1) per step
- Window max/min needs a monotonic deque, not a running value
- Average = window sum ÷ k; compare sums to avoid float error

QUICK SUMMARY

- add right, drop left — O(1) per step
- window `[r - k + 1, r]`
- count arrays — anagram windows
- O(n) — total time

Sheet 064

Sliding Window — Variable Size

expand / shrink · longest without repeats · minimum window substring · at most K distinct · exactly K = atMost(K) − atMost(K-1)

GROW RIGHT, SHRINK LEFT

When the window size isn't fixed, **expand** the right edge one step at a time and **shrink** from the left whenever the window breaks the rule. Each index enters and leaves at most once, so the scan is **O(n)**. For "longest" answers, record after shrinking; for "shortest", record **while** shrinking a valid window.

RECOGNIZE IT WHEN

- "Longest / shortest substring or subarray such that …"
- "At most k distinct", "no repeats", "sum ≥ target"
- Validity is **monotonic**: shrinking never breaks a valid window
- All numbers non-negative (for sum windows)

TEMPLATE: LONGEST VALID WINDOW

```
l ← 0
best ← 0
for r ← 0 to n − 1
    add(arr[r])                 // expand
    while window invalid
        remove(arr[l])          // shrink
        l ← l + 1
    best ← max(best, r − l + 1)

// exactly K = atMost(K) − atMost(K − 1)
```

LONGEST SUBSTRING WITHOUT REPEATS

```
function lengthOfLongestSubstring(s)
    last ← new Map()                // char → last index
    (l, best) ← (0, 0)
    for r ← 0 to length(s) − 1
        c ← s[r]
        if last.has(c) and last[c] ≥ l
            l ← last[c] + 1         // jump past repeat
        last[c] ← r
        best ← max(best, r − l + 1)
    return best
```

MINIMUM WINDOW SUBSTRING

```
function minWindow(s, t)
    need ← counts of t (missing keys = 0)
    missing ← length(t)                  // chars still needed
    (l, bestL, bestLen) ← (0, 0, ∞)
    for r ← 0 to length(s) − 1
        if need[s[r]] > 0
            missing ← missing − 1
        need[s[r]] ← need[s[r]] − 1      // negative = surplus
        while missing = 0                // valid: shrink
            if r − l + 1 < bestLen
                (bestL, bestLen) ← (l, r − l + 1)
            need[s[l]] ← need[s[l]] + 1
            if need[s[l]] > 0
                missing ← missing + 1
            l ← l + 1
    if bestLen = ∞ return ""
    return s[bestL .. bestL + bestLen − 1]
```

PRACTICE

PROBLEM	INVALID WHEN
Longest Substring w/o Repeats	a char repeats
Longest Repeating Char Replacement	len − maxFreq > k
Min Size Subarray Sum	record while sum ≥ target
Fruit Into Baskets	> 2 distinct
Subarrays with K Distinct	atMost(K) − atMost(K-1)
Max Consecutive Ones III	zeros > k

⚠ COMMON MISTAKES

- Using a variable window on arrays with negatives: shrinking may not help (use prefix sums)
- Moving `l` backward in the no-repeat trick: guard with `last[c] ≥ l`
- Counting "exactly K" directly: windows aren't monotonic; subtract two atMost counts

✓ TIPS & BEST PRACTICES

- Write `add`, `remove`, and the validity test first; the loop is always the same
- Character Replacement: `maxFreq` never needs to decrease
- Store the best window's start and length, slice once at the end

QUICK SUMMARY

- expand right, shrink left — O(n)
- longest — record after shrinking
- shortest — record while valid
- exactly K = atMost(K) − atMost(K-1)

Sheet 065

Prefix Sums & Difference Arrays

prefix[i] · range sum O(1) · subarray sum = k (prefix + map) · 2D prefix sums · difference array range updates · prefix XOR / product

PRECOMPUTE ONCE, ANSWER IN O(1)

A **prefix sum** array stores the running total up to each index, so any range sum is one subtraction. Combined with a hash map of prefix counts, it finds subarrays with a target sum even when numbers are **negative** (where sliding windows fail). A **difference array** is the reverse trick: apply many range updates in O(1) each, then rebuild with one prefix pass.

RECOGNIZE IT WHEN

- Many range-sum queries on a fixed array
- "Count subarrays with sum = k / divisible by k"
- Many "add v to every item in [l, r]" updates
- Matrix region sums (2D prefix)

PREFIX SUMS AND SUBARRAY SUM = K

```
prefix ← new Array(n + 1, 0)      // prefix[i] = sum arr[0..i-1]
for i ← 0 to n - 1
    prefix[i + 1] ← prefix[i] + arr[i]
// sum of arr[l..r] = prefix[r + 1] - prefix[l]

function subarraySum(nums, k)     // counts, negatives OK
    seen ← new Map()
    seen[0] ← 1                   // the empty prefix
    (sum, count) ← (0, 0)
    for each x in nums
        sum ← sum + x
        count ← count + seen.getOrDefault(sum - k, 0)
        seen[sum] ← seen.getOrDefault(sum, 0) + 1
    return count
```

2D PREFIX AND DIFFERENCE ARRAY

```
// 2D: P has an extra row and column of zeros
P[i+1][j+1] ← g[i][j] + P[i][j+1]
              + P[i+1][j] - P[i][j]
// region (r1,c1)…(r2,c2):
// P[r2+1][c2+1] - P[r1][c2+1]
//   - P[r2+1][c1] + P[r1][c1]

diff ← new Array(n + 1, 0)
for each (l, r, v) in updates  // +v on [l..r]
    diff[l] ← diff[l] + v
    diff[r + 1] ← diff[r + 1] - v
// prefix-sum diff → final values
```

VARIANTS

PROBLEM TYPE	STORE
range sum query	prefix array
subarray sum = k	map: prefix sum → count
subarray sum divisible by k	map: prefix mod k → count
longest subarray with sum k	map: prefix → first index
equal 0s and 1s	treat 0 as -1, sum = 0
range XOR	`px[r+1] ^ px[l]`
product except self	prefix × suffix products

PRACTICE

PROBLEM
Range Sum Query - Immutable
Subarray Sum Equals K
Contiguous Array
Range Sum Query 2D
Car Pooling (difference)
Product of Array Except Self

⚠ COMMON MISTAKES

- Forgetting `seen[0] ← 1` : misses subarrays that start at index 0
- Adding the current prefix to the map **before** looking up `sum - k`
- Negative `mod` when grouping by remainder: normalize with `((s mod k) + k) mod k`

✓ TIPS & BEST PRACTICES

- Use length n + 1 prefixes so `prefix[0] = 0` and no index needs a special case
- Difference arrays shine for "bookings", "car pooling", "flights"
- If the array changes between queries, use a Fenwick tree (Sheet 093)

QUICK SUMMARY

- `prefix[r+1] - prefix[l]` — range sum
- map of prefix counts — subarray targets
- difference array — O(1) range updates
- 2D prefix — inclusion–exclusion

Sheet 066

Kadane's Algorithm & Subarrays

max subarray sum · reset rule · tracking indices · max product subarray · circular subarray · best time to buy / sell stock

BEST SUBARRAY IN ONE PASS

Kadane's algorithm finds the maximum-sum contiguous subarray in O(n). At each index, the best subarray **ending here** either extends the previous one or **restarts** at this element, whichever is larger. The overall answer is the best of those. The same idea solves product subarrays (track min and max) and the classic stock-profit problem.

RECOGNIZE IT WHEN

- "Maximum / minimum sum of a contiguous subarray"
- "Maximum product subarray" (negatives flip signs)
- Buy once, sell once for max profit
- Circular arrays: max of normal vs total - min subarray

KADANE: EXTEND OR RESTART

```
function maxSubArray(nums)
    cur ← nums[0]                  // best ending here
    best ← nums[0]                 // best seen anywhere
    for i ← 1 to length(nums) - 1
        cur ← max(nums[i], cur + nums[i])
        best ← max(best, cur)
    return best

// circular: max(best, total - minSubarray)
// unless every number is negative
```

TRACKING THE INDICES

```
function maxSubArrayRange(nums)
    (cur, start) ← (0, 0)
    (best, bl, br) ← (-∞, 0, 0)
    for i ← 0 to length(nums) - 1
        if cur ≤ 0
            (cur, start) ← (nums[i], i)   // restart
        else
            cur ← cur + nums[i]
        if cur > best
            (best, bl, br) ← (cur, start, i)
    return (best, bl, br)
```

MAXIMUM PRODUCT SUBARRAY

```
function maxProduct(nums)
    (hi, lo, best) ← (nums[0], nums[0], nums[0])
    for i ← 1 to length(nums) - 1
        x ← nums[i]
        c ← (x, hi * x, lo * x)
        (hi, lo) ← (max(c), min(c))  // sign flips
        best ← max(best, hi)
    return best
```

BEST TIME TO BUY AND SELL STOCK

```
function maxProfit(prices)
    (minPrice, profit) ← (∞, 0)
    for each p in prices
        minPrice ← min(minPrice, p)
        profit ← max(profit, p - minPrice)
    return profit
// = Kadane on daily price differences
```

⚠ COMMON MISTAKES

- Starting `best` at 0: wrong when every number is negative
- Tracking only the max in product subarrays: a negative times the min becomes the new max
- Updating `hi` before computing `lo` with the old value: compute both from a saved tuple

✓ TIPS & BEST PRACTICES

- Practice: Maximum Subarray, Max Product Subarray, Max Sum Circular Subarray, Best Time to Buy and Sell Stock
- Kadane is 1D DP with O(1) state (Sheet 080)
- Max sum rectangle in a matrix: fix two rows, run Kadane on column sums

QUICK SUMMARY

- `cur = max(x, cur + x)` — extend or restart
- `best` — max over all `cur`
- track `hi` and `lo` — products
- min so far — stock profit

Monotonic Stack

next greater element · daily temperatures · stock span · largest rectangle in histogram · trapping rain water

ANSWER "NEXT GREATER" FOR EVERYONE IN O(N)

A **monotonic stack** keeps its items in increasing or decreasing order. When a new item breaks the order, everything it pops has just found its **next greater (or smaller) element**: the new item. Each index is pushed and popped once, so the whole pass is **O(n)**, replacing an O(n²) "scan right for each element".

WHICH STACK?

- Next **greater**: keep a **decreasing** stack
- Next **smaller**: keep an **increasing** stack
- Previous greater: the stack top after popping
- Store **indexes** so you can compute distances

NEXT GREATER ELEMENT / DAILY TEMPERATURES

```
function nextGreater(nums)
    res ← new Array(length(nums), -1)
    stack ← []                              // indexes; values decreasing
    for i ← 0 to length(nums) - 1
        while not stack.isEmpty() and nums[stack.peek()] < nums[i]
            j ← stack.pop()
            res[j] ← nums[i]                // Daily Temperatures: i - j
        stack.push(i)
    return res                              // still on stack → no answer
```

CODE EXPLAINED

- → decreasing stack — waiting for something bigger
- → pop — the current item answers it
- → `i - j` — distance (days to wait)
- → circular array: loop `2n` times using `i mod n`

LARGEST RECTANGLE IN HISTOGRAM

```
function largestRectangle(h)
    h.append(0)                     // flush the stack
    stack ← []                      // increasing heights
    best ← 0
    for i ← 0 to length(h) - 1
        while not stack.isEmpty()
                and h[stack.peek()] ≥ h[i]
            height ← h[stack.pop()]
            left ← if stack.isEmpty() then -1 else stack.peek()
            best ← max(best, height * (i - left - 1))
        stack.push(i)
    return best
```

TRAPPING RAIN WATER (STACK)

```
function trap(h)
    (stack, water) ← ([], 0)
    for i ← 0 to length(h) - 1
        while not stack.isEmpty()
                and h[i] > h[stack.peek()]
            bottom ← stack.pop()
            if stack.isEmpty() break
            left ← stack.peek()
            width ← i - left - 1
            depth ← min(h[left], h[i]) - h[bottom]
            water ← water + width * depth
        stack.push(i)
    return water
```

⚠ COMMON MISTAKES

- Storing values instead of indexes: can't compute widths or distances
- Wrong comparison (`<` vs `≤`) with duplicates: double-counted or missed ranges
- Forgetting the leftover stack items: they have no next greater (-1 / 0)

✓ TIPS & BEST PRACTICES

- Practice: Next Greater Element I/II, Daily Temperatures, Online Stock Span, Car Fleet
- Histogram: popped bar's width runs from the new top + 1 to i - 1
- Rain water also has an O(1)-space two-pointer solution

QUICK SUMMARY

- decreasing stack — next greater
- increasing stack — next smaller
- push / pop once — O(n)
- sentinel 0 — flush remaining bars

Monotonic Deque

sliding window maximum · deque of indices · pop stale front · shortest subarray with sum ≥ K · jump game VI

WINDOW MAX OR MIN IN O(1) PER STEP

A **monotonic deque** keeps indexes whose values are decreasing from front to back. The **front** is the current window's maximum. Before adding a new index, pop from the **back** every item it beats (they can never be the max again); pop from the **front** when an index slides out of the window. Every index enters and leaves once: **O(n)** total.

RECOGNIZE IT WHEN

- "Maximum / minimum of every window of size k"
- DP where `dp[i]` needs the max of the last k values
- Shortest subarray with sum ≥ K **with negatives**
- Front = answer; back = where you drop losers

SLIDING WINDOW MAXIMUM

```
function maxSlidingWindow(nums, k)
    dq ← new Deque()                    // indexes; values decreasing
    out ← []
    for i ← 0 to length(nums) − 1
        if not dq.isEmpty() and dq.front() ≤ i − k
            dq.popFront()               // fell out of the window
        while not dq.isEmpty() and nums[dq.back()] ≤ nums[i]
            dq.popBack()                // can never be the max again
        dq.pushBack(i)
        if i ≥ k − 1
            out.append(nums[dq.front()])
    return out
```

CODE EXPLAINED

- → front — index of the window max
- → pop front — expired index
- → pop back — dominated by the new item
- → window min — keep values increasing instead
- → O(n) time, O(k) space

SHORTEST SUBARRAY WITH SUM ≥ K

```
function shortestSubarray(nums, K)
    P ← prefix sums, length n + 1
    dq ← new Deque()          // P increasing
    best ← ∞
    for j ← 0 to n
        while not dq.isEmpty()
                and P[j] − P[dq.front()] ≥ K
            best ← min(best, j − dq.popFront())
        while not dq.isEmpty()
                and P[dq.back()] ≥ P[j]
            dq.popBack()
        dq.pushBack(j)
    return if best = ∞ then −1 else best
```

JUMP GAME VI (DEQUE-OPTIMIZED DP)

```
function maxResult(nums, k)
    dp ← new Array(n)
    dp[0] ← nums[0]
    dq ← new Deque([0])       // dp decreasing
    for i ← 1 to n − 1
        if dq.front() < i − k
            dq.popFront()
        dp[i] ← nums[i] + dp[dq.front()]
        while not dq.isEmpty()
                and dp[dq.back()] ≤ dp[i]
            dq.popBack()
        dq.pushBack(i)
    return dp[n − 1]
```

⚠ COMMON MISTAKES

- Storing values in the deque: you can't tell when one has left the window
- Checking expiry with `<` instead of `≤ i − k`: the window becomes k + 1 wide
- Using a heap: works, but O(n log n) and needs lazy deletion

✓ TIPS & BEST PRACTICES

- Practice: Sliding Window Maximum, Shortest Subarray with Sum ≥ K, Jump Game VI, Constrained Subsequence Sum
- Longest subarray with |max − min| ≤ limit: two deques (max and min)
- Deque here = Python `collections.deque`, Java `ArrayDeque`

QUICK SUMMARY

- deque of indexes — values monotonic
- front — window answer
- pop back — dominated items
- O(n) — each index in and out once

Intervals & Sweep Line

sort by start · merge intervals · insert interval · overlap test · meeting rooms II · non-overlapping (greedy by end) · event sweep

RANGES ON A LINE

Interval problems almost always start by **sorting** (by start to merge, by end to schedule greedily). After sorting, one pass compares each interval with the last kept one. A **sweep line** turns intervals into +1 / -1 events at their start and end times; the running total at any moment is how many intervals overlap there.

OVERLAP TEST

- Closed: `a.start ≤ b.end and b.start ≤ a.end`
- Half-open: `a.start < b.end and b.start < a.end`
- Sorted by start: overlap iff `cur.start ≤ last.end`
- Ask whether touching endpoints count as overlapping

MERGE INTERVALS

```
function merge(intervals)
    sortBy(intervals, lambda iv: iv.start)
    out ← [intervals[0]]
    for each (s, e) in intervals[1..]
        last ← out[length(out) - 1]
        if s ≤ last.end
            last.end ← max(last.end, e)  // extend
        else
            out.append((s, e))
    return out
```

INSERT INTERVAL

```
function insert(ivs, ns, ne)      // sorted, disjoint
    (out, i) ← ([], 0)
    while i < n and ivs[i].end < ns       // before
        out.append(ivs[i])
        i ← i + 1
    while i < n and ivs[i].start ≤ ne     // overlap
        ns ← min(ns, ivs[i].start)
        ne ← max(ne, ivs[i].end)
        i ← i + 1
    out.append((ns, ne))
    append ivs[i..] to out                // after
    return out
```

MEETING ROOMS II (SWEEP LINE)

```
function minRooms(meetings)
    events ← []
    for each (s, e) in meetings
        events.append((s, +1))
        events.append((e, -1))
    sort(events)          // ties: -1 before +1
    (cur, best) ← (0, 0)
    for each (t, d) in events
        cur ← cur + d
        best ← max(best, cur)
    return best
```

NON-OVERLAPPING INTERVALS (GREEDY BY END)

```
function eraseOverlap(intervals)
    sortBy(intervals, lambda iv: iv.end)
    (keep, lastEnd) ← (0, -∞)
    for each (s, e) in intervals
        if s ≥ lastEnd              // fits after last
            keep ← keep + 1
            lastEnd ← e
    return length(intervals) - keep // removals
```

⚠ COMMON MISTAKES

- Sorting by end to merge, or by start to schedule: each task needs the other key
- Merging into a new interval but forgetting to extend `end` with `max`
- Sweep ties: processing a start before an end at the same time adds a false room

✓ TIPS & BEST PRACTICES

- Practice: Merge Intervals, Insert Interval, Non-overlapping Intervals, Meeting Rooms I/II, Minimum Arrows
- Meeting Rooms II also works with a min-heap of end times (Sheet 044)
- Interval List Intersections: two pointers, advance the one that ends first

QUICK SUMMARY

- sort by start — merge
- sort by end — keep the most intervals
- +1 / -1 events — max overlap
- overlap — `a.start ≤ b.end and b.start ≤ a.end`

Sheet 070

Matrix Traversal & Manipulation

transpose · rotate 90° in place · spiral order · diagonal traversal · set matrix zeroes · search sorted matrix

WALKING AND RESHAPING GRIDS

Matrix problems reuse a few index tricks: **transpose** swaps `m[i][j]` with `m[j][i]`, **reversing** rows mirrors left-right, and together they **rotate** in place. **Boundaries** (top, bottom, left, right) that shrink inward drive spiral order. Cells on one **diagonal** share `r - c`; on an anti-diagonal they share `r + c`.

QUICK FACTS

- R × C matrix: `m[r][c]`, rows first
- Rotate 90° clockwise = transpose, then reverse each row
- Sorted rows + columns: start at the top-right corner
- Flatten: index `r * C + c` ↔ `(i div C, i mod C)`

ROTATE 90° CLOCKWISE (IN PLACE)

```
function rotate(m)                    // n × n
    n ← length(m)
    for i ← 0 to n - 1                // 1) transpose
        for j ← i + 1 to n - 1
            (m[i][j], m[j][i]) ← (m[j][i], m[i][j])
    for each row in m                 // 2) mirror
        reverse(row)

// counter-clockwise: transpose,
// then reverse each column
```

SEARCH A SORTED MATRIX (STAIRCASE)

```
function searchMatrix(m, t)     // rows & cols sorted
    (r, c) ← (0, C - 1)         // top-right corner
    while r < R and c ≥ 0
        if m[r][c] = t
            return true
        if m[r][c] > t
            c ← c - 1           // drop this column
        else
            r ← r + 1           // drop this row
    return false
// O(R + C)
```

SPIRAL ORDER

```
function spiral(m)
    (top, bottom, left, right) ← (0, R - 1, 0, C - 1)
    out ← []
    while top ≤ bottom and left ≤ right
        for c ← left to right
            out.append(m[top][c])
        top ← top + 1
        for r ← top to bottom
            out.append(m[r][right])
        right ← right - 1
        if top ≤ bottom
            for c ← right down to left
                out.append(m[bottom][c])
            bottom ← bottom - 1
        if left ≤ right
            for r ← bottom down to top
                out.append(m[r][left])
            left ← left + 1
    return out
```

MORE TRICKS

PROBLEM	TRICK
Set Matrix Zeroes	first row / col as markers: O(1) space
Diagonal Traverse	group by `r + c`, alternate direction
Valid Sudoku	box id `(r div 3) * 3 + c div 3`
Game of Life	encode old/new state in spare bits
Toeplitz Matrix	`m[r][c] = m[r-1][c-1]`
Search 2D Matrix I	binary search as one flat array

⚠ COMMON MISTAKES

- Swapping every pair during transpose (`j` from 0): each swap is undone
- Spiral without the `top ≤ bottom` / `left ≤ right` checks: duplicates single rows
- Mixing `R` and `C` for non-square matrices

✓ TIPS & BEST PRACTICES

- Draw a 3×3 and a 3×4 example before coding any traversal
- Use a direction array and turn on bounds for a simpler spiral
- Copying the matrix is fine unless the problem says in place

QUICK SUMMARY

- transpose + reverse rows — rotate right
- shrinking bounds — spiral
- `r - c` / `r + c` — diagonals
- top-right start — sorted matrix search

Cyclic Sort & In-Place Index Marking

values 1..n · swap to home index · find missing number · find duplicates · negate-to-mark · first missing positive

USE THE ARRAY AS ITS OWN HASH TABLE

When values fall in a known range like **1…n**, each value has a natural **home index** (`value - 1`). **Cyclic sort** swaps every value to its home in O(n); afterwards any index holding the wrong value reveals a missing or duplicate number. Alternatively, **negate** `nums[v - 1]` to mark that v was seen. Both use **O(1) extra space**.

RECOGNIZE IT WHEN

- Values in `1…n` or `0…n` and "O(1) extra space"
- "Find the missing / duplicate / first missing positive"
- "All numbers disappeared", "set mismatch"
- Input may be modified in place

CYCLIC SORT

```
function cyclicSort(nums)       // values 1..n
    i ← 0
    while i < length(nums)
        j ← nums[i] - 1         // home index
        if nums[i] ≠ nums[j]
            swap(nums, i, j)    // recheck slot i
        else
            i ← i + 1
// each swap places one value for good:
// at most n swaps → O(n)
```

NEGATE TO MARK: FIND DUPLICATES

```
function findDuplicates(nums)  // values 1..n
    out ← []
    for each x in nums
        j ← abs(x) - 1
        if nums[j] < 0
            out.append(abs(x)) // seen before
        else
            nums[j] ← -nums[j] // mark seen
    return out
// Disappeared Numbers: mark, then report
// every index still positive (i + 1)
```

FIRST MISSING POSITIVE

```
function firstMissingPositive(nums)
    n ← length(nums)
    for i ← 0 to n - 1
        while nums[i] ≥ 1 and nums[i] ≤ n
                and nums[nums[i] - 1] ≠ nums[i]
            swap(nums, i, nums[i] - 1)    // send home
    for i ← 0 to n - 1
        if nums[i] ≠ i + 1
            return i + 1                  // first gap
    return n + 1
```

PRACTICE

PROBLEM	APPROACH
Missing Number (0…n)	sum or XOR
Find All Duplicates	negate marking
Disappeared Numbers	negate, then scan
Set Mismatch	cyclic sort or marks
First Missing Positive	cyclic sort, ignore out-of-range
Find the Duplicate Number	Floyd cycle (Sheet 031)

⚠ COMMON MISTAKES

- Using `nums[i] ≠ i + 1` as the swap test with duplicates: infinite loop
- Reading `nums[x - 1]` after `x` was negated: always use `abs(x)`
- Modifying input when the problem forbids it (Find the Duplicate Number)

✓ TIPS & BEST PRACTICES

- Shift the range first if values are `0…n-1` (home = value) or `0…n`
- Missing Number without modifying input: `n(n+1)/2 - sum` or XOR
- Restore signs afterward if the caller needs the original array

QUICK SUMMARY

- home index — `value - 1`
- swap until home — cyclic sort
- negative sign — seen flag
- O(n) time, O(1) space

Recursion Fundamentals

FUNCTIONS THAT CALL THEMSELVES

A **recursive** function solves a problem by calling itself on a **smaller** version of the same problem until it reaches a **base case** it can answer directly. Each call waits on the call stack until its sub-call returns. The key skill is to **trust the recursion**: assume the call on the smaller input is correct, and only write how to combine its answer.

EVERY RECURSION NEEDS

- A **base case** that stops (empty, 0, leaf, null)
- Progress: each call's input is strictly smaller
- A clear statement of what `f(x)` returns
- Depth that fits the stack (~10^3–10^4 frames)

BASE CASE, RECURSIVE CASE, TRACE

```
function factorial(n)
    if n ≤ 1 return 1              // base case
    return n * factorial(n - 1)    // smaller input

// factorial(3)
//   = 3 * factorial(2)
//         = 2 * factorial(1)
//               = 1          ← base case returns first
//         = 2 * 1 = 2
//   = 3 * 2 = 6

function factTail(n, acc ← 1)      // tail call: last action
    if n ≤ 1 return acc
    return factTail(n - 1, acc * n)
```

RECURSION → ITERATION

```
function sumTree(root)             // recursive
    if root = null return 0
    return root.val + sumTree(root.left)
                    + sumTree(root.right)

function sumTreeIter(root)         // explicit stack
    (total, stack) ← (0, [root])
    while not stack.isEmpty()
        node ← stack.pop()
        if node = null continue
        total ← total + node.val
        stack.push(node.left)
        stack.push(node.right)
    return total
```

WRITING A RECURSIVE FUNCTION

STEP	ASK
1. contract	what exactly does `f(x)` return?
2. base case	smallest input I can answer directly?
3. shrink	which smaller input(s) do I call on?
4. trust	assume those calls are correct
5. combine	build my answer from theirs

TAIL-CALL OPTIMIZATION

LANGUAGE	OPTIMIZES TAIL CALLS?
Scheme, Haskell	yes (required)
Scala / Kotlin	with `@tailrec` / `tailrec`
C / C++	often, at higher -O levels
Python, Java, C#	no
JavaScript	spec says yes; only Safari does

⚠ COMMON MISTAKES

- Missing or unreachable base case: infinite recursion → stack overflow
- Recursing on an input that isn't smaller (`f(n)` calling `f(n)`)
- Re-solving the same subproblem many times: add memoization (Sheet 078)

✓ TIPS & BEST PRACTICES

- Trace the smallest two inputs by hand to check the base case
- Python: `sys.setrecursionlimit` helps a little; iteration helps more
- Practice: Fibonacci, Pow(x, n), Reverse Linked List (recursive), Max Depth of Tree

QUICK SUMMARY

- base case — stop condition
- recursive case — smaller input
- trust the recursion — only combine
- explicit stack — same logic, no depth limit

Divide & Conquer

split / solve / combine · merge sort · fast power · majority element · Master Theorem cases · closest pair of points

SPLIT, SOLVE, COMBINE

Divide and conquer breaks a problem into independent pieces (usually halves), solves each recursively, and **combines** the results. Unlike DP, the pieces don't overlap, so no caching is needed. Its cost follows the recurrence `T(n) = a·T(n/b) + combine cost`, which the Master Theorem (Sheet 024) solves.

RECOGNIZE IT WHEN

- The answer for the whole follows from answers for halves
- A cheap combine step exists (merge, compare, multiply)
- Halving cuts work: O(n²) → O(n log n), O(n) → O(log n)
- Tree problems: each subtree is an independent half

TEMPLATE AND FAST POWER

```
function solve(problem)
    if problem is small
        return answer directly      // base case
    (A, B) ← split(problem)         // usually halves
    return combine(solve(A), solve(B))

function power(x, n)                // O(log n)
    if n = 0 return 1
    half ← power(x, n div 2)        // solve once, reuse
    if n mod 2 = 0
        return half * half
    return half * half * x
// negative n: 1 / power(x, -n)
```

MAJORITY ELEMENT (D&C)

```
function majority(a, lo, hi)        // O(n log n)
    if lo = hi return a[lo]
    mid ← (lo + hi) div 2
    left ← majority(a, lo, mid)
    right ← majority(a, mid + 1, hi)
    if left = right return left
    cl ← count of left in a[lo..hi]
    cr ← count of right in a[lo..hi]
    return if cl > cr then left else right

// Boyer-Moore voting: O(n), O(1) space
```

CLASSIC DIVIDE-AND-CONQUER

ALGORITHM	SPLIT	COMBINE	TOTAL
binary search	keep 1 half	none	O(log n)
merge sort	2 halves	merge O(n)	O(n log n)
quick sort	partition O(n)	none	O(n log n) avg
fast power	n → n/2	1 multiply	O(log n)
count inversions	2 halves	merge + count	O(n log n)
closest pair of points	by x	check middle strip	O(n log n)
Karatsuba multiply	3 half-size products	add / shift	O(n^1.58)

⚠ COMMON MISTAKES

- Calling `power(x, n div 2)` twice: O(n) instead of O(log n)
- Halves that overlap or share state: that's DP territory, not D&C
- Unbalanced splits (always 1 and n - 1): degrades to O(n²)

✓ TIPS & BEST PRACTICES

- Draw the recursion tree: depth × work per level gives the total
- Closest pair: only points within d of the middle line need checking, ~7 neighbors each
- Practice: Pow(x, n), Sort an Array, Majority Element, Construct Tree from Traversals

QUICK SUMMARY

- split — independent subproblems
- combine — where the real work is
- `T(n) = aT(n/b) + f(n)` — cost
- reuse a half result — fast power

Backtracking Template

choose / explore / unchoose · state & path · decision tree · pruning · copy on record · branches^depth cost

TRY, RECURSE, UNDO

Backtracking builds a solution one choice at a time, exploring a **decision tree** with DFS. At each step it **chooses** an option, **explores** deeper, then **unchooses** it to try the next one. **Pruning** (skipping choices that can't lead to a valid answer) is what keeps it practical. Cost is roughly branches^depth.

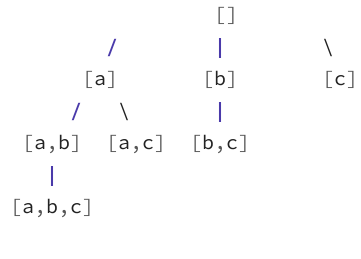
RECOGNIZE IT WHEN

- "Generate all …" combinations, permutations, subsets
- "Find any arrangement satisfying constraints"
- n is small (≤ 10–20)
- Need the actual solutions, not just a count

THE TEMPLATE

```
function backtrack(path, choices)
    if path is complete
        results.append(copy(path))    // record a COPY
        return
    for each c in choices
        if not valid(c, path)
            continue                  // prune
        path.append(c)                // choose
        backtrack(path, choices')     // explore
        path.pop()                    // unchoose
```

DECISION TREE: SUBSETS OF [A, B, C]




```
each edge = one choice
each node = one partial path
```

LETTER COMBINATIONS OF A PHONE NUMBER

```
function letterCombinations(digits)
    if digits is empty return []
    keys ← { '2': "abc", '3': "def", …, '9': "wxyz" }
    out ← []
    function bt(i, path)
        if i = length(digits)
            out.append(join(path))
            return
        for each ch in keys[digits[i]]
            path.append(ch)
            bt(i + 1, path)
            path.pop()
    bt(0, [])
    return out
```

SEARCH SPACE SIZES

GENERATING	COUNT
subsets	2^n
k-combinations	C(n, k)
permutations	n!
phone letters	$\le 4^n$
balanced parens	Catalan(n)
× copy each result	× n

⚠ COMMON MISTAKES

- Appending `path` itself to results: every stored result is the same emptied list
- Forgetting to undo (`pop` , unmark): later branches see stale choices
- Pruning after recursing instead of before: no time saved

✓ TIPS & BEST PRACTICES

- Sort input first so you can `break` once values get too large
- Pass indexes, not sliced copies, to avoid O(n) work per call
- Practice: Letter Combinations, Generate Parentheses, Subsets, Permutations, Combination Sum

QUICK SUMMARY

- choose → explore → unchoose
- copy the path when recording
- prune early — the real speedup
- ≈ branches^depth — cost

Subsets & Combinations

include / exclude · start index · skip duplicates (sort + check) · combination sum I / II · k-combinations · bitmask enumeration

CHOOSING SUBSETS OF ITEMS

Subsets and combinations ignore order, so a **start index** stops you from reusing earlier items (which would create the same set in another order). Every node of the subset tree is an answer; for combinations, only nodes of the right size or sum are. With duplicate values, **sort** and skip a value equal to the previous one **at the same depth**.

VARIANTS

- Subsets: record at every node
- k-combinations: record when `length(path) = k`
- Combination Sum: reuse allowed → recurse with `i`
- Combination Sum II: no reuse → `i + 1` + skip duplicates

SUBSETS (AND SUBSETS II)

```
function subsets(nums)           // 2ⁿ subsets
    sort(nums)                   // needed for II
    out ← []
    function bt(start, path)
        out.append(copy(path))   // every node
        for i ← start to length(nums) - 1
            if i > start and nums[i] = nums[i - 1]
                continue         // II: skip dup
            path.append(nums[i])
            bt(i + 1, path)
            path.pop()
    bt(0, [])
    return out
```

COMBINATION SUM (REUSE ALLOWED)

```
function combinationSum(cands, target)
    sort(cands)
    out ← []
    function bt(start, path, remain)
        if remain = 0
            out.append(copy(path))
            return
        for i ← start to length(cands) - 1
            if cands[i] > remain
                break            // rest too big
            path.append(cands[i])
            bt(i, path, remain - cands[i])
            path.pop()
    bt(0, [], target)
    return out
```

K-COMBINATIONS

```
function combine(n, k)               // choose k of 1..n
    out ← []
    function bt(start, path)
        if length(path) = k
            out.append(copy(path))
            return
        need ← k - length(path)
        for i ← start to n - need + 1   // prune: room left
            path.append(i)
            bt(i + 1, path)
            path.pop()
    bt(1, [])
    return out
```

PRACTICE

PROBLEM	TWIST
Subsets	record every node
Subsets II	sort + skip dup
Combinations	size k, prune range
Combination Sum	recurse with `i`
Combination Sum II	`i + 1` + skip dup
Combination Sum III	k numbers from 1–9

⚠ COMMON MISTAKES

- Skipping duplicates with `i > 0` instead of `i > start`: drops valid sets like [1, 1]
- Recursing with `i + 1` when reuse is allowed (or `i` when it isn't)
- Looping from 0 instead of `start`: generates the same set in different orders

✓ TIPS & BEST PRACTICES

- Bitmask subsets: `for mask ← 0 to 2ⁿ - 1`, take item i when `(mask >> i) & 1 = 1`
- Sort once up front; it enables both `break` pruning and duplicate skipping
- Need only the count? Use DP (knapsack, Sheet 082) instead

QUICK SUMMARY

- start index — no reordering
- `i > start` and equal — skip duplicates
- `bt(i, …)` — reuse allowed
- `mask >> i & 1` — bitmask subsets

Permutations

EVERY ORDERING

A **permutation** uses every item once in some order, so each level of the tree may pick **any unused item**: track them with a `used[]` array (or by swapping into place). There are **n!** permutations, so this is only feasible for n ≤ ~10. For the next or kth ordering, skip enumeration and compute it directly.

VARIANTS

- All permutations: `used[]` or swap method
- With duplicates: sort, skip `nums[i] = nums[i-1]` if `not used[i-1]`
- Next permutation: O(n), in place
- kth permutation: factorial number system, O(n²)

PERMUTATIONS WITH USED[]

```
function permute(nums)          // n! · n
    n ← length(nums)
    (out, used) ← ([], new Array(n, false))
    function bt(path)
        if length(path) = n
            out.append(copy(path))
            return
        for i ← 0 to n - 1
            if used[i] continue
            // with duplicates (sorted input):
            // if i > 0 and nums[i] = nums[i-1]
            //    and not used[i-1]: continue
            used[i] ← true
            path.append(nums[i])
            bt(path)
            path.pop()
            used[i] ← false
    bt([])
    return out
```

SWAP-IN-PLACE METHOD

```
function permuteSwap(a, k)
    if k = length(a)
        out.append(copy(a))
        return
    for i ← k to length(a) - 1
        swap(a, k, i)           // fix a[k]
        permuteSwap(a, k + 1)
        swap(a, k, i)           // undo

// no extra used[] array;
// output order differs from
// the lexicographic one
```

NEXT PERMUTATION

```
function nextPermutation(a)
    i ← length(a) - 2
    while i ≥ 0 and a[i] ≥ a[i + 1]
        i ← i - 1               // rightmost "dip"
    if i ≥ 0
        j ← length(a) - 1
        while a[j] ≤ a[i]
            j ← j - 1           // rightmost bigger
        swap(a, i, j)
    reverse(a, i + 1, length(a) - 1)
// [1,2,3]→[1,3,2]→[2,1,3]→…→[3,2,1]→[1,2,3]
```

KTH PERMUTATION (N = 4, K = 9)

STEP	WORK
`k - 1 = 8`	0-indexed
`8 div 3! = 1`	pick index 1 of [1,2,3,4] → 2
`8 mod 6 = 2; 2 div 2! = 1`	index 1 of [1,3,4] → 3
`2 mod 2 = 0; 0 div 1! = 0`	index 0 of [1,4] → 1
`result`	2 3 1 4

⚠ COMMON MISTAKES

- Duplicate-skip with `used[i-1]` instead of `not used[i-1]` : still produces duplicates
- Forgetting to reset `used[i]` after the call
- Generating all n! permutations to find the next one

✓ TIPS & BEST PRACTICES

- Practice: Permutations, Permutations II, Next Permutation, Permutation Sequence
- Next permutation reversed (rightmost bump, smaller swap) gives the previous one
- n ≤ 8 is the usual comfort limit (8! = 40,320)

QUICK SUMMARY

- `used[]` — pick any unused item
- swap / unswap — in-place method
- dip, swap, reverse — next permutation
- n! — always the cost of listing them

Board & Grid Backtracking

N-Queens (column / diagonal sets) · Sudoku solver · word search · palindrome partitioning · restore visited cell

CONSTRAINT PUZZLES ON BOARDS

Board problems are backtracking with **constraint checks**: place something in a cell, verify rows, columns, diagonals, or boxes, recurse, then **restore** the cell. O(1) checks come from keeping **sets** of what each row, column, or diagonal already contains. For word paths on a grid, mark a cell visited before recursing and unmark it afterward.

CONSTRAINT TRICKS

- Diagonal id: `r - c`; anti-diagonal id: `r + c`
- Sudoku box id: `(r div 3) * 3 + c div 3`
- Mark visited with a sentinel like `'#'`, then restore

N-QUEENS

```
function solveNQueens(n)
    (cols, diag, anti) ← (new Set(), new Set(), new Set())
    out ← []
    function place(r, queens)
        if r = n
            out.append(copy(queens))
            return
        for c ← 0 to n - 1
            if cols.has(c) or diag.has(r - c)
                    or anti.has(r + c)
                continue                // attacked
            add c, r - c, r + c to the sets
            queens.append(c)
            place(r + 1, queens)
            queens.pop()
            remove c, r - c, r + c from the sets
    place(0, [])
    return out                    // queens[r] = col
```

SUDOKU SOLVER

```
function solve(board)
    cell ← next empty cell
    if cell = null return true      // full
    for each d in "123456789"
        if canPlace(cell, d)        // row/col/box sets
            board[cell] ← d
            mark d used in its row, col, box
            if solve(board)
                return true
            board[cell] ← '.'       // undo
            unmark d
    return false
```

WORD SEARCH

```
function exist(board, word)
    function dfs(r, c, i)
        if i = length(word) return true
        if not inBounds(r, c) or board[r][c] ≠ word[i]
            return false
        tmp ← board[r][c]
        board[r][c] ← '#'           // visited
        found ← dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1)
             or dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1)
        board[r][c] ← tmp           // restore
        return found
    for each cell (r, c)
        if dfs(r, c, 0) return true
    return false
```

PALINDROME PARTITIONING

```
function partition(s)
    out ← []
    function bt(start, path)
        if start = length(s)
            out.append(copy(path))
            return
        for end ← start to length(s) - 1
            if isPalindrome(s, start, end)
                path.append(s[start..end])
                bt(end + 1, path)
                path.pop()
    bt(0, [])
    return out
```

⚠ COMMON MISTAKES

- Not restoring the cell after Word Search's DFS: later starts see `'#'`
- Checking queen attacks by scanning the board: O(n) per check instead of O(1)
- Word Search on many words: run one Trie-guided DFS instead (Word Search II, Sheet 091)

✓ TIPS & BEST PRACTICES

- Practice: N-Queens, N-Queens II, Sudoku Solver, Word Search, Palindrome Partitioning
- Precompute `isPal[i][j]` with DP to speed up partitioning
- Return early on the first solution when only one is needed

QUICK SUMMARY

- sets per row / col / diagonal — O(1) checks
- `r - c`, `r + c` — diagonal ids
- mark → recurse → restore
- fail fast — prune before recursing

DP Fundamentals & Memoization

overlapping subproblems · optimal substructure · state · transition · base case · top-down memo · brute force → memo

SOLVE EACH SUBPROBLEM ONCE

Dynamic programming applies when a problem breaks into subproblems that **overlap** (the same ones come up again and again) and the best answer is built from best answers to subproblems (**optimal substructure**). **Memoization** is the easiest way in: write the brute-force recursion, then cache each result by its arguments so it's computed only once.

RECOGNIZE IT WHEN

- "Count the number of ways …"
- "Minimum / maximum cost, length, profit …"
- "Is it possible to reach / form …"
- Each choice changes what's possible later

BRUTE FORCE → MEMOIZED

```
// Climbing Stairs: 1 or 2 steps at a time
function ways(n)                    // O(2ⁿ): repeats work
    if n ≤ 1 return 1
    return ways(n - 1) + ways(n - 2)

memo ← new Map()
function waysMemo(n)                // O(n): each n once
    if n ≤ 1 return 1
    if memo.has(n) return memo[n]
    memo[n] ← waysMemo(n - 1) + waysMemo(n - 2)
    return memo[n]

// ways(5) calls ways(3) twice and ways(2)
// three times; the memo answers repeats in O(1)
```

MEMO WITH A RICHER STATE: COIN CHANGE

```
function coinChange(coins, amount)   // fewest coins
    memo ← new Map()
    function best(rem)               // state = rem
        if rem = 0 return 0
        if rem < 0 return ∞
        if memo.has(rem) return memo[rem]
        res ← ∞
        for each c in coins          // transition
            res ← min(res, 1 + best(rem - c))
        memo[rem] ← res
        return res
    ans ← best(amount)
    return if ans = ∞ then -1 else ans
```

DESIGNING A DP

STEP	QUESTION	COIN CHANGE
state	what changes between subproblems?	`rem` amount
transition	how do states connect?	`1 + best(rem - c)`
base case	smallest answers?	`best(0) = 0`
answer	which state is the goal?	`best(amount)`
cost	states × work per state	amount × coins

MEMO KEYS

STATE	KEY
one index	`i`
two indexes	`(i, j)` or `i * m + j`
index + flag	`(i, holding)`
index + set	`(i, mask)`

⚠ COMMON MISTAKES

- Memoizing on something that isn't the full state: different situations share a cache entry
- Mutable arguments (lists) as part of the state: use an index instead
- Greedy instead of DP for Coin Change: coins [1, 3, 4], amount 6 → greedy 3 coins, best 2

✓ TIPS & BEST PRACTICES

- Get the brute-force recursion right first; caching is the easy part
- Python: `@functools.cache` memoizes a function in one line
- Deep memo recursion can overflow; convert to tabulation (Sheet 079)

QUICK SUMMARY

- overlapping subproblems + optimal substructure
- state · transition · base case · answer
- memo = recursion + cache
- time = states × work per state

Tabulation & Space Optimization

FILL A TABLE BOTTOM-UP

Tabulation computes DP answers iteratively, smallest subproblems first, storing them in an array. It has no recursion overhead or depth limit, and it makes **space optimization** easy: if each row depends only on the previous one or two, keep just those. To output the actual solution (not just its value), **walk back** through the table.

MEMO VS TABLE

- Memo: only visits needed states; easy to write
- Table: no stack limit; often faster; fill order matters
- Fill order = an order where dependencies come first
- Space: n² table → two rows → one row

TABLE, THEN ROLLING VARIABLES

```
function climbStairs(n)              // O(n) space
    dp ← new Array(n + 1, 0)
    (dp[0], dp[1]) ← (1, 1)
    for i ← 2 to n                   // dependencies first
        dp[i] ← dp[i − 1] + dp[i − 2]
    return dp[n]

function climbStairsO1(n)            // O(1) space
    (prev2, prev1) ← (1, 1)
    for i ← 2 to n
        (prev2, prev1) ← (prev1, prev1 + prev2)
    return prev1
```

TWO-ROW TRICK (LCS LENGTH)

```
function lcsLength(a, b)             // O(m) space
    prev ← new Array(m + 1, 0)
    for i ← 1 to n
        cur ← new Array(m + 1, 0)
        for j ← 1 to m
            if a[i-1] = b[j-1]
                cur[j] ← prev[j-1] + 1
            else
                cur[j] ← max(prev[j], cur[j-1])
        prev ← cur                   // slide down
    return prev[m]
```

RECONSTRUCTING THE ANSWER (LCS STRING)

```
// needs the full dp table from Sheet 083
(i, j, out) ← (n, m, [])
while i > 0 and j > 0
    if a[i-1] = b[j-1]
        out.append(a[i-1])           // part of the LCS
        (i, j) ← (i − 1, j − 1)
    else if dp[i-1][j] ≥ dp[i][j-1]
        i ← i − 1                    // came from above
    else
        j ← j − 1                    // came from the left
return reverse(out)
```

FILL ORDER

DEPENDENCY	LOOP
`dp[i-1], dp[i-2]`	i ascending
`dp[i+1] (suffix)`	i descending
`dp[i-1][j], dp[i][j-1]`	rows, then cols
`dp[i+1][j-1] (ranges)`	by length
`0/1 knapsack in 1D`	capacity descending

⚠ COMMON MISTAKES

- Wrong fill order: reading a cell that hasn't been computed yet
- Dropping to one row when cells need values from the **same** row's old version
- Space-optimizing away the table you later need to reconstruct the answer

✓ TIPS & BEST PRACTICES

- Write the memo version first, then convert: each recursive call becomes a table read
- Pad tables with an extra row/column of base values to avoid bounds checks
- Keep the full table in interviews unless asked to optimize space

QUICK SUMMARY

- tabulation — bottom-up loops
- rolling variables — O(1) for 1D
- two rows — O(m) for 2D
- walk back — rebuild the solution

1D DP Patterns

climbing stairs · house robber · decode ways · coin change (min coins) · word break · jump game

ONE INDEX OF STATE

In **1D DP**, `dp[i]` answers the question for a prefix (first i items) or a suffix. Each problem differs only in its **transition**: which earlier states feed `dp[i]` and how they combine (add for counting ways, `min` / `max` for optimizing, `or` for feasibility). Most compress to O(1) space with rolling variables.

RECURRENCES

PROBLEM	DP[I] =
Climbing Stairs	`dp[i-1] + dp[i-2]`
House Robber	`max(dp[i-1], dp[i-2] + x)`
Decode Ways	1-digit + 2-digit options
Coin Change	`min(dp[i - c] + 1)`
Word Break	any `dp[j]` and `s[j..i)` is a word
Jump Game	greedy: `reach ← max(reach, i + a[i])`

HOUSE ROBBER

```
function rob(nums)
    (prev2, prev1) ← (0, 0)
    for each x in nums
        cur ← max(prev1, prev2 + x)  // skip or take
        (prev2, prev1) ← (prev1, cur)
    return prev1
// circular street (Robber II):
// max(rob(nums[1..]), rob(nums[..n-2]))
```

DECODE WAYS

```
function numDecodings(s)
    prev2 ← 1                          // dp[i-2]
    prev1 ← if s[0] = '0' then 0 else 1
    for i ← 1 to length(s) - 1
        cur ← 0
        if s[i] ≠ '0'
            cur ← prev1                // one digit
        two ← toInt(s[i-1..i])
        if two ≥ 10 and two ≤ 26
            cur ← cur + prev2          // two digits
        (prev2, prev1) ← (prev1, cur)
    return prev1
```

COIN CHANGE (TABULATED)

```
function coinChange(coins, amount)
    dp ← new Array(amount + 1, ∞)
    dp[0] ← 0
    for a ← 1 to amount
        for each c in coins
            if c ≤ a
                dp[a] ← min(dp[a], dp[a - c] + 1)
    if dp[amount] = ∞ return -1
    return dp[amount]
```

WORD BREAK

```
function wordBreak(s, words)
    dict ← toSet(words)
    dp ← new Array(n + 1, false)  // prefix len i OK?
    dp[0] ← true
    for i ← 1 to n
        for j ← 0 to i - 1
            if dp[j] and dict.has(s[j..i-1])
                dp[i] ← true
                break
    return dp[n]
```

⚠ COMMON MISTAKES

- Decode Ways: treating `"06"` as valid; a leading zero makes a two-digit code invalid
- Coin Change: initializing `dp` with 0 instead of ∞, so `min` never moves
- Word Break: building substrings longer than the longest word (wasted O(n) slices)

✓ TIPS & BEST PRACTICES

- Practice: Climbing Stairs, Min Cost Climbing Stairs, House Robber I/II, Decode Ways, Coin Change, Word Break
- Limit Word Break's inner loop to `i - maxWordLen … i - 1`
- Jump Game is DP-shaped but a greedy reach pointer solves it in O(n)

QUICK SUMMARY

- `dp[i]` — answer for the first i items
- add — counting · min/max — optimizing
- rolling variables — O(1) space
- base case `dp[0]` — the empty prefix

2D Grid DP

unique paths · obstacles · minimum path sum · maximal square · dungeon game · triangle

DP ON A GRID OF CELLS

When you move through a grid only **right and down** (or along rows), each cell's answer depends on the cell above and the cell to the left, so a row-by-row fill works. Counting paths **adds** the two; cheapest paths take the **min**. Some problems are easier backward (from the goal), and padding the table with an extra row/column removes edge checks.

RECOGNIZE IT WHEN

- Moves restricted to right/down, or row to row
- "Number of unique paths", "minimum path sum"
- "Largest square / rectangle of 1s"
- "Minimum health / fuel needed" → fill from the end

UNIQUE PATHS WITH OBSTACLES

```
function uniquePaths(grid)        // 1 = blocked
    dp ← R × C grid of 0
    for r ← 0 to R − 1
        for c ← 0 to C − 1
            if grid[r][c] = 1
                dp[r][c] ← 0
            else if r = 0 and c = 0
                dp[r][c] ← 1
            else
                up ← if r > 0 then dp[r−1][c] else 0
                left ← if c > 0 then dp[r][c−1] else 0
                dp[r][c] ← up + left
    return dp[R−1][C−1]
// min path sum: grid[r][c] + min(up, left)
```

MAXIMAL SQUARE

```
function maximalSquare(m)
    dp ← (R+1) × (C+1) grid of 0  // padded
    best ← 0
    for r ← 1 to R
        for c ← 1 to C
            if m[r−1][c−1] = '1'
                dp[r][c] ← 1 + min(dp[r−1][c],
                                   dp[r][c−1],
                                   dp[r−1][c−1])
                best ← max(best, dp[r][c])
    return best * best
// dp[r][c] = side of the biggest square
// whose bottom-right corner is (r, c)
```

TRIANGLE (BOTTOM-UP, 1 ROW)

```
function minimumTotal(tri)
    dp ← copy(tri[last row])
    for i ← rows − 2 down to 0
        for j ← 0 to i
            dp[j] ← tri[i][j] + min(dp[j], dp[j + 1])
    return dp[0]
```

DUNGEON GAME (FILL FROM THE GOAL)

```
function minHP(d)                 // need ≥ 1 HP always
    need ← (R+1) × (C+1) grid of ∞
    need[R][C−1] ← 1
    need[R−1][C] ← 1
    for r ← R − 1 down to 0
        for c ← C − 1 down to 0
            next ← min(need[r+1][c], need[r][c+1])
            need[r][c] ← max(1, next − d[r][c])
    return need[0][0]
```

⚠ COMMON MISTAKES

- Initializing the whole first row to 1 when an obstacle sits in it: cells past it are 0
- Filling Dungeon Game forward: the answer depends on the future, so go backward
- Returning the max side instead of the area in Maximal Square

✓ TIPS & BEST PRACTICES

- Practice: Unique Paths I/II, Minimum Path Sum, Triangle, Maximal Square, Dungeon Game
- Unique Paths has a closed form: C(R + C − 2, R − 1)
- Reduce to one row: `dp[c] ← dp[c] + dp[c − 1]`

QUICK SUMMARY

- `dp[r][c]` from up and left
- add — count · min — cost
- pad the table — no bounds checks
- backward fill — requirements depend on the future

Knapsack Family

0/1 knapsack · unbounded knapsack · reverse capacity loop · partition equal subset sum · target sum · coin change II (ways)

PICK ITEMS UNDER A CAPACITY

In the **0/1 knapsack**, each item is taken at most once to maximize value within a weight limit. In 1D form, loop capacity **backward** so an item can't be reused in the same pass. Looping **forward** gives the **unbounded** version (unlimited copies). Many problems are knapsack in disguise: subset sums, target sums, and coin combinations.

DISGUISES

- Partition into equal halves → subset sum = total / 2
- Target Sum (±) → subset sum = (total + target) / 2
- Coin Change II → unbounded, count ways
- Ones and Zeroes → 2D capacity (zeros, ones)

0/1 VS UNBOUNDED (1D ARRAYS)

```
function knapsack01(wt, val, W)          // each item at most once
    dp ← new Array(W + 1, 0)             // dp[w] = best value at capacity w
    for i ← 0 to n - 1
        for w ← W down to wt[i]          // BACKWARD: no reuse this pass
            dp[w] ← max(dp[w], dp[w - wt[i]] + val[i])
    return dp[W]

function knapsackUnbounded(wt, val, W)   // unlimited copies
    dp ← new Array(W + 1, 0)
    for i ← 0 to n - 1
        for w ← wt[i] to W               // FORWARD: item may repeat
            dp[w] ← max(dp[w], dp[w - wt[i]] + val[i])
    return dp[W]
```

CODE EXPLAINED

- → 2D form: `dp[i][w] = max(skip, take)`
- → backward loop — reads values from before item i
- → forward loop — reads values that may include item i
- → time O(n · W), space O(W)
- → W up to ~10^5 is fine; 10^9 is not

PARTITION EQUAL SUBSET SUM

```
function canPartition(nums)
    total ← sum(nums)
    if total mod 2 = 1 return false
    target ← total div 2
    dp ← new Array(target + 1, false)
    dp[0] ← true
    for each x in nums
        for s ← target down to x
            dp[s] ← dp[s] or dp[s - x]
    return dp[target]
```

COIN CHANGE II (COUNT COMBINATIONS)

```
function change(amount, coins)
    dp ← new Array(amount + 1, 0)
    dp[0] ← 1
    for each c in coins              // coins OUTER
        for a ← c to amount
            dp[a] ← dp[a] + dp[a - c]
    return dp[amount]
// amount outer, coins inner counts
// ORDERED ways (Combination Sum IV)
```

⚠ COMMON MISTAKES

- Looping capacity forward in 0/1 knapsack: silently allows reusing an item
- Swapping loop order in Coin Change II: counts [1,2] and [2,1] separately
- Target Sum: skipping the check that `total + target` is even and non-negative

✓ TIPS & BEST PRACTICES

- Practice: Partition Equal Subset Sum, Target Sum, Coin Change II, Last Stone Weight II, Ones and Zeroes
- Feasibility uses `or`, counting uses `+`, optimizing uses `max` / `min`
- Big sums with small n? Meet-in-the-middle or bitset shifts instead

QUICK SUMMARY

- 0/1 — capacity backward
- unbounded — capacity forward
- items outer — combinations
- O(n · W) — pseudo-polynomial

String DP

longest common subsequence · edit distance · longest palindromic subsequence · distinct subsequences · wildcard / regex matching

TWO STRINGS, ONE TABLE

Two-string DP uses `dp[i][j]` = the answer for the prefixes `a[0..i-1]` and `b[0..j-1]`. Each cell looks at three neighbors: **diagonal** (both characters used, usually when they match), **up** (drop a char of a), and **left** (drop a char of b). Row 0 and column 0 hold the empty-string base cases.

RECOGNIZE IT WHEN

- Two sequences compared: subsequence, alignment, edits
- "Minimum operations to convert a into b"
- Pattern matching with `?`, `*`, or `.`
- One string vs its reverse → palindromic subsequence

LONGEST COMMON SUBSEQUENCE

```
function lcs(a, b)
    dp ← (n+1) × (m+1) grid of 0
    for i ← 1 to n
        for j ← 1 to m
            if a[i-1] = b[j-1]
                dp[i][j] ← dp[i-1][j-1] + 1
            else
                dp[i][j] ← max(dp[i-1][j],
                               dp[i][j-1])
    return dp[n][m]
// LPS(s) = lcs(s, reverse(s))
```

EDIT DISTANCE

```
function minDistance(a, b)
    dp[i][0] ← i, dp[0][j] ← j      // base
    for i ← 1 to n
        for j ← 1 to m
            if a[i-1] = b[j-1]
                dp[i][j] ← dp[i-1][j-1]
            else
                dp[i][j] ← 1 + min(
                    dp[i-1][j],      // delete
                    dp[i][j-1],      // insert
                    dp[i-1][j-1])    // replace
    return dp[n][m]
```

WILDCARD MATCHING (? AND *)

```
function isMatch(s, p)
    dp ← (n+1) × (m+1) grid of false
    dp[0][0] ← true
    for j ← 1 to m                  // * can match empty
        dp[0][j] ← dp[0][j-1] and p[j-1] = '*'
    for i ← 1 to n
        for j ← 1 to m
            if p[j-1] = '*'
                dp[i][j] ← dp[i][j-1] or dp[i-1][j]
            else if p[j-1] = '?' or p[j-1] = s[i-1]
                dp[i][j] ← dp[i-1][j-1]
    return dp[n][m]
```

MORE RECURRENCES

PROBLEM	WHEN CHARS MATCH / ELSE
Distinct Subsequences	`up + diag` / `up`
Shortest Common Supersequence	`n + m - lcs`
Delete Ops for Two Strings	`n + m - 2·lcs`
Interleaving String	`dp[i][j]` from up or left
Regex (`.` and `x*`)	`*`: zero copies `[j-2]` or one more `[i-1]`

⚠ COMMON MISTAKES

- Off-by-one between `dp` indexes and string indexes: `dp[i]` refers to `a[i - 1]`
- Forgetting base row/column: edit distance needs `dp[i][0] = i`
- Regex `x*`: treating `*` alone instead of as a pair with the char before it

✓ TIPS & BEST PRACTICES

- Practice: LCS, Edit Distance, Longest Palindromic Subsequence, Distinct Subsequences, Wildcard / Regex Matching
- Draw the table for two 3-letter strings before coding
- Only the previous row is needed for the length: O(min(n, m)) space

QUICK SUMMARY

- `dp[i][j]` — prefixes of a and b
- diagonal — both chars used
- up / left — skip a char
- row 0 / col 0 — empty-string base

Longest Increasing Subsequence

O(n²) DP · O(n log n) tails + binary search · number of LIS · Russian doll envelopes · longest chain

LONGEST INCREASING SUBSEQUENCE

The **LIS** is the longest set of items, in original order (not necessarily adjacent), that strictly increases. The O(n²) DP asks, for each i, which earlier smaller item gives the longest run ending at i. The O(n log n) version keeps `tails[k]`, the **smallest possible tail** of an increasing run of length k + 1, and binary-searches where each new value fits.

RECOGNIZE IT WHEN

- "Longest increasing / non-decreasing subsequence"
- Nesting items: envelopes, boxes, stacking
- "Longest chain of pairs"
- Minimum deletions to make a sequence sorted = n - LIS

O(N²) DP

```
function lengthOfLIS(nums)
    dp ← new Array(n, 1)      // LIS ending at i
    for i ← 1 to n - 1
        for j ← 0 to i - 1
            if nums[j] < nums[i]
                dp[i] ← max(dp[i], dp[j] + 1)
    return max(dp)
```

O(N LOG N) WITH TAILS[]

```
function lisFast(nums)
    tails ← []
    for each x in nums
        i ← lowerBound(tails, x)   // first ≥ x
        if i = length(tails)
            tails.append(x)        // longer run
        else
            tails[i] ← x           // smaller tail
    return length(tails)
// non-decreasing: use upperBound
```

NUMBER OF LIS

```
function findNumberOfLIS(nums)
    len ← new Array(n, 1)
    cnt ← new Array(n, 1)
    for i ← 0 to n - 1
        for j ← 0 to i - 1
            if nums[j] < nums[i]
                if len[j] + 1 > len[i]
                    (len[i], cnt[i]) ← (len[j] + 1, cnt[j])
                else if len[j] + 1 = len[i]
                    cnt[i] ← cnt[i] + cnt[j]
    L ← max(len)
    return sum of cnt[i] where len[i] = L
```

TAILS[] FOR [10,9,2,5,3,7,101,18]

X	TAILS
10, 9, 2	[2]
5	[2, 5]
3	[2, 3]
7	[2, 3, 7]
101	[2, 3, 7, 101]
18	[2, 3, 7, 18] → 4

⚠ COMMON MISTAKES

- Reading `tails` as the actual subsequence: it's only valid for its length
- Russian Doll Envelopes sorted by height ascending on ties: equal widths nest wrongly
- Using `upperBound` for strictly increasing runs (or `lowerBound` for non-decreasing)

✓ TIPS & BEST PRACTICES

- Russian Doll: sort width ↑, height ↓ on ties, then LIS on heights
- Longest chain of pairs: sort by end and pick greedily (O(n log n))
- Practice: LIS, Number of LIS, Russian Doll Envelopes, Max Length of Pair Chain

QUICK SUMMARY

- `dp[i]` — LIS ending at i, O(n²)
- `tails` + binary search — O(n log n)
- sort + LIS — 2D nesting
- n - LIS — deletions to sort

Interval DP

dp[i][j] over ranges · iterate by length · burst balloons · matrix chain multiplication · min cuts palindrome partitioning

DP OVER SUB-RANGES

Interval DP stores `dp[i][j]`, the answer for the slice `i..j`, built from smaller slices. Fill it by **increasing length** so every shorter range is ready. Many problems try every **split point** k inside the range; others (like Burst Balloons) pick the element handled **last**, which makes the two sides independent.

RECOGNIZE IT WHEN

- Merging / bursting / cutting where neighbors change
- "Optimal order to combine a sequence"
- Palindromic substructure inside a string
- n ≤ ~500 ($O(n^3)$ is fine)

THE SHAPE

```
for len ← 1 to n              // short ranges first
    for i ← 0 to n - len
        j ← i + len - 1
        if len = 1
            dp[i][j] ← base(i)
        else
            dp[i][j] ← best over k in i..j-1 of
                combine(dp[i][k], dp[k+1][j], i, k, j)

// Matrix Chain: dims d[0..n]
// combine = left + right + d[i]·d[k+1]·d[j+1]
```

BURST BALLOONS (LAST ONE WINS)

```
function maxCoins(nums)
    a ← [1] + nums + [1]
    m ← length(a)
    dp ← m × m grid of 0        // open range (l, r)
    for len ← 2 to m - 1
        for l ← 0 to m - 1 - len
            r ← l + len
            for k ← l + 1 to r - 1    // k burst LAST
                gain ← a[l] * a[k] * a[r]
                dp[l][r] ← max(dp[l][r],
                    dp[l][k] + gain + dp[k][r])
    return dp[0][m - 1]
```

PALINDROME PARTITIONING II (MIN CUTS)

```
function minCut(s)
    isPal ← n × n grid of false
    for len ← 1 to n                        // interval fill
        for i ← 0 to n - len
            j ← i + len - 1
            isPal[i][j] ← s[i] = s[j] and (len ≤ 2 or isPal[i+1][j-1])
    cuts ← new Array(n, 0)
    for j ← 0 to n - 1
        if isPal[0][j]
            cuts[j] ← 0
        else
            cuts[j] ← min over i in 1..j with isPal[i][j] of cuts[i-1] + 1
    return cuts[n - 1]
```

⚠ COMMON MISTAKES

- Filling by `i` then `j` instead of by length: reads ranges not yet computed
- Burst Balloons by choosing the **first** balloon: the sides still depend on each other
- Forgetting the padded `1`s around Burst Balloons

✓ TIPS & BEST PRACTICES

- Practice: Burst Balloons, Minimum Cost Tree From Leaf Values, Palindrome Partitioning II, Strange Printer, Stone Game
- Longest Palindromic Subsequence: `dp[i+1][j-1] + 2` if ends match, else max of the two shrinks
- Memoized `solve(i, j)` avoids thinking about fill order

QUICK SUMMARY

- `dp[i][j]` — answer for the range
- iterate by length
- split point k — combine halves
- last element — independent sides

State Machine DP

hold / not-hold states · stock with cooldown · stock with fee · at most k transactions · paint house

A FEW STATES PER STEP

Some problems are a walk through days or items where you're always in one of a few **states** (holding a stock or not, cooling down, which color was used last). Keep one value per state, and for each new item compute every state's best from the previous step's states using the allowed **transitions**. It's 1D DP with a small extra dimension.

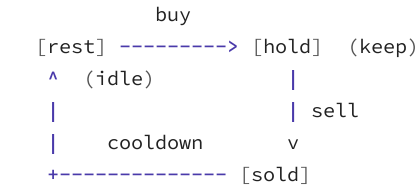
RECOGNIZE IT WHEN

- Stock problems with cooldown, fees, or k trades
- "Adjacent items can't share a color / choice"
- Rules like "after X you must Y"
- Draw the state diagram first; code follows it

STOCK WITH COOLDOWN

```
function maxProfit(prices)
    (hold, sold, rest) ← (-∞, 0, 0)
    for each p in prices
        (hold, sold, rest) ←
            (max(hold, rest - p),    // keep or buy
             hold + p,               // sell today
             max(rest, sold))        // idle / cool down
    return max(sold, rest)
```

STATE DIAGRAM



AT MOST K TRANSACTIONS

```
function maxProfitK(k, prices)
    buy ← new Array(k + 1, -∞)
    sell ← new Array(k + 1, 0)
    for each p in prices
        for t ← 1 to k
            buy[t] ← max(buy[t], sell[t - 1] - p)
            sell[t] ← max(sell[t], buy[t] + p)
    return sell[k]
// k ≥ n/2 → unlimited: sum every rise
```

FEE · PAINT HOUSE

```
function maxProfitFee(prices, fee)
    (hold, cash) ← (-prices[0], 0)
    for each p in prices[1..]
        (hold, cash) ← (max(hold, cash - p),
                        max(cash, hold + p - fee))
    return cash

function minCost(costs)    // 3 colors
    (r, g, b) ← (0, 0, 0)
    for each (cr, cg, cb) in costs
        (r, g, b) ← (cr + min(g, b),
                     cg + min(r, b),
                     cb + min(r, g))
    return min(r, g, b)
```

⚠ COMMON MISTAKES

- Updating states one at a time so a new value feeds another update in the same step
- Allowing a buy straight from `sold` in the cooldown version
- Starting `hold` at 0: you can't hold a stock you never paid for

✓ TIPS & BEST PRACTICES

- Practice: Best Time to Buy and Sell Stock II, III, IV, with Cooldown, with Fee; Paint House
- Compute all new states from a saved tuple of old ones
- Stock III is the k = 2 case of the k-transaction code

QUICK SUMMARY

- one variable per state
- transitions — from the state diagram
- simultaneous update — tuple assignment
- answer — best of the end states

Tree & DAG DP

post-order DP · house robber III · tree diameter · longest path in DAG · rerooting · binary tree cameras

DP ON TREES AND DAGS

On a tree, compute each node's DP values in **post-order**: children first, then combine. Return a small **tuple** of states (e.g. best if this node is used vs. not used). On a DAG, process nodes in **topological order** so every predecessor is final before you use it. **Rerooting** gets the answer for every possible root in two passes.

RECOGNIZE IT WHEN

- Choose nodes with rules about parent/child ("no two adjacent")
- Longest / heaviest path in a tree or DAG
- Answer needed "for every node as root"
- Covering nodes (cameras, guards) at minimum cost

HOUSE ROBBER III

```
function rob(root)
    function dfs(node)        // → (robThis, skipThis)
        if node = null return (0, 0)
        (lr, ls) ← dfs(node.left)
        (rr, rs) ← dfs(node.right)
        robThis ← node.val + ls + rs
        skipThis ← max(lr, ls) + max(rr, rs)
        return (robThis, skipThis)
    return max(dfs(root))
```

LONGEST PATH IN A DAG

```
function longestPath(n, adj)  // adj[u] = [(v, w)]
    order ← topoSort(n, adj)  // Sheet 049
    dist ← new Array(n, 0)
    for each u in order
        for each (v, w) in adj[u]
            dist[v] ← max(dist[v], dist[u] + w)
    return max(dist)
// shortest path in a DAG: same, with min
// (works with negative weights)
```

REROOTING: SUM OF DISTANCES IN TREE

```
function sumOfDistances(n, adj)
    size ← new Array(n, 1)
    ans ← new Array(n, 0)
    function post(u, p)             // root 0 answer
        for each v in adj[u] where v ≠ p
            post(v, u)
            size[u] ← size[u] + size[v]
            ans[u] ← ans[u] + ans[v] + size[v]
    function pre(u, p)              // shift root u → v
        for each v in adj[u] where v ≠ p
            ans[v] ← ans[u] − size[v] + (n − size[v])
            pre(v, u)
    post(0, −1)
    pre(0, −1)
    return ans
```

BINARY TREE CAMERAS

```
function minCameras(root)
    cams ← 0
    // 0 needs cover · 1 covered · 2 camera
    function dfs(node)
        if node = null return 1
        (l, r) ← (dfs(node.left), dfs(node.right))
        if l = 0 or r = 0
            cams ← cams + 1
            return 2
        if l = 2 or r = 2 return 1
        return 0
    if dfs(root) = 0
        cams ← cams + 1
    return cams
```

⚠ COMMON MISTAKES

- Using `max(robThis, skipThis)` of a child when the parent is robbed: the child must be skipped
- Longest path in a general graph with cycles: NP-hard; DAG only
- Rerooting with only one pass: the second (top-down) pass is what moves the root

✓ TIPS & BEST PRACTICES

- Practice: House Robber III, Binary Tree Max Path Sum, Diameter, Sum of Distances in Tree, Cameras
- Return tuples instead of recomputing children for each state
- Iterative post-order avoids recursion limits on 10^5-node trees

QUICK SUMMARY

- post-order — children before parent
- tuple of states — used / not used
- topological order — DAG DP
- rerooting — every node as root, O(n)

Bitmask DP

dp[mask] · visited-set as integer · traveling salesman · task assignment · partition to k equal subsets · n ≤ 20 signal

SUBSETS AS INTEGERS

When n is small (**≤ 20**), the set of items already used fits in an integer **mask**: bit i is 1 if item i is used. `dp[mask]` (optionally with the last item) stores the best answer for that subset, and transitions add one unused item. There are 2^n masks, so expect **$O(2^n \cdot n)$** or **$O(2^n \cdot n^2)$** time.

BIT OPERATIONS USED

- Test: `(mask >> i) & 1 = 1`
- Add: `mask | (1 << i)`
- Full set: `(1 << n) - 1`
- Submasks: `sub ← (sub - 1) & mask` until 0

TRAVELING SALESMAN (HELD-KARP)

```
function tsp(dist)                          // n ≤ ~16-20
    FULL ← (1 << n) - 1
    dp ← 2^n × n grid of ∞                  // dp[mask][i]: visited, at i
    dp[1][0] ← 0                            // start at city 0
    for mask ← 1 to FULL
        for i ← 0 to n - 1
            if dp[mask][i] = ∞ continue
            for j ← 0 to n - 1
                if (mask >> j) & 1 = 1 continue    // already visited
                nm ← mask | (1 << j)
                dp[nm][j] ← min(dp[nm][j], dp[mask][i] + dist[i][j])
    return min over i of dp[FULL][i] + dist[i][0]
```

CODE EXPLAINED

- → mask — set of visited cities
- → `dp[mask][i]` — cheapest way to visit mask ending at i
- → transition — go to an unvisited j
- → close the tour back to city 0
- → $O(2^n \cdot n^2)$ vs n! brute force

ASSIGNMENT (WORKERS → TASKS)

```
function assign(cost)          // cost[worker][task]
    dp ← new Array(2^n, ∞)
    dp[0] ← 0
    for mask ← 0 to 2^n - 1
        w ← popcount(mask)     // next worker
        if w = n continue
        for t ← 0 to n - 1
            if (mask >> t) & 1 = 0
                nm ← mask | (1 << t)
                dp[nm] ← min(dp[nm],
                             dp[mask] + cost[w][t])
    return dp[2^n - 1]
```

PARTITION TO K EQUAL SUBSETS

```
function canPartitionK(nums, k)
    if sum(nums) mod k ≠ 0 return false
    target ← sum(nums) div k
    dp ← new Array(2^n, -1)  // current bucket fill
    dp[0] ← 0
    for mask ← 0 to 2^n - 1
        if dp[mask] = -1 continue
        for i ← 0 to n - 1
            if (mask >> i) & 1 = 0
                    and dp[mask] + nums[i] ≤ target
                nm ← mask | (1 << i)
                dp[nm] ← (dp[mask] + nums[i]) mod target
    return dp[2^n - 1] = 0
```

⚠ COMMON MISTAKES

- Using bitmask DP with n = 40: 2^{40} states won't fit; look for another approach
- Operator precedence: `mask >> i & 1 = 1` needs parentheses in most languages
- Allocating `2^n × n` arrays of 64-bit values without checking memory (n = 20 → 160 MB)

✓ TIPS & BEST PRACTICES

- The small-n constraint (≤ 20) is the main signal for bitmasks
- Process masks in increasing order: adding a bit always makes the mask larger
- Practice: Shortest Path Visiting All Nodes, Partition to K Equal Sum Subsets, Can I Win

QUICK SUMMARY

- mask — subset as an integer
- `dp[mask]` / `dp[mask][last]` — state
- `mask | (1 << j)` — add an item
- $O(2^n \cdot n)$ — only for small n

Greedy Fundamentals

TAKE THE BEST CHOICE NOW

A **greedy** algorithm makes the locally best choice at each step and never revisits it. It's correct only when the problem has the **greedy-choice property**: some optimal solution starts with the greedy choice. Prove it with an **exchange argument** (swapping any optimal solution's first choice for the greedy one doesn't make it worse), or find a counterexample.

RECOGNIZE IT WHEN

- A natural "best next" rule exists (earliest end, cheapest, farthest)
- Sorting reveals the order to decide in
- DP works but every state picks the same kind of option
- Reach / coverage questions ("can you get to the end?")

JUMP GAME I AND II

```
function canJump(nums)
    reach ← 0                    // farthest index so far
    for i ← 0 to length(nums) - 1
        if i > reach return false
        reach ← max(reach, i + nums[i])
    return true

function minJumps(nums)          // BFS by ranges
    (jumps, end, far) ← (0, 0, 0)
    for i ← 0 to length(nums) - 2
        far ← max(far, i + nums[i])
        if i = end               // current jump used up
            jumps ← jumps + 1
            end ← far
    return jumps
```

GAS STATION

```
function canCompleteCircuit(gas, cost)
    if sum(gas) < sum(cost) return -1
    (start, tank) ← (0, 0)
    for i ← 0 to length(gas) - 1
        tank ← tank + gas[i] - cost[i]
        if tank < 0
            start ← i + 1        // no start in
            tank ← 0             // [start..i] works
    return start

// enough total gas guarantees the
// last reset point succeeds
```

GREEDY WORKS VS FAILS

PROBLEM	GREEDY?	WHY
activity selection	yes	earliest end leaves most room
coin change, US coins	yes	canonical coin system
coin change [1, 3, 4], 6	no	greedy 4+1+1, best 3+3
fractional knapsack	yes	take best value/weight first
0/1 knapsack	no	can't split items: use DP
shortest path ≥ 0 weights	yes	Dijkstra is greedy

EXCHANGE ARGUMENT

STEP	DO
1	take any optimal solution O
2	find its first difference from greedy G
3	swap in G's choice
4	show the result is no worse
5	repeat → G is optimal

⚠ COMMON MISTAKES

- Trusting a greedy rule without testing small counterexamples
- Gas Station: restarting at `start + 1` instead of `i + 1` (O(n²) and still wrong)
- Using greedy for 0/1 knapsack or general coin change: those need DP

✓ TIPS & BEST PRACTICES

- Brute-force small inputs and compare with the greedy answer before trusting it
- Most greedy solutions start with a sort; ask "sorted by what?"
- Practice: Jump Game I/II, Gas Station, Assign Cookies, Lemonade Change

QUICK SUMMARY

- local best choice, never undone
- exchange argument — proof
- counterexample — disproof
- often sort + one pass, O(n log n)

Greedy Patterns

REUSABLE GREEDY MOVES

Most greedy interview problems use one of a handful of moves: **sort by end time** and keep what fits, schedule the **most frequent** item first, **extend** a segment until everything inside it is closed, **sort by a difference** (cost saved), or **merge the two smallest** repeatedly (Huffman). Recognizing the move is most of the work.

QUICK FACTS

- Activity selection: sort by end, keep non-overlapping (Sheet 069)
- Huffman coding: optimal prefix code via a min-heap
- Scheduling with cooldown: the most frequent task sets the frame
- Pairing problems: sort, then two pointers

TASK SCHEDULER

```
function leastInterval(tasks, n)    // cooldown n
    freq ← counts of each task
    maxF ← max(freq)
    numMax ← tasks with frequency maxF
    // A _ _ A _ _ A   (maxF - 1) frames of n + 1
    frame ← (maxF - 1) * (n + 1) + numMax
    return max(frame, length(tasks))

// if tasks overflow the frame,
// no idle slots are needed
```

PARTITION LABELS

```
function partitionLabels(s)
    last ← map char → last index in s
    (start, end, out) ← (0, 0, [])
    for i ← 0 to length(s) - 1
        end ← max(end, last[s[i]])
        if i = end              // all chars closed
            out.append(end - start + 1)
            start ← i + 1
    return out
// "ababcbacadefegdehijhklij" → [9, 7, 8]
```

HUFFMAN CODING (TOTAL BITS)

```
function huffmanCost(freqs)
    heap ← new MinHeap(freqs)
    total ← 0
    while heap.size() > 1
        a ← heap.pop()          // two rarest
        b ← heap.pop()
        total ← total + a + b   // merge cost
        heap.push(a + b)
    return total
// same shape: Min Cost to Connect Sticks
```

MORE GREEDY MOVES

PROBLEM	MOVE
Two City Scheduling	sort by `costA - costB`, first n go to A
Hand of Straights	smallest card starts a run of W
Assign Cookies	sort both, match smallest that fits
Boats to Save People	sort, pair heaviest with lightest
Min Arrows to Burst Balloons	sort by end, shoot at end
Queue Reconstruction by Height	tallest first, insert at k

⚠ COMMON MISTAKES

- Task Scheduler: forgetting `max(frame, length(tasks))` when there are many task types
- Partition Labels: cutting at the first repeat instead of the furthest last index
- Hand of Straights: starting runs from arbitrary cards instead of the smallest

✓ TIPS & BEST PRACTICES

- Write the greedy rule in one sentence before coding it
- Ties matter: decide the tie-break order explicitly when sorting
- Practice: Task Scheduler, Partition Labels, Two City Scheduling, Hand of Straights

QUICK SUMMARY

- sort by end — fit the most intervals
- most frequent first — scheduling
- extend to last occurrence — partitions
- merge two smallest — Huffman

Tries (Prefix Trees)

node children map / array[26] · end-of-word flag · insert / search / startsWith · word search II · autocomplete · wildcard search

A TREE OF PREFIXES

A **trie** stores strings character by character along a path from the root; words that share a prefix share nodes. Insert, search, and prefix checks take **O(L)** for a word of length L, independent of how many words are stored. Mark nodes where a word ends with a flag (or store the word there).

RECOGNIZE IT WHEN

- Many prefix queries: autocomplete, "starts with"
- Searching many words at once (Word Search II)
- Wildcards like `.` matching any letter

TRIE

```
class Trie                      // node: children + isWord
    root ← new TrieNode()
    function insert(word)                  // O(L)
        node ← root
        for each ch in word
            if not node.children.has(ch)
                node.children[ch] ← new TrieNode()
            node ← node.children[ch]
        node.isWord ← true
    function walk(prefix)                  // node or null
        node ← root
        for each ch in prefix
            if not node.children.has(ch) return null
            node ← node.children[ch]
        return node
    function search(word)
        node ← walk(word)
        return node ≠ null and node.isWord
    function startsWith(prefix)
        return walk(prefix) ≠ null
```

WORD SEARCH II (TRIE-GUIDED DFS)

```
function findWords(board, words)
    root ← trie of words; end node stores .word
    out ← []
    function dfs(r, c, node)
        ch ← board[r][c]
        if ch = '#' or not node.children.has(ch) return
        node ← node.children[ch]
        if node.word ≠ null
            out.append(node.word)
            node.word ← null               // report once
        board[r][c] ← '#'
        for each (nr, nc) in inBoundsNeighbors(r, c)
            dfs(nr, nc, node)
        board[r][c] ← ch
    for each cell (r, c)
        dfs(r, c, root)
    return out
```

WILDCARD SEARCH ('.')

```
function searchDot(node, word, i)
    if i = length(word)
        return node.isWord
    ch ← word[i]
    if ch = '.'
        for each child in node.children
            if searchDot(child, word, i + 1)
                return true
        return false
    if not node.children.has(ch)
        return false
    return searchDot(node.children[ch],
                     word, i + 1)
```

PRACTICE

PROBLEM	TWIST
Implement Trie	insert / search / startsWith
Design Add and Search Words	`.` wildcard DFS
Word Search II	trie-guided board DFS
Replace Words	shortest root prefix
Autocomplete	walk prefix, then DFS

⚠ COMMON MISTAKES

- Treating any reached node as a word: check the end-of-word flag
- Running Word Search I for each word: one shared trie DFS is far faster
- Using `array[26]` when the alphabet is larger (Unicode, digits, symbols)

✓ TIPS & BEST PRACTICES

- Binary trie on bits: Maximum XOR of Two Numbers
- Prune leaf nodes after finding a word to speed up Word Search II
- Store counts at nodes to answer "how many words start with …"

QUICK SUMMARY

- path = prefix
- `isWord` — end-of-word flag
- O(L) — insert / search / prefix
- trie + DFS — many-word search

Segment Trees

build · range query · point update · array layout · lazy propagation · range sum / min / max

RANGE QUERIES WITH UPDATES

A **segment tree** stores an answer (sum, min, max, gcd…) for each segment of the array: the root covers everything, and each node splits its range in half. Any range query or point update touches **O(log n)** nodes. **Lazy propagation** postpones range updates by storing them at covering nodes until a query needs to go deeper.

RECOGNIZE IT WHEN

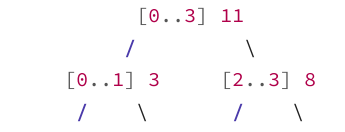
- Range sum / min / max with values that change
- Range updates ("add v to l..r") plus range queries
- Prefix sums fail because updates are frequent
- Counting by value ranges (with coordinate compression)

BUILD AND QUERY (RANGE SUM)

```
// node 1 = root; children 2i and 2i+1
// t has 4n slots
function build(a, node, l, r)
    if l = r
        t[node] ← a[l]
        return
    m ← (l + r) div 2
    build(a, 2 * node, l, m)
    build(a, 2 * node + 1, m + 1, r)
    t[node] ← t[2 * node] + t[2 * node + 1]

function query(node, l, r, ql, qr)
    if qr < l or r < ql return 0      // no overlap
    if ql ≤ l and r ≤ qr return t[node]   // inside
    m ← (l + r) div 2
    return query(2 * node, l, m, ql, qr)
         + query(2 * node + 1, m + 1, r, ql, qr)
```

SHAPE FOR A = [2, 1, 5, 3]




```
query(1..2) visits:
 [0..1] → partial → [1] = 1
 [2..3] → partial → [2] = 5
 total 6
```

POINT UPDATE

```
function update(node, l, r, i, val)
    if l = r
        t[node] ← val             // a[i] ← val
        return
    m ← (l + r) div 2
    if i ≤ m
        update(2 * node, l, m, i, val)
    else
        update(2 * node + 1, m + 1, r, i, val)
    t[node] ← t[2 * node] + t[2 * node + 1]
```

LAZY PROPAGATION (RANGE ADD)

```
// fully covered node: add v·(r − l + 1)
// to t[node], add v to lazy[node], stop
function push(node, l, r)          // before
    if lazy[node] ≠ 0               // going down
        m ← (l + r) div 2
        apply(2 * node, l, m, lazy[node])
        apply(2 * node + 1, m + 1, r, lazy[node])
        lazy[node] ← 0
function apply(node, l, r, v)
    t[node] ← t[node] + v * (r − l + 1)
    lazy[node] ← lazy[node] + v
```

⚠ COMMON MISTAKES

- Allocating `2n` slots for the recursive version: it needs up to `4n`
- Returning the wrong identity on no overlap: 0 for sum, ∞ for min, -∞ for max
- Forgetting `push` before recursing in lazy trees: children miss pending adds

✓ TIPS & BEST PRACTICES

- Sum-only with point updates? A Fenwick tree (Sheet 093) is shorter
- Iterative bottom-up segment trees use `2n` slots and no recursion
- Practice: Range Sum Query – Mutable, Count of Smaller Numbers After Self, My Calendar III

QUICK SUMMARY

- node covers `[l..r]` ; children split it
- query / update — O(log n)
- identity value — no-overlap result
- lazy — deferred range updates

Fenwick Trees (Binary Indexed Trees)

lowbit i & -i · prefix sum query · point update · 1-indexing · count of smaller after self · vs segment tree

PREFIX SUMS THAT CAN CHANGE

A **Fenwick tree** (binary indexed tree) supports **point update** and **prefix sum** in O(log n) with a single array and about ten lines of code. Index i stores the sum of a block that ends at i and has length `i & -i` (its **lowest set bit**). Updates climb by adding the lowbit; queries descend by subtracting it.

QUICK FACTS

- Arrays are **1-indexed** (index 0 unused)
- Range sum = `prefix(r) - prefix(l - 1)`
- Range add + point query: build it over a difference array
- Only for invertible operations like `+` (not min/max)

FENWICK TREE

```
class Fenwick
    function init(n)
        this.n ← n
        this.tree ← new Array(n + 1, 0)
    function add(i, delta)            // a[i] += delta
        while i ≤ this.n
            this.tree[i] ← this.tree[i] + delta
            i ← i + (i & -i)          // next covering block
    function prefix(i)                // a[1] + … + a[i]
        s ← 0
        while i > 0
            s ← s + this.tree[i]
            i ← i - (i & -i)          // drop lowest bit
        return s
    function rangeSum(l, r)
        return this.prefix(r) - this.prefix(l - 1)
```

WHAT TREE[I] COVERS

I	BINARY	COVERS
1	0001	a[1]
2	0010	a[1..2]
3	0011	a[3]
4	0100	a[1..4]
5	0101	a[5]
6	0110	a[5..6]
7	0111	a[7]
8	1000	a[1..8]

COUNT OF SMALLER NUMBERS AFTER SELF

```
function countSmaller(nums)
    ranks ← compress(nums)            // values → 1..m
    bit ← new Fenwick(m)
    out ← new Array(n, 0)
    for i ← n - 1 down to 0           // right to left
        out[i] ← bit.prefix(ranks[i] - 1)   // smaller, already seen
        bit.add(ranks[i], 1)
    return out
```

FENWICK VS SEGMENT TREE

	FENWICK	SEGMENT
code size	~10 lines	~30+ lines
memory	n + 1	up to 4n
ops	invertible (+, xor)	any associative
range update	via difference trick	lazy propagation

⚠ COMMON MISTAKES

- Using index 0: `i & -i` is 0 there, so the loops never move
- Forgetting to compress large values before using them as indexes
- Expecting range min/max: Fenwick can't subtract a min

✓ TIPS & BEST PRACTICES

- Shift a 0-indexed array by one: `add(i + 1, x)`
- Inversion count: same as Count Smaller, summed
- Practice: Range Sum Query – Mutable, Count of Smaller Numbers After Self, Reverse Pairs

QUICK SUMMARY

- `i & -i` — lowbit
- add — climb with `i += lowbit`
- prefix — descend with `i -= lowbit`
- O(log n) — both operations

LRU & LFU Caches

hash map + doubly linked list · O(1) get / put · eviction · frequency buckets · min-frequency tracking

FIXED-SIZE CACHES WITH O(1) OPERATIONS

An **LRU** cache evicts the **least recently used** key. Pair a **hash map** (key → node) with a **doubly linked list** ordered by recency: the map finds a node in O(1), the list moves or removes it in O(1). An **LFU** cache evicts the **least frequently used** key (ties: least recent), using frequency buckets and a running minimum frequency.

LIST HELPERS

```
function remove(node)
    node.prev.next ← node.next
    node.next.prev ← node.prev
function addFront(node)        // after head
    (node.prev, node.next) ← (head, head.next)
    head.next.prev ← node
    head.next ← node
```

LRU CACHE

```
class LRUCache
    function init(capacity)
        this.cap ← capacity
        this.map ← new Map()          // key → node
        (head, tail) ← (new Node(), new Node())
        link(head, tail)              // sentinels
    function get(key)
        if not this.map.has(key) return -1
        node ← this.map[key]
        remove(node)
        addFront(node)                // most recent
        return node.val
    function put(key, val)
        if this.map.has(key)
            remove(this.map[key])
        node ← new Node(key, val)
        addFront(node)
        this.map[key] ← node
        if length(this.map) > this.cap
            lru ← tail.prev           // oldest
            remove(lru)
            this.map.delete(lru.key)
```

LFU CACHE

```
class LFUCache
    vals, freq ← new Map(), new Map()
    buckets ← freq → ordered set of keys
    minF ← 0
    function touch(key)
        f ← freq[key]
        buckets[f].remove(key)
        if f = minF and buckets[f] is empty
            minF ← f + 1
        freq[key] ← f + 1
        buckets[f + 1].add(key)    // newest last
    function get(key)
        if not vals.has(key) return -1
        touch(key)
        return vals[key]
    function put(key, v)
        if cap = 0 return
        if vals.has(key)
            vals[key] ← v
            touch(key)
            return
        if length(vals) = cap
            old ← first key in buckets[minF]
            buckets[minF].remove(old)
            delete vals[old] and freq[old]
        (vals[key], freq[key], minF) ← (v, 1, 1)
        buckets[1].add(key)
```

⚠ COMMON MISTAKES

- Evicting before inserting the new key when the key already existed
- Storing only values in list nodes: eviction needs the **key** to delete from the map
- LFU: forgetting to reset `minF` to 1 when inserting a new key

✓ TIPS & BEST PRACTICES

- Shortcuts: Python `OrderedDict`, Java `LinkedHashMap` with access order
- Practice: LRU Cache, LFU Cache, Design Browser History
- Mention thread safety (a lock) if asked about production use

QUICK SUMMARY

- map + doubly linked list — LRU
- front = newest, back = evict
- freq buckets + `minF` — LFU
- sentinels — no edge cases

Sheet 095

Data Structure Design Problems

min stack · insert / delete / getRandom O(1) · time-based key-value store · peeking & flatten iterators · hit counter

COMPOSING STRUCTURES TO HIT A TARGET COST

Design problems ask for a class whose operations each meet a time bound. The approach is always the same: list the operations, note what each needs (lookup by key → map, random index → array, order → list or tree, recency → queue), and **combine** structures so every operation is fast. Keep them in sync on every change.

BUILDING BLOCKS

- Map + array → O(1) insert, delete, random pick
- Map of sorted lists + binary search → time-travel lookups
- Stack of iterators → flatten nested lists

INSERT / DELETE / GETRANDOM O(1)

```
class RandomizedSet
    vals ← []                    // for random index
    pos ← new Map()              // value → index
    function insert(x)
        if pos.has(x) return false
        pos[x] ← length(vals)
        vals.append(x)
        return true
    function remove(x)
        if not pos.has(x) return false
        i ← pos[x]
        last ← vals[length(vals) - 1]
        (vals[i], pos[last]) ← (last, i)   // fill hole
        vals.pop()
        pos.delete(x)
        return true
    function getRandom()
        return vals[randomInt(0, length(vals) - 1)]
```

TIME-BASED KEY-VALUE STORE

```
class TimeMap
    store ← map key → list of (t, value)
    function set(key, value, t)  // t increasing
        store[key].append((t, value))
    function get(key, t)
        list ← store[key]
        i ← upperBound(list, t) - 1
        if i < 0 return ""       // nothing ≤ t
        return list[i].value
class HitCounter                 // last 300 s
    q ← new Queue()
    function hit(t)
        q.enqueue(t)
    function getHits(t)
        while not q.isEmpty() and q.front() ≤ t - 300
            q.dequeue()
        return length(q)
```

FLATTEN NESTED LIST ITERATOR

```
class NestedIterator
    stack ← [iterator(nestedList)]
    function hasNext()
        while not stack.isEmpty()
            it ← stack.peek()
            if not it.hasNext()
                stack.pop()                  // level done
            else if it.peekNext() is an integer
                return true
            else                             // go deeper
                stack.push(iterator(it.next().getList()))
        return false
    function next()
        return stack.peek().next()
```

PRACTICE

PROBLEM	COMBO
Min Stack	two stacks (032)
RandomizedSet	map + array
Time Based Key-Value	map + binary search
Design Hit Counter	queue
Peeking Iterator	one-item buffer (016)
Design Twitter	map + heap merge

⚠ COMMON MISTAKES

- RandomizedSet: deleting from the middle (O(n)) instead of swapping in the last item
- Updating one structure but not the other: the map points at stale indexes
- Nested iterator: doing all the flattening in the constructor when lazy is expected

✓ TIPS & BEST PRACTICES

- Write each operation's target Big O next to its signature
- Remove the last element **after** updating its index in the map
- Heavy traffic: keep per-second buckets, not every hit

QUICK SUMMARY

- map + array — O(1) random access
- sorted list + binary search — by timestamp
- stack of iterators — nested data
- queue — sliding time window

Number Theory & Digit Manipulation

GCD (Euclid) · LCM · Sieve of Eratosthenes · prime factorization · divisors to √n · n mod 10 / n div 10 · reverse integer

DIVISIBILITY, PRIMES, AND DIGITS

A few number-theory tools cover most interview math. **Euclid's algorithm** computes the GCD in O(log n). The **Sieve of Eratosthenes** lists every prime up to n in O(n log log n). **Trial division up to √n** factors a number or finds its divisors, because factors come in pairs `(d, n / d)`. Digits come off one at a time with `mod 10` and `div 10`.

QUICK FACTS

- `gcd(a, 0) = a`; `lcm(a, b) = a / gcd(a, b) · b`
- Divisors pair up: test `d` only while `d · d ≤ n`
- A number's digit count is $\lfloor \log_{10} n \rfloor + 1$
- Every integer > 1 has a unique prime factorization

GCD, LCM, SIEVE

```
function gcd(a, b)              // O(log min(a, b))
    while b ≠ 0
        (a, b) ← (b, a mod b)
    return a

function lcm(a, b)
    return a div gcd(a, b) * b // divide first

function sieve(n)               // primes ≤ n
    isPrime ← new Array(n + 1, true)
    (isPrime[0], isPrime[1]) ← (false, false)
    p ← 2
    while p * p ≤ n
        if isPrime[p]
            for m ← p * p to n step p
                isPrime[m] ← false
        p ← p + 1
    return isPrime
```

FACTORIZE, REVERSE INTEGER

```
function factorize(n)           // O(√n)
    out ← []
    d ← 2
    while d * d ≤ n
        while n mod d = 0
            out.append(d)
            n ← n div d
        d ← d + 1
    if n > 1 out.append(n)      // leftover prime
    return out

function reverseInt(x)          // 0 if it overflows int32
    (sign, x, rev) ← (sign(x), abs(x), 0)
    while x > 0
        d ← x mod 10            // last digit
        if rev > (2147483647 - d) div 10
            return 0
        rev ← rev * 10 + d
        x ← x div 10            // drop last digit
    return sign * rev
```

DIGIT TRICKS

GOAL	EXPRESSION
last digit	`n mod 10`
drop last digit	`n div 10`
append digit d	`n * 10 + d`
digit sum	loop `s += n mod 10; n ← n div 10`
digital root	`1 + (n - 1) mod 9` (n > 0)
palindrome number	reverse half and compare

PRACTICE

PROBLEM	TOOL
Count Primes	sieve
GCD of Strings	gcd of lengths
Ugly Number	divide out 2, 3, 5
Reverse Integer	digits + overflow check
Happy Number	digit squares + cycle check
Max Points on a Line	gcd-reduced slopes (100)

⚠ COMMON MISTAKES

- `lcm(a, b) = a * b div gcd`: `a * b` can overflow before dividing
- Sieve marking from `2p` instead of p^2: correct but slower
- Testing primality up to n instead of √n: O(n) per number

✓ TIPS & BEST PRACTICES

- Precompute a sieve once when many primality queries arrive
- Smallest-prime-factor sieve: factor any number ≤ n in O(log n)
- Use `abs` and track the sign separately for negative digit math

QUICK SUMMARY

- `gcd` — Euclid, O(log n)
- sieve — all primes ≤ n
- √n — trial division bound
- `mod 10` / `div 10` — peel digits

Sheet 097

Modular Arithmetic & Fast Exponentiation

mod 10^9+7 · (a·b) mod m · negative mod fix · binary exponentiation · modular inverse (Fermat) · overflow-safe multiply

ARITHMETIC THAT WRAPS AROUND

When answers are astronomically large, problems ask for them **modulo $10^9 + 7$**. Addition, subtraction, and multiplication can be reduced at every step, so numbers never overflow. **Division** can't: multiply by the **modular inverse** instead. **Binary exponentiation** computes `a^n mod m` in O(log n) by squaring.

WHY $10^9 + 7$?

- It's **prime**, so inverses exist (Fermat)
- It fits in a signed 32-bit int
- Product of two values < $10^9 + 7$ fits in 64 bits
- 998244353 is the other common prime

RULES, FAST POWER, INVERSE

```
MOD ← 10^9 + 7
(a + b) mod m = ((a mod m) + (b mod m)) mod m
(a * b) mod m = ((a mod m) * (b mod m)) mod m
(a - b) mod m = ((a - b) mod m + m) mod m        // keep it non-negative

function power(base, exp, m)                     // O(log exp)
    result ← 1
    base ← base mod m
    while exp > 0
        if exp & 1 = 1                           // this bit is set
            result ← result * base mod m
        base ← base * base mod m                 // square for the next bit
        exp ← exp >> 1
    return result

function inverse(a, m)                           // m prime, a mod m ≠ 0
    return power(a, m - 2, m)                    // Fermat's little theorem
// a / b  (mod m)  =  a * inverse(b, m) mod m
```

CODE EXPLAINED

- → reduce after every `+` and `*`
- → negative results: add `m` before the final `mod`
- → `exp & 1` — current binary digit of the exponent
- → squaring `base` steps through a^1, a^2, a^4, a^8 …
- → inverse needs a prime modulus (else extended Euclid)

POWER(3, 13, ∞): 13 = 1101_2

BIT	BASE	RESULT
1	3	3
0	9	3
1	81	243
1	6561	1,594,323 = 3^{13}

PRACTICE

PROBLEM	TOOL
`Pow(x, n)`	fast power (floats, negative n)
`Super Pow`	power with digit array exponent
`Count Good Numbers`	fast power mod p
`Unique Paths mod p`	nCr with inverses (098)
`Climbing Stairs (huge n)`	matrix fast power

⚠ COMMON MISTAKES

- Reducing only at the end: intermediate products already overflowed
- Dividing directly under a modulus: `(a / b) mod m ≠ (a mod m) / (b mod m)`
- Negative `mod` in Java/C/JS after subtraction

✓ TIPS & BEST PRACTICES

- Multiplying two 64-bit residues mod a 64-bit prime needs 128-bit math or mulmod
- Fibonacci(10^{18}) mod p: 2×2 matrix fast power
- Keep `MOD` as a constant to avoid typos like `10^9 + 9`

QUICK SUMMARY

- reduce every step — no overflow
- `((x mod m) + m) mod m` — non-negative
- fast power — O(log n) by squaring
- `power(a, m - 2)` — inverse for prime m

Combinatorics & Counting

nCr · Pascal's triangle · precomputed factorials · permutations vs combinations · Catalan numbers · stars and bars · pigeonhole

COUNTING WITHOUT LISTING

Combinatorics counts arrangements with formulas instead of enumerating them. **Permutations** count orderings, **combinations** count selections, and **nCr** shows up in grid paths, binomial expansions, and probability. For many `nCr mod p` queries, precompute factorials and inverse factorials once in O(n), then answer each in O(1).

RECOGNIZE IT WHEN

- "How many ways …" with no constraints between choices
- Grid paths with only right/down moves
- Answers like Catalan numbers (1, 1, 2, 5, 14, 42 …)
- Answer required modulo $10^9 + 7$

FACTORIALS AND NCR MOD P

```
fact[0] ← 1
for i ← 1 to N
    fact[i] ← fact[i − 1] * i mod MOD
invFact[N] ← power(fact[N], MOD − 2, MOD)
for i ← N down to 1
    invFact[i − 1] ← invFact[i] * i mod MOD

function nCr(n, r)                    // O(1)
    if r < 0 or r > n return 0
    return fact[n] * invFact[r] mod MOD
                   * invFact[n − r] mod MOD
```

PASCAL'S TRIANGLE (NO MODULUS NEEDED)

```
C ← (n+1) × (n+1) grid of 0
for i ← 0 to n
    C[i][0] ← 1
    for j ← 1 to i
        C[i][j] ← C[i−1][j−1] + C[i−1][j]
// C[i][j] = choose j of i
// row 4: 1 4 6 4 1
// O(n²) time, good for n ≤ ~1000
```

COUNTING FORMULAS

COUNT	FORMULA	EXAMPLE
orderings of n	n!	3 books: 6
ordered picks of r	n! / (n − r)!	gold/silver of 5: 20
unordered picks of r	C(n, r) = n! / (r!(n − r)!)	2 of 5: 10
picks with repetition	C(n + r − 1, r)	stars and bars
multiset orderings	n! / (a! b! …)	"AAB": 3
grid paths R × C	C(R + C − 2, R − 1)	3 × 7 grid: 28
subsets of n	2^n	4 items: 16

CATALAN NUMBERS

COUNTS	C_n
formula	C(2n, n) / (n + 1)
valid parentheses	n pairs
BST shapes	n nodes
polygon triangulations	n + 2 sides
first terms	1, 1, 2, 5, 14, 42

⚠ COMMON MISTAKES

- Computing `n!` directly for n > 20: overflows 64 bits
- Dividing factorials under a modulus instead of multiplying by inverse factorials
- Using combinations when order matters (or the reverse)

✓ TIPS & BEST PRACTICES

- Pigeonhole: n + 1 items in n boxes → some box holds two (duplicate-finding proofs)
- Stars and bars: distribute n identical items into k boxes = C(n + k − 1, k − 1)
- Practice: Pascal's Triangle, Unique Paths, Unique BSTs, Count Sorted Vowel Strings

QUICK SUMMARY

- n! — orderings
- C(n, r) — selections
- fact + invFact — O(1) nCr mod p
- Catalan — nested / tree shapes

Randomized Algorithms & Sampling

Fisher-Yates shuffle · reservoir sampling · weighted random pick (prefix + binary search) · rejection sampling · random pivot

USING RANDOMNESS CORRECTLY

Randomized algorithms rely on a uniform `randomInt(lo, hi)`. **Fisher-Yates** shuffles so every ordering is equally likely. **Reservoir sampling** picks uniformly from a stream of unknown length in O(1) memory. **Weighted picks** use prefix sums plus binary search. **Rejection sampling** builds one uniform range out of another by throwing away the leftovers.

QUICK FACTS

- `randomInt(lo, hi)` here includes both ends
- Random pivots make quicksort / quickselect O(n log n) / O(n) expected
- Seed the generator in tests for reproducible runs
- Correctness argument: show each outcome's probability is equal

SHUFFLE AND RESERVOIR SAMPLING

```
function shuffle(a)              // Fisher-Yates
    for i ← length(a) - 1 down to 1
        j ← randomInt(0, i)      // includes i
        swap(a, i, j)

function reservoir(stream)       // one item, O(1) memory
    (chosen, count) ← (null, 0)
    for each x in stream
        count ← count + 1
        if randomInt(1, count) = 1
            chosen ← x           // prob. 1 / count
    return chosen
// k items: keep the first k, then replace a
// random slot with probability k / count
```

WEIGHTED PICK AND REJECTION SAMPLING

```
class WeightedPicker             // Random Pick with Weight
    function init(w)
        this.prefix ← running sums of w
        // w = [1, 3] → prefix = [1, 4]
    function pick()
        total ← this.prefix[length(this.prefix) - 1]
        target ← randomInt(1, total)
        return lowerBound(this.prefix, target)

function rand10()                // from rand7()
    while true
        x ← (rand7() - 1) * 7 + rand7()   // 1..49
        if x ≤ 40                // reject 41..49
            return (x - 1) mod 10 + 1
```

PRACTICE

PROBLEM	TECHNIQUE
Shuffle an Array	Fisher-Yates
Linked List Random Node	reservoir sampling
Random Pick Index	reservoir over matches
Random Pick with Weight	prefix sums + binary search
Implement Rand10() Using Rand7()	rejection sampling
Insert Delete GetRandom O(1)	map + array (095)

BIASED VS UNBIASED

CODE	UNIFORM?
`j ← randomInt(0, i)`	yes
`j ← randomInt(0, n - 1)  each step`	no
`random() mod n`	slightly biased
`reject then retry`	yes

⚠ COMMON MISTAKES

- Shuffling by swapping with any index `0..n-1`: some orders become more likely
- Reservoir sampling with `randomInt(0, count)`: off by one, wrong probabilities
- Using `random() mod n` for large n: low values come up slightly more often

✓ TIPS & BEST PRACTICES

- Prove a sampler by checking the final probability of one item
- Expected loop count for rand10: 49/40 ≈ 1.2 iterations
- Randomized hash seeds defend against crafted collision attacks

QUICK SUMMARY

- Fisher-Yates — swap with `randomInt(0, i)`
- reservoir — keep with prob. 1 / count
- prefix + binary search — weighted pick
- rejection — discard and retry

Computational Geometry Basics

cross product orientation · Euclidean vs Manhattan distance · slope as reduced fraction · max points on a line · rectangle overlap · convex hull (monotone chain) · point in polygon

POINTS, LINES, AND SHAPES

Most geometry problems reduce to the **cross product**: its sign says whether three points turn left, turn right, or are collinear. Compare **squared** distances to avoid square roots, and store slopes as **reduced integer fractions** (via gcd) instead of floats. Sorting points plus cross products gives the **convex hull** in O(n log n).

EXACTNESS RULES

- Prefer integer math: no `/`, no `sqrt`, no float slopes
- Normalize slope signs so (1, -2) and (-1, 2) match
- Manhattan distance for grid moves; Euclidean for straight lines
- Touching edges: decide if they count as overlap

CROSS PRODUCT AND MAX POINTS ON A LINE

```
function cross(o, a, b)        // > 0 left · < 0 right · 0 collinear
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x)

function maxPoints(pts)                          // O(n²)
    best ← 1
    for i ← 0 to n - 1
        slopes ← new Map()
        for j ← i + 1 to n - 1
            (dx, dy) ← (pts[j].x - pts[i].x, pts[j].y - pts[i].y)
            g ← gcd(abs(dx), abs(dy))
            (dx, dy) ← (dx div g, dy div g)      // reduced fraction
            if dx < 0 or (dx = 0 and dy < 0)
                (dx, dy) ← (-dx, -dy)            // one sign convention
            slopes[(dx, dy)] ← slopes.getOrDefault((dx, dy), 0) + 1
            best ← max(best, slopes[(dx, dy)] + 1)
    return best
```

CONVEX HULL (MONOTONE CHAIN)

```
function convexHull(pts)          // O(n log n)
    sort(pts) by (x, y)
    function half(seq)
        h ← []
        for each p in seq         // k = length(h)
            while k ≥ 2 and cross(h[k-2], h[k-1], p) ≤ 0
                h.pop()           // not a left turn
            h.append(p)
        h.pop()                   // shared endpoint
        return h
    return half(pts) + half(reverse(pts))
```

RECTANGLES AND POLYGONS

```
function overlap(a, b)        // (x1, y1, x2, y2)
    w ← min(a.x2, b.x2) - max(a.x1, b.x1)
    h ← min(a.y2, b.y2) - max(a.y1, b.y1)
    return w > 0 and h > 0    // area = w * h

// point in polygon (ray casting):
// count edges crossed by a ray to
// the right of the point;
// odd → inside, even → outside
```

⚠ COMMON MISTAKES

- Storing slopes as floats: `1/3` and `2/6` may differ in the last bit
- Computing `sqrt` to compare distances: compare squared values instead
- Convex hull with `< 0` instead of `≤ 0`: keeps collinear points (decide which you want)

✓ TIPS & BEST PRACTICES

- Practice: Max Points on a Line, Rectangle Overlap, Rectangle Area, Erect the Fence, K Closest Points
- Use 64-bit ints for cross products of coordinates up to 10^9
- Rotate 45° (`u = x + y`, `v = x - y`) to turn Manhattan into Chebyshev distance

QUICK SUMMARY

- `cross(o, a, b)` — turn direction
- squared distance — no sqrt
- gcd-reduced slope — exact lines
- monotone chain — hull in O(n log n)

Linear-Time String Algorithms: KMP, Z & Manacher

LPS / failure array · KMP search · Z-array · pattern search O(n + m) · expand around center · Manacher O(n)

LINEAR-TIME STRING PREPROCESSING

Three algorithms build an array that captures repeated structure, so later steps never re-compare characters. **KMP**'s `lps` array says how far to fall back after a mismatch. The **Z-array** gives, for each position, how long it matches the string's prefix. **Manacher** gives every center's palindrome radius. Each runs in **O(n)**.

PICK ONE

- Find pattern in text: KMP or Z on `p + "$" + t`
- Repeated substring / shortest period: `n - lps[n - 1]`
- Longest palindromic substring: Manacher (or O(n²) expand-around-center)
- Shortest palindrome by adding a prefix: KMP on `s + "#" + rev(s)`

KMP: FAILURE (LPS) ARRAY

```
function buildLPS(p)
    lps ← new Array(m, 0)
    k ← 0          // current prefix length
    for i ← 1 to m - 1
        while k > 0 and p[i] ≠ p[k]
            k ← lps[k - 1]     // fall back
        if p[i] = p[k]
            k ← k + 1
        lps[i] ← k
    return lps
// "aabaaab" → [0,1,0,1,2,2,3]
```

KMP SEARCH

```
function kmpSearch(t, p)      // O(n + m)
    lps ← buildLPS(p)
    (k, out) ← (0, [])
    for i ← 0 to n - 1
        while k > 0 and t[i] ≠ p[k]
            k ← lps[k - 1]
        if t[i] = p[k]
            k ← k + 1
        if k = m
            out.append(i - m + 1)
            k ← lps[k - 1]    // keep going
    return out
```

Z-ARRAY

```
function zArray(s)
    z ← new Array(n, 0)
    (l, r) ← (0, 0)            // window [l, r)
    for i ← 1 to n - 1
        if i < r
            z[i] ← min(r - i, z[i - l])   // reuse
        while i + z[i] < n
                and s[z[i]] = s[i + z[i]]
            z[i] ← z[i] + 1
        if i + z[i] > r
            (l, r) ← (i, i + z[i])
    return z
```

MANACHER

```
function manacher(s)
    t ← "^#" + join(s, "#") + "#$"   // odd only
    p ← new Array(length(t), 0)      // radii
    (c, r) ← (0, 0)
    for i ← 1 to length(t) - 2
        if i < r
            p[i] ← min(r - i, p[2 * c - i])
        while t[i + p[i] + 1] = t[i - p[i] - 1]
            p[i] ← p[i] + 1
        if i + p[i] > r
            (c, r) ← (i, i + p[i])
    k ← index of max(p)
    return s[(k - p[k]) div 2 ..
             (k - p[k]) div 2 + p[k] - 1]
```

⚠ COMMON MISTAKES

- Resetting `k` to 0 after a mismatch: that's the naive O(n·m) search
- Z search without a separator: matches can run past the pattern into the text
- Manacher without sentinels `^` and `$`: the expansion walks off the ends

✓ TIPS & BEST PRACTICES

- Expand-around-center (O(n²), 2n - 1 centers) is usually enough in interviews
- Practice: Find the Index of First Occurrence, Repeated Substring Pattern, Longest Palindromic Substring, Shortest Palindrome
- `lps[i]` = longest proper prefix of `p[0..i]` that's also its suffix

QUICK SUMMARY

- `lps` — where to resume after a mismatch
- Z — prefix-match length at each i
- Manacher — palindrome radius per center
- all O(n)

Rolling Hash & Rabin-Karp

polynomial hash · base & mod · slide update · collision check · double hashing · longest duplicate substring (binary search + hash)

COMPARE SUBSTRINGS BY HASH

A **polynomial hash** turns a string into a number: $h = c_0 \cdot B^{m-1} + c_1 \cdot B^{m-2} + \ldots + c_{m-1} \pmod M$. Sliding the window one step removes the leading character's term and appends a new one in **O(1)**, so all windows of length m hash in O(n). Equal strings always have equal hashes; unequal ones rarely do, so **verify** on a match or use two moduli.

CHOOSING B AND M

- B: a prime larger than the alphabet (e.g. 131, 911382323)
- M: a large prime ($10^9 + 7$) or $2^{61} - 1$
- **Double hashing**: two (B, M) pairs, compare both
- Collision chance with one 10^9 modulus ≈ 1 in 10^9 per pair

RABIN-KARP SEARCH

```
function rabinKarp(t, p)
    m ← length(p)
    if m > n return []
    pw ← power(B, m - 1, M)                  // weight of the leading char
    (hp, ht) ← (0, 0)
    for i ← 0 to m - 1
        hp ← (hp * B + code(p[i])) mod M
        ht ← (ht * B + code(t[i])) mod M
    out ← []
    for i ← 0 to n - m
        if hp = ht and t[i..i+m-1] = p       // verify: hashes can collide
            out.append(i)
        if i + m < n                          // slide right by one
            ht ← (ht - code(t[i]) * pw mod M + M) mod M
            ht ← (ht * B + code(t[i + m])) mod M
    return out
```

CODE EXPLAINED

- → `pw` — B^{m-1}, the leading character's weight
- → build both hashes for the first window
- → remove left char, shift by B, add right char
- → `+ M` keeps the value non-negative
- → expected O(n + m)

PREFIX HASHES: ANY SUBSTRING IN O(1)

```
(h[0], pw[0]) ← (0, 1)
for i ← 0 to n - 1
    h[i + 1] ← (h[i] * B + code(s[i])) mod M
    pw[i + 1] ← pw[i] * B mod M

function hashOf(l, r)            // s[l..r]
    x ← h[r + 1] - h[l] * pw[r - l + 1]
    return ((x mod M) + M) mod M
```

LONGEST DUPLICATE SUBSTRING

```
function longestDup(s)           // binary search on L
    (lo, hi, best) ← (1, n - 1, "")
    while lo ≤ hi
        L ← (lo + hi) div 2
        start ← findDup(L)       // -1 if none:
        if start ≠ -1            // set of hashOf(i, i+L-1)
            best ← s[start .. start + L - 1]
            lo ← L + 1
        else
            hi ← L - 1
    return best
```

⚠ COMMON MISTAKES

- Skipping verification with a small modulus: collisions give false matches
- Forgetting `+ M` after subtraction: negative hashes never match
- Overflow in `ht * B` with 64-bit M: use $2^{61} - 1$ tricks or 128-bit products

✓ TIPS & BEST PRACTICES

- Binary search + hashing: any "longest repeated / common substring" question
- Practice: Find the Index of First Occurrence, Repeated DNA Sequences, Longest Duplicate Substring
- For exact guarantees use KMP / Z (Sheet 101) or a suffix array

QUICK SUMMARY

- polynomial hash — string → number
- rolling update — O(1) per slide
- prefix hashes — any substring in O(1)
- verify or double-hash — collisions

Sheet 103

Pattern Recognition Guide

"sorted" → binary search / two pointers · "contiguous" → window / prefix · "top k" → heap · "all combinations" → backtracking · "min steps" → BFS · "count ways" → DP

FROM PROBLEM WORDING TO TECHNIQUE

Scan the problem for **signals**: the shape of the input, the words in the question, and the size limits. Each row maps a signal to the pattern that usually solves it and the sheet that covers it. Read the constraints first; when two patterns fit, they usually break the tie (Sheet 026).

INPUT CLUES

INPUT LOOKS LIKE	TRY	SHEET
sorted array	binary search · two pointers	056 · 062
pair / sum to target	hash map · two pointers	035 · 062
contiguous subarray / substring	window · prefix sums · Kadane	063–066
linked list	fast/slow · reverse · dummy	031
binary tree / BST	DFS recursion · BFS · inorder	037–039
grid	flood fill DFS · BFS	048
edges / dependencies	BFS/DFS · topo sort · union-find	045–050
intervals	sort + scan · sweep line	069
stream of numbers	heap · two heaps	044
many words / prefixes	trie	091
values in 1…n	cyclic sort · index marking	071

QUESTION CLUES

QUESTION SAYS	TRY	SHEET
top / kth / k most	heap · quickselect	044 · 061
min steps (equal cost)	BFS	047
cheapest path	Dijkstra · Bellman-Ford	051–052
all subsets / combos / orders	backtracking	074–077
count ways · min cost · possible?	dynamic programming	078–088
next greater · brackets	monotonic stack · stack	067 · 032
max / min of each window	monotonic deque	068
connected groups	union-find · DFS	046 · 050
order with prerequisites	topological sort	049
minimize the max / maximize the min	binary search on answer	057
range queries + updates	Fenwick · segment tree	092–093

CONSTRAINT CLUES

MAX N	THINK
≤ 10	permutations, brute force
≤ 20	subsets, bitmask DP
≤ 500	$O(n^3)$ DP, Floyd-Warshall
≤ 5,000	$O(n^2)$ DP
≤ 10^5 – 10^6	O(n log n): sort, heap, binary search
≥ 10^9	O(log n) or math

WHEN STUCK

TRY	BECAUSE
sort the input	unlocks two pointers, greedy, binary search
add a hash map	turns a search loop into O(1)
think backward	from the goal, from the end of the array
solve a smaller case	reveals recursion / DP structure
fix one dimension	turns 2D problems into 1D
precompute	prefix sums, next-greater arrays

⚠ COMMON MISTAKES

- Matching on one keyword and ignoring the constraints
- Using BFS distances on weighted edges, or DP where greedy suffices
- Coding before confirming the chosen pattern fits a small example

✓ TIPS & BEST PRACTICES

- Underline input shape and question words, then try that pattern's template (104)
- Keep a log of problems you misclassified; it sharpens recognition fast
- Combine patterns: sort + two pointers, prefix sums + hash map

QUICK SUMMARY

- input shape → candidate structures
- question words → candidate patterns
- constraints → target complexity
- templates → Sheet 104

Algorithm Templates — Quick Reference

binary search · sliding window · BFS · DFS · backtracking · union-find · Dijkstra · DP memo

BINARY SEARCH (FIRST TRUE)

```
(lo, hi) ← (low, high)
while lo < hi
    mid ← lo + (hi - lo) div 2
    if ok(mid)
        hi ← mid
    else
        lo ← mid + 1
return lo
```

SLIDING WINDOW (LONGEST)

```
l ← 0
for r ← 0 to n - 1
    add(a[r])
    while invalid()
        remove(a[l])
        l ← l + 1
    best ← max(best, r - l + 1)
```

DP MEMO

```
memo ← new Map()
function f(s)
    if isBase(s) return base(s)
    if memo.has(s) return memo[s]
    memo[s] ← best of f(next)
              over each choice
    return memo[s]
```

BFS (SHORTEST STEPS)

```
queue ← new Queue([start])
seen ← new Set([start])
steps ← 0
while not queue.isEmpty()
    repeat length(queue) times
        u ← queue.dequeue()
        if u = goal return steps
        for each v in neighbors(u)
            if not seen.has(v)
                seen.add(v)
                queue.enqueue(v)
    steps ← steps + 1
return -1
```

DIJKSTRA

```
dist ← new Array(n, ∞)
dist[s] ← 0
heap ← new MinHeap([(0, s)])
while not heap.isEmpty()
    (d, u) ← heap.pop()
    if d > dist[u] continue
    for each (v, w) in adj[u]
        if d + w < dist[v]
            dist[v] ← d + w
            heap.push((dist[v], v))
return dist
```

BACKTRACKING

```
function bt(start, path)
    if done(path)
        out.append(copy(path))
        return
    for i ← start to n - 1
        if not valid(i) continue
        path.append(a[i])
        bt(i + 1, path)
        path.pop()
```

UNION-FIND

```
function find(x)
    while parent[x] ≠ x
        parent[x] ← parent[parent[x]]
        x ← parent[x]
    return x
function union(a, b)
    (ra, rb) ← (find(a), find(b))
    if ra = rb return false
    if size[ra] < size[rb]
        (ra, rb) ← (rb, ra)
    parent[rb] ← ra
    size[ra] ← size[ra] + size[rb]
    return true
```

DFS (COMPONENTS)

```
function dfs(u)
    seen.add(u)
    for each v in adj[u]
        if not seen.has(v)
            dfs(v)
count ← 0
for each u in nodes
    if not seen.has(u)
        dfs(u)
        count ← count + 1
```

⚠ COMMON MISTAKES

- Binary search: `hi ← mid` needs `lo < hi`
- BFS: mark seen when enqueuing

✓ TIPS & BEST PRACTICES

- Windows: record longest after shrinking
- Backtracking: copy the path

QUICK SUMMARY

- Details: 056 · 064 · 078 · 047
- 051 · 074 · 050 · 046